SMS++ — A User Manual

For SMS++ 0.7.0 (October 2026)

Antonio Frangioni

Contents

SMS++ — A User Manual **6**
Abstract . . . 6
Acknowledgements . . . 6
License . . . 6
Conventions . . . 7
Table of contents (high-level) . . . 7

1. Introduction **9**
1.1 What SMS++ is . . . 9
1.2 What SMS++ is not . . . 10
1.3 Audience and prerequisites . . . 10
1.4 How to read this manual . . . 11
1.5 Relation to the Doxygen reference and to the SMS++ Wiki . . . 11
1.6 A note on style . . . 12

2. Installation and first build **13**
2.1 Dependencies . . . 13
2.2 The umbrella project and its modules . . . 13
2.3 Two build systems, two use cases . . . 13
2.4 First smoke tests . . . 14
2.5 Troubleshooting . . . 14

3. The mental model **15**
3.1 Block: a sub-problem with semantics . . . 15
3.2 Solver: anything that can compute on a Block . . . 16
3.3 Two representations of the same model . . . 16
3.4 The lifecycle of a Block . . . 18
3.5 The three running examples . . . 18
3.6 How Part II reuses the three running examples . . . 19

4. Block **21**
4.1 Concept . . . 21
4.2 The identity rule . . . 21
4.3 Static versus dynamic . . . 22
4.4 Structure of the class . . . 22
4.5 Inline example: constructing an `MCFBlock` . . . 23
4.6 Common idioms . . . 24

5. Variable, Constraint, Objective **25**
5.1 Concept . . . 25
5.2 Variable . . . 25
5.3 Constraint . . . 26
5.4 Objective . . . 27
5.5 Inline example: the abstract representation of `BinaryKnapsackBlock` . . . 27
5.6 API outline . . . 28
5.7 Idioms . . . 29

6. Solver **30**
6.1 Concept . . . 30
6.2 Specialised versus general-purpose . . . 31
6.3 The `sol_type` return enum . . . 31
6.4 Parameters . . . 32
6.5 Asynchronous computation . . . 32
6.6 Feasibility and optimality checks . . . 33
6.7 Inline example: attaching an `MCFSolver` to an `MCFBlock` . . . 33
6.8 API outline . . . 34
6.9 Idioms . . . 34

7. Physical vs Abstract representation **36**
7.1 Two faces of the same model . . . 36
7.2 Why two representations . . . 36
7.3 On-demand construction of the abstract representation . . . 37
7.4 The lifetime of the abstract representation . . . 37
7.5 Feasibility and optimality checks across both faces . . . 37
7.6 Status — under development: the half-baked `Solution` . . . 38
7.7 Idioms . . . 38

8. Modification and the Janus discipline **40**
8.1 Concept . . . 40
8.2 The hierarchy of Modifications . . . 40
8.3 The two streams . . . 41
8.4 The Janus discipline . . . 41
8.5 The four `ModParam` values . . . 43
8.6 Channels and `GroupModification` . . . 44
8.7 The "anyone listening" filter . . . 45
8.8 Inline example: a cost change in `MCFBlock`, observed from both faces . . . 45
8.9 Idioms . . . 47
8.10 Forward reference to `Change` . . . 47

9. Solution **48**
9.1 Concept . . . 48
9.2 Whose `Block` a `Solution` belongs to . . . 48
9.3 Serialisation and combination . . . 48
9.4 The standard abstract `Solutions` . . . 49
9.5 The `Block`-specific physical `Solutions` . . . 49
9.6 Inline example: a CFL solution to netCDF and back . . . 49
9.7 Status — under development: closing the half-baked `Solution` . . . 50
9.8 API outline . . . 50
9.9 Idioms . . . 51

10. R3Block: Reformulation, Relaxation, Restriction **52**
10.1 Concept . . . 52
10.2 Why an R3 Block, and not a copy of the `Variables` . . . 52
10.3 The R3 Block API . . . 52
10.4 The trivial case: the copy . . . 54
10.5 The non-trivial case: CFL's MCF flow relaxation . . . 54
10.6 Keeping an R3 Block in sync: `UpdateSolver` . . . 54
10.7 Inline example: a CFL flow relaxation kept in sync . . . 55
10.8 Idioms . . . 55

11. Configuration **57**
11.1 Concept . . . 57
11.2 `SimpleConfiguration<T>` . . . 57
11.3 `BlockConfig` and the `[C/O/R]` family . . . 57
11.4 `BlockSolverConfig` and `RBlockSolverConfig` . . . 59
11.5 `ComputeConfig` . . . 59
11.6 The `f_extra_Configuration` escape hatch . . . 59

11.7 Loading and serialising . . . 59
11.8 Inline example: selecting a CFL formulation and a solver . . . 60
11.9 Idioms . . . 60

12. Sub-Block and the recursive flow of Modifications **61**
12.1 The recursive Block tree . . . 61
12.2 How a model is split across the tree . . . 61
12.3 The recursive flow of Modifications . . . 62
12.4 Locking propagates down the tree . . . 62
12.5 Inline example: CFL in the Knapsack Formulation . . . 62
12.6 Idioms . . . 63

13. The Function family **64**
13.1 The base `Function` . . . 64
13.2 `C05Function`: first-order information and linearization pools . . . 65
13.3 Modifications, directionality, and reoptimization . . . 65
13.4 `C15Function`: second-order information . . . 66
13.5 The "easy" leaf Functions . . . 66
13.6 Where the family points next . . . 67
13.7 Idioms . . . 67

14. LagBFunction and BendersBFunction **68**
14.1 Two hybrids, one pattern . . . 68
14.2 `LagBFunction`: the Lagrangian function of a `Block` . . . 68
14.3 `BendersBFunction`: the value function of a `Block` . . . 69
14.4 Why "almost any `Block`, almost any `Solver`" . . . 70
14.5 Idioms . . . 70

15. The methods factory **72**
15.1 The problem it solves . . . 72
15.2 Adapter functions over a base `Block*` . . . 72
15.3 The six standard signature families . . . 72
15.4 `register_method`, `get_method`, `get_method_name` . . . 73
15.5 Inline example: changing weights by name . . . 73
15.6 Relation to the class-name factories . . . 74
15.7 The dark side: registration versus the linker . . . 74
15.8 Idioms . . . 74

16. Change **76**
16.1 Concept . . . 76
16.2 Abstract and specific `Changes` . . . 76
16.3 Why `Change` exists . . . 77
16.4 The reference example: `BinaryKnapsackBlock`'s `Changes` . . . 77
16.5 `Change` versus `Modification`: which to use . . . 78

17. Parallel and asynchronous computation **79**
17.1 Locking a `Block` . . . 79
17.2 Asynchronous computation . . . 80
17.3 Solver `State` and checkpointing . . . 80
17.4 Inline example: a parallel Lagrangian dual . . . 80
17.5 Idioms . . . 81

18. Factories and netCDF serialisation **82**
18.1 The class-name factories . . . 82
18.2 The dark side, discharged: factories versus the linker . . . 82
18.3 Why netCDF . . . 83
18.4 The dark side of netCDF: groups are not free . . . 83
18.5 The serialisation contract . . . 84
18.6 Idioms . . . 84

Recipe R1 — Solving a Min-Cost Flow instance **85**
Goal . . . 85
Concepts used . . . 85
The code . . . 85
Walk-through . . . 86
Expected output . . . 87
Variations . . . 87

Recipe R2 — Reoptimizing a Binary Knapsack **88**
Goal . . . 88
Concepts used . . . 88
The code . . . 88
Walk-through . . . 89
Expected output . . . 89
Variations . . . 89

Recipe R3 — CFL three ways: cuts / MCF relaxation / Lagrangian **91**
Goal . . . 91
Concepts used . . . 91
The pattern . . . 91
The three configurations . . . 92
Walk-through . . . 92
Expected behaviour . . . 93
Variations . . . 93

Recipe R4 — CFL via Lagrangian decomposition with `PrimalProximalHeur` **94**
Goal . . . 94
Concepts used . . . 94
What the configuration sets up . . . 94
The setup, in code . . . 95
Walk-through . . . 95
From a bound to a feasible solution: `PrimalProximalHeur` . . . 96
Expected behaviour . . . 96
Variations . . . 96

Recipe R5 — CFL via Benders cuts with a user-cut callback **97**
Goal . . . 97
Concepts used . . . 97
What the configuration sets up . . . 97
How a solve proceeds . . . 97
Walk-through . . . 98
`BendersDecompositionSolver` . . . 98
Expected behaviour . . . 98
Variations . . . 98

Appendix A — Writing a new `:Block` **99**
A.1 The “physical” representation and a choice of “abstract” one . . . 99
A.2 The class skeleton and factory registration . . . 99
A.3 `load()` and the physical representation . . . 101
A.4 Generating the abstract representation . . . 101
A.5 A `chg_*()` method and its `Modification` . . . 103
A.6 `add_Modification()`: catching abstract changes . . . 103
A.7 `is_feasible()` on the physical representation . . . 105
A.8 Optional pieces (sketched) . . . 105
A.9 Checklist . . . 105

Appendix B — Writing a new `:Solver` **106**
B.1 `Solver` versus `CDASolver` . . . 106
B.2 The `compute()` contract . . . 106
B.3 Parameters . . . 106

B.4 Reacting to `Modification`s . . . 106
B.5 Optional: `compute_async` and `State` . . . 107
B.6 Worked example: a first-fit heuristic for `BinPackingBlock` . . . 107

Appendix C — Writing a new `:Modification` / `:Change` **110**
C.1 Why a `:Block` needs its own `:Modification`s . . . 110
C.2 Naming and structure conventions . . . 110
C.3 Registering with the factory . . . 111
C.4 Defining a `:Change` (optional) . . . 111
C.5 Checklist . . . 112

Appendix D — Concept-to-Block map **113**

Glossary **115**

Bibliography **118**
The framework . . . 118
Decomposition and nonsmooth optimization . . . 118
Serialisation . . . 118
Funding acknowledgement . . . 118

SMS++ — A User Manual

Antonio Frangioni *Dipartimento di Informatica, Università di Pisa* `frangio@di.unipi.it`

Version 0.1 — for SMS++ 0.7.0 (October 2026)

Abstract

SMS++ is an open-source C++-20 software framework for structured mathematical optimization. Its central abstraction is the `Block`, a nested unit that carries both the *semantics* of a mathematical sub-problem (its problem data, in natural form) and, on demand, an *abstract representation* of that sub-problem (its `Variable`s, `Constraint`s, `Objective`) suitable for general-purpose solvers. A `Block` can have any number of sub-`Block`, recursively, and any number of `Solver` attached. Specialised `Solver` exploit the structure of a specific `:Block`; general-purpose `Solver` operate on the abstract representation. The framework handles the lazy propagation of changes via the `Modification` (and, in the new beta `Change`) mechanism, the configuration of `Block` trees and `Solver` chains, the reformulation / relaxation / restriction of `Block`s through the `R3Block` mechanism, and coarse-grained parallel computation.

This manual is a self-contained companion to the SMS++ Doxygen reference and to the SMS++ Wiki. It introduces the concepts and idioms of SMS++ progressively, threads three running `Block` examples (`MCFBlock`, `BinaryKnapsackBlock`, `CapacitatedFacilityLocationBlock`) through the exposition, and collects a handful of end-to-end recipes that recombine the concepts into runnable programs.

Acknowledgements

The development of SMS++ would not have been possible without the contributions of many co-authors over the years, prominent among them Rafael Durbano Lobato, Donato Meoli, Enrico Gorgone, Federica Di Pasquale, Francesca Demelas, Kostas Tavlaridis-Gyparakis, Niccolò Iardella, and Benoit Tran. Specific contributions are credited in the source files and in the project's `CHANGELOG.md`.

Recent work on SMS++, including much of what is documented here, has been carried out in the framework of the **RESILIENT project**; the project is acknowledged in the form indicated in the umbrella project's `README.md`.

This manual was drafted with the assistance of Claude (Anthropic). The conceptual structure, technical content, and editorial choices are the responsibility of the author; the AI assistance is acknowledged here once and not repeated elsewhere in the text.

License

SMS++ is distributed under the GNU Lesser General Public License v3.0 (LGPL-3); this manual is distributed under the same terms.

Conventions

- **Class and method names** are typeset in `monospace`, e.g. `Block`, `add_Modification(...)`. Concrete classes derived from an abstract base are sometimes written `:Solver`, `:Block` to emphasise their derived nature, following the convention adopted in the slide decks that accompany the project.
- **File-path and source-tree references** are typeset in `monospace` too, e.g. `SMS++/include/Block.h`. Whenever a precise pointer into the source is given, the form is `file:line`.
- **Doxygen cross-references** are URLs to the public Doxygen at `https://smspp.gitlab.io/smspp-project/`. The Doxygen is generated with `CREATE_SUBDIRS = YES`, so each class page lives in a two-level hashed subdirectory; a typical URL has the form `https://smspp.gitlab.io/smspp-project/<xx>/<yy>/class_s_m_spp__di__unipi__it_1_1_<name>.html`, where `<xx>/<yy>` is a hash computed by Doxygen from the class name. If a cross-reference link should ever break (for instance because the Doxygen has been regenerated with different settings), the reader can navigate from https://smspp.gitlab.io/smspp-project/annotated.html, the alphabetical class index, to reach the page from a stable entry point. A handful of classes (notably `MCFBlock`) are documented as part of a Doxygen *group* rather than as a stand-alone class page; for these the link points to the header-file page (`_<header>_8h.html`), which is the canonical entry.
- **Wiki cross-references** are to `https://gitlab.com/smspp/smspp-project/-/wikis/home` and are written `[Wiki: page-name]`.
- **Mathematical notation** follows the conventions adopted in the slide decks. Decision variables and parameters are usually italicised; sets are uppercase calligraphic where needed for clarity.
- **Status admonitions** flag the maturity of a feature explicitly. Four forms appear in the text:
  - **Status — mature.** The feature is stable, tested, in `develop` and in tagged releases. Implicit unless stated.
  - **Status — beta.** The feature is in `develop` but its interface or semantics may change in incompatible ways.
  - **Status — under development.** The feature is partly present but is actively being reworked; the manual describes both what exists today and the direction of the intended revision.
  - **Status — planned.** The feature is documented as intended but is not yet released. The manual indicates explicitly that it cannot be used today.

When in doubt about a claim made in this manual, the authoritative sources are, in order of decreasing specificity: the public repo at https://gitlab.com/smspp/smspp-project); the public Doxygen reference; the SMS++ Wiki.

Table of contents (high-level)

- **Part I — Foundations.**
  - Chapter 1. Introduction.
  - Chapter 2. Installation and first build.
  - Chapter 3. The mental model.
- **Part II — Concepts and Idioms.**
  - Chapter 4. Block.
  - Chapter 5. Variable, Constraint, Objective.
  - Chapter 6. Solver.
  - Chapter 7. Physical vs Abstract representation.
  - Chapter 8. Modification and the Janus discipline.
  - Chapter 9. Solution.
  - Chapter 10. R3Block: Reformulation, Relaxation, Restriction.
  - Chapter 11. Configuration.
  - Chapter 12. Sub-Block and the recursive flow of Modifications.
  - Chapter 13. The Function family.
  - Chapter 14. LagBFunction and BendersBFunction.
  - Chapter 15. The methods factory.
  - Chapter 16. Change.
  - Chapter 17. Parallel and asynchronous computation.
  - Chapter 18. Factories and netCDF serialisation.
- **Part III — End-to-end Recipes.**
  - Recipe R1. Solving a Min-Cost Flow instance.

  - Recipe R2. Reoptimizing a Binary Knapsack.
  - Recipe R3. CFL three ways: cuts / MCF relaxation / Lagrangian.
  - Recipe R4. CFL via Lagrangian decomposition with `PrimalProximalHeur`.
  - Recipe R5. CFL via Benders cuts with a user-cut callback.
- **Part IV — Developer's Guide (Appendix).**
  - Appendix A. Writing a new `:Block`.
  - Appendix B. Writing a new `:Solver`.
  - Appendix C. Writing a new `:Modification` / `:Change`.

Bibliography, glossary, and index follow.

1. Introduction

1.1 What SMS++ is

SMS++ is a C++-20 software framework for *structured* mathematical optimization, designed primarily for researchers and practitioners who need to build, combine and solve mathematical models in which the problem structure is exploited algorithmically rather than ignored. The version covered by this manual is **0.7.0**, released in October 2026, and is distributed under the GNU Lesser General Public Licence v3.0.

The framework rests on a small number of abstractions, none of which is original in isolation: nested model decomposition, separation of problem semantics from problem syntax, lazy propagation of changes, algorithmic reformulation. What is, in our view, less common is that SMS++ pursues all of them at the same time, in a single coherent class hierarchy, and at a level of generality that does not assume a specific problem class.

The cornerstone of the framework is the abstract class `Block`, which represents "a (fragment of a) mathematical model with a well-understood semantic". A concrete `:Block` derived from it encodes a problem class with specific structure (a min-cost flow problem, a knapsack problem, a unit-commitment problem, a stochastic multi-stage program, an investment problem, ...). A `Block` can contain any number of `Variable`s, `Constraint`s and a single `Objective`, organised in *groups* (single, vectors, multi-dimensional arrays, or lists for the dynamic case). It can also contain any number of nested sub-`Block`, recursively, and it can in turn be a sub-`Block` of a father `Block`. Each `Block` admits two representations of the underlying mathematical model: a *physical* one, consisting of the problem data in its natural form (a graph, a table of weights and profits, a network of generation units...), and an *abstract* one, consisting of the explicit `Variable`s, `Constraint`s and `Objective` that a general-purpose solver would expect. The two representations coexist; the abstract one is built on demand.

A `Solver` is, in SMS++, anything that can compute on a `Block`. Any number of `Solver` can be attached to the same `Block`. A specialised `:Solver` written for a specific `:Block` reads the physical representation directly; a general-purpose `:Solver` reads the abstract representation; the two coexist with no compromise on either side. The interface is uniform: a `Solver` can return optimality, infeasibility, unboundedness, time-limited or iteration-limited termination; it can deliver any number of (approximately optimal, approximately feasible) solutions on demand; it can be invoked synchronously or asynchronously through `compute_async()`.

Changes to a `Block` propagate to all `Solver` listening to it (and to its ancestors) through the `Modification` mechanism: each `Solver` maintains a list of pending `Modification`s that it consumes when it is next invoked, and reacts to them as best it can (reoptimizing, if possible). The framework guarantees that a change made via either representation produces the appropriate `Modification` for the other representation too; we call this the **Janus discipline**, and it is one of the central ideas of SMS++ (Chapter 8). A new, complementary concept, `Change`, augments the `Modification` notification with the data needed to *apply* the change to a different copy of the `Block` and, if requested, to *undo* it. **Status — beta**: only a small number of `:Change` classes exist at version 0.7.0 (in particular for `BinaryKnapsackBlock` and `SATBlock`), and the interface is subject to change.

The framework supports *algorithmic reformulation* of a `Block` through the `R3Block` mechanism: a `Block` can produce another `Block` that represents an equivalent reformulation, a relaxation, or a restriction of itself, with explicit support for moving solutions and `Modification`s between the original and the reformulation (Chapter 10). It supports *configuration* through the recursive `BlockConfig` / `BlockSolverConfig` machinery (Chapter 11), and *coarse-grained parallel computation* through recursive locks, asynchronous compute methods, and Solver checkpointing (Chapter 17). It serialises `Block`s, `Solution`s, `Configuration`s and `Change`s to netCDF, a binary hierarchical file format that maps naturally onto the nested structure of a `Block` tree (Chapter 18).

Existing concrete `:Block` classes in the public modules of the project cover, among others, min-cost flow (`MCFBlock`), multicommodity flow and design (`MMCFBlock`), capacitated facility location (`CapacitatedFacilityLocationBlock`), single- and mixed-integer binary knapsack (`BinaryKnapsackBlock`), unit commitment in its

many flavours (`UCBlock`, `ThermalUnitBlock`, `BatteryUnitBlock`, `IntermittentUnitBlock`, `DCNetworkBlock`, `SlackUnitBlock`), stochastic multi-stage programs (`SDDPBlock`, `StochasticBlock`, `TwoStageStochasticBlock`), strategic investment (`InvestmentBlock`), and several others. Existing `:Solver` classes include wrappers to external libraries (`MCFSolver` wrapping `MCFClass`, `MCFLemonSolver` wrapping LEMON, the `MILPSolver` family wrapping CPLEX, Gurobi, HiGHS and SCIP, `SDDPSolver` wrapping StOpt), SMS++-native specialised solvers (`DPBinaryKnapsackSolver`, `ThermalUnitDPSolver`, `SDDPGreedySolver`), and SMS++-native generic structure-exploiting solvers (`LagrangianDualSolver` and its derived `PrimalProximalHeur`, `BundleSolver` and its parallel variant `ParallelBundleSolver`).

This manual focuses on the framework, not on the catalogue. Three `:Block` are used pervasively as worked examples — `MCFBlock`, `BinaryKnapsackBlock`, `CapacitatedFacilityLocationBlock` — chosen because together they exercise every major concept of SMS++ while remaining small enough to fit in a manual.

1.2 What SMS++ is not

It is useful to clarify a number of things that SMS++ *is not*, so that the reader can decide quickly whether the framework fits the task at hand.

SMS++ is **not an algebraic modelling language**. A `:Block` is C++ code, written explicitly by the author of that `:Block`. The framework does provide a number of modelling-language-style conveniences — most notably the `AbstractBlock` class, which reads `.mps` and `.lp` files and exposes them as a `Block` with a single, "syntactic" abstract representation — but this is a convenience entry point, not the canonical use of the framework. The canonical use is to write a `:Block` that captures the *structure* of a problem class, in C++, once, and then to reuse it across applications.

SMS++ is **not for the faint of heart**. The framework is written primarily for algorithmic experts; the audience of an algebraic modelling language — practitioners who write `min cx s.t. Ax` $\leq$ `b` and hand it to a solver — is *not* the audience of SMS++. End users may nonetheless benefit from the catalogue of pre-defined `:Block` and `:Solver`: a user who needs to solve a unit-commitment problem can load an instance into a `UCBlock` and attach a `:MILPSolver` without ever writing a new `:Block`. Even in this scenario, however, the framework's idioms and the role of configuration are not always intuitive, and reading at least Part I of this manual is recommended.

SMS++ is **not interface-stable**. Version 0.7.0 is the current public release; the framework is under active development and a non-negligible amount of further evolution is expected, including changes that may not preserve backward compatibility. Concrete `:Block` and `:Solver` are extensively tested in the project's test suite (`tests/` and `<Block>/test/` directories), but their public APIs may evolve. The manual flags **Status — beta** features explicitly; mature features have no such flag.

SMS++ is **not bundled with one solver**. The framework provides its own native specialised solvers, but it also relies on wrappers to a range of external libraries: CPLEX, Gurobi, HiGHS and SCIP (through the `MILPSolver` family); `MCFClass` and LEMON for min-cost flow; StOpt for stochastic dual dynamic programming. The user chooses which of these external dependencies to install (Chapter 2).

1.3 Audience and prerequisites

This manual addresses three audiences, listed in decreasing order of specificity.

The **primary** audience consists of *algorithm developers*: researchers, PhD students, and practitioners who want to implement a structure-exploiting solution method on top of an existing `Block`, or to extend the framework with a new `:Block`, a new `:Solver`, a new `:Modification` or a new `:Change`. For this audience, the manual is meant to fill the gap between the high-level slide decks that accompany the project and the method-by-method Doxygen reference: it explains the *idioms* and the *conventions* that make SMS++ a coherent whole.

A **secondary** audience consists of *Block authors*: researchers who need to express a new structured problem class as a `:Block` and expose it to the existing `:Solver` catalogue. This audience needs to understand the framework deeply enough to make the design decisions that come with a new `:Block` — what is "physical", what is "abstract", which `:Modification` to issue, what shape of `:Solution` to produce — and is well served by reading the entire manual, with particular attention to Part II (concepts) and Appendix A (writing a new `:Block`).

A **tertiary** audience consists of *sophisticated end users* who have a problem in hand, find an existing `:Block` that fits, and want to use it through one of the existing `:Solver`s. This audience can afford to read selectively:

Part I, the relevant `Recipe` in Part III, and the cross-referenced Doxygen pages for the specific `:Block` and `:Solver` of interest.

The minimum technical prerequisites are: working knowledge of **C++-20** (concepts, ranges, smart pointers, lambdas, range-based for); familiarity with **CMake** and, for the developer-loop workflow, with classical **Make**; a working installation of a C++-20 compiler (recent `gcc`, `clang`, or MSVC); and at least an operational understanding of the optimization paradigms that the `:Block` of interest implicates. For Recipes R4 and R5 in Part III, basic familiarity with Lagrangian and Benders decomposition is assumed.

1.4 How to read this manual

The manual is organised in four parts plus front matter and back matter; the rationale for the architecture is given here so that the reader can plan the reading sequence that best suits their needs.

Part I — Foundations (Chapters 1–3) sets the stage. Chapter 1 is this introduction. Chapter 2 covers installation and the first build; it is deliberately short because the SMS++ Wiki and the umbrella project's `README.md` are the authoritative source for the detailed steps. Chapter 3 is the *mental model* chapter: it introduces the high-level picture of `Block`, `Solver`, the two representations, the lifecycle of a `Block`, and the three running examples. A reader who reads only Part I should leave with a coherent picture of what SMS++ is, even if not yet able to write code against it.

Part II — Concepts and Idioms (Chapters 4–18) is the core of the manual. Each chapter introduces one concept, defines it precisely, locates it in the source tree and the Doxygen reference, and illustrates it with a short, compilable snippet that uses one of the three running examples. The order of the chapters is chosen so that *every concept is introduced before it is used*: the dependency graph of the concepts has been linearised. Each Part II chapter is short (typically two to four pages typeset) and self-contained.

Part III — End-to-end Recipes (R1–R5) collects five complete, runnable programs that recombine the concepts introduced in Part II into solutions to recognisable scenarios: solve an MCF instance; reoptimize a knapsack; solve a CFL problem three different ways (MILP, MCF relaxation, Lagrangian); solve a CFL problem with `LagrangianDualSolver` plus `PrimalProximalHeur`; solve a CFL problem with Benders cuts emitted via a user-cut callback. Each recipe is three to five pages, lists the concepts used, gives the full code, walks through it line by line, and reports expected output.

Part IV — Developer's Guide (Appendices A–C) is the manual for *extending* the framework: writing a new `:Block`, a new `:Solver`, a new `:Modification` or a new `:Change`. Appendix A develops a small but complete `BinPackingBlock` from scratch as a pedagogical example.

A pragmatic reader who already knows the concepts and just wants a working program may read Part I and jump directly to the relevant Recipe in Part III; the recipe will name the concepts it uses, and the reader can drill back into Part II only as needed. Conversely, a reader who wants a complete picture should read in order, and use the Recipes as the runnable counterparts of the concepts.

1.5 Relation to the Doxygen reference and to the SMS++ Wiki

This manual is one of three sources of documentation for SMS++, each with a specific role.

The **Doxygen reference**, at https://smspp.gitlab.io/smspp-project/, is the authoritative *API reference*. Every public class, every public method, every member field is documented there, with its complete signature, its parameters, its return values, its preconditions and postconditions, its exceptions, and (for the better-documented classes) inline mathematical notes about what it computes. The Doxygen pages are generated from the inline comments in the source files, which are kept in sync with the code as it evolves. The canonical way to *look up a specific method* is to consult Doxygen; the manual you are reading does not duplicate that information. Throughout the manual, when a class or method is introduced, the text indicates the corresponding Doxygen URL.

The **SMS++ Wiki**, at https://gitlab.com/smspp/smspp-project/-/wikis/home, is the authoritative source for *installation, build, customisation and troubleshooting* instructions: how to fetch the umbrella project and its sub-modules, how to configure CMake, how to enable optional external solver interfaces, how to deal with platform-specific issues. Chapter 2 of this manual gives a brief overview of the build system; for the details, the Wiki is the reference.

This **manual** is the third complement. Its scope is the *concepts and idioms* of SMS++ and a collection of *end-to-end recipes*. It explains why the framework is the way it is, how the abstractions fit together, what the recurring idioms are, and what a few representative end-to-end uses look like. It complements, but does not substitute for, either the Doxygen reference or the Wiki.

When the three sources of documentation disagree, the source code is the ultimate authority, the Doxygen reference comes second, and this manual comes third. We have tried to keep all three consistent at the version covered (0.7.0), but in case of discrepancy the reader is invited to file an issue against the umbrella project.

1.6 A note on style

A few stylistic choices that recur throughout the manual are worth stating upfront.

We use **understatement**. SMS++ is a research-grade software framework with definite strengths, but also with definite limitations that we acknowledge openly. Where a feature is in beta or planned but not yet released, the corresponding **Status** admonition appears explicitly. Where SMS++ does not yet support a use case that one might reasonably expect, we say so.

We **avoid comparing SMS++ with other modelling systems**. There is plenty of useful comparative work in the literature; the manual, however, is about SMS++ on its own terms. A reader who wants to position SMS++ within the broader landscape of modelling and optimization frameworks will find that material elsewhere.

We **prefer code to abstract description** wherever a short snippet clarifies more than a paragraph of English. Every snippet in the manual is written against the SMS++ source tree of version 0.7.0.

We **cite precisely**. Whenever we make a claim about how a class behaves, we point to either its Doxygen page or a `file:line` in the source. The reader who wants to verify any claim should be able to do so by following the citation.

We **mark beta and planned features explicitly**. The reader should never have to guess whether a feature can be relied upon in a production codebase; the manual will say so.

With these preliminaries out of the way, Chapter 2 turns to the practical question of how to fetch SMS++ and build it.

2. Installation and first build

This chapter is deliberately short. The SMS++ Wiki and the umbrella project's `README.md` are the authoritative, always-current source for the detailed installation steps; what follows is an orientation, plus the one idea — the two coexisting build systems — that a newcomer most needs to understand before consulting them.

2.1 Dependencies

The core SMS++ library requires a C++-20 compiler (a recent `g++`, `clang++`, or MSVC), Boost, the C++ interface to netCDF (`netCDF-cxx4`), and Eigen.

Beyond the core, individual modules pull in optional dependencies, each of which can be omitted if the corresponding functionality is not needed:

- the `MILPSolver` family wraps the MIP back-ends CPLEX, Gurobi, SCIP and HiGHS (the last two open-source);
- `MCFSolver` wraps the legacy MCFClass algorithms, `MCFLemonSolver` wraps LEMON;
- `SDDPSolver` wraps the SDDP engine of the StOpt project.

For a first installation, the project provides "zero-waste" scripts that fetch and install the dependencies in default locations: `INSTALL.sh` (Linux / macOS) and `INSTALL.ps1` (Windows), each accepting `--without-<dependency>` flags (`--without-cplex`, `--without-gurobi`, `--without-scip`, `--without-highs`, `--without-stopt`, `--without-lemon`, `--without-smspp`) to skip the parts not wanted. The exact invocations are in the project `README.md`; the Wiki's requirements guide has the detail.

2.2 The umbrella project and its modules

SMS++ is distributed as an *umbrella* project, `smspp-project`, whose role is to provide a one-stop way to download and build all the related modules together (and to produce a unified Doxygen and track cross-module issues). At any time it points to the latest releases of: the core library (`smspp`), the leaf and composite `:Block`s (`MCFBlock`, `BinaryKnapsackBlock`, `CapacitatedFacilityLocationBlock`, `MMCFBlock`, `UCBlock`, `SDDPBlock`, `StochasticBlock`, `TwoStageStochasticBlock`, `InvestmentBlock`, ...), the `:Solvers` (`MILPSolver`, `LagrangianDualSolver`, `BundleSolver`, ...), and the shared `tests` and `tools` repositories. Cloning the umbrella (with its sub-modules) is the recommended way to obtain a coherent set.

2.3 Two build systems, two use cases

SMS++ can be built either with CMake or with plain makefiles, and the choice is not arbitrary — the two suit different workflows.

CMake — for "fire and forget". CMake builds *off-source* (the artefacts go into a separate build tree). This makes it the better choice for a one-off "build, install, and forget" installation: the developer who simply wants SMS++ available to link against from their own project configures once, builds, installs, and never thinks about it again. The standard interactive front-end `ccmake` makes the configuration step pleasant: it presents the available options — which optional solver interfaces to enable, where dependencies live, debug vs release — as a menu to toggle, without hand-editing any file. Several configuration options are documented in the Wiki's customisation page.

Makefiles — for the developer loop. Every module, starting with the core, also ships hand-crafted makefiles that build *on-source* (the artefacts sit next to the source). The on-source build is much better suited to the

tight "edit, compile, test" loop of someone *developing* SMS++ components, because the build products are right there and the loop is short. The core library is built by `SMS++/lib/makefile-lib` into `lib/libSMS++.a`; an executable's own makefile then declares which modules it needs.

The module-dependency mechanism in the makefiles rests on two include files that *every* module provides:

- `makefile-c` — the "complete" include: it pulls in the module's own headers and compilation rules, *all the modules it depends on* (which by definition includes the core), and all the external libraries.
- `makefile-s` — the "sub" include: the same, but *without* the core, so that the core is not pulled in more than once.

The rule an executable's makefile follows is therefore simple: **include exactly one `makefile-c`** — the one for the module that brings in the core — **and include every other module through its `makefile-s`.** This keeps the core's headers and objects in the build exactly once while letting an executable compose any set of modules. The existing testers (`MCFClassSolver/test`, `tests/CapacitatedFacilityLocation`, ...) are the reference examples to copy from.

In both build systems, external dependencies are found automatically if they sit in their OS-default locations; if not, the supported way to point at them is to copy the right `extlib/makefile-default-paths-<os>` file to `extlib/makefile-paths` (which is `.gitignore`-d, so local settings never leak into the repository) and set the `*_ROOT` values there. This single file is read by both CMake and the makefiles.

2.4 First smoke tests

A point worth understanding here is *why the tests are organised the way they are.* A `:Block` on its own is, in general, not very testable: without an appropriate `:Solver` one cannot even `compute()` it, and it would be wrong to make a `:Solver` a *prerequisite* of a `:Block` (the whole point of the framework is that `:Block`s and `:Solver`s are independent). This is precisely why `tests/` is a **separate repository**: the testers that need both a `:Block` and a `:Solver` from different modules live there, not inside any one module. Among the running examples, `tests/CapacitatedFacilityLocation` is the one used by Recipes R3–R5.

Some modules, however, *do* carry their own tester under a local `test/` directory — namely those that bundle a `:Solver` and so are self-contained: `MCFClassSolver` (whose `MCFSolver` solves an `MCFBlock`), `BinaryKnapsackBlock` (with `DPBinaryKnapsackSolver`), `CapacitatedFacilityLocationBlock`, and `SDDPBlock`. Building and running one of these is the quickest way to confirm that the toolchain, the dependencies and the paths are all in order. The `MCFClassSolver/test` tester — the runnable counterpart of Recipe R1 — is a good first target: it builds the core, the `MCFBlock` module and the `MCFSolver`, loads a small flow instance, solves it, and checks the result.

2.5 Troubleshooting

Two failure modes are common enough to flag here, both covered in depth elsewhere in this manual or in the Wiki:

- a runtime error of the form *"XXXX not present in YYYY factory"*, despite the class being in the source tree, is almost always the linker having optimised away a translation unit whose only job was to register a class; §18.2 explains the cause and the fixes (linker flags or `SMSpp_ensure_load`).
- a dependency that the build cannot find is almost always a `*_ROOT` path issue; copy `extlib/makefile-default-paths-<os>` to `extlib/makefile-paths` and correct the relevant value, as in §2.3.

For anything beyond these, the SMS++ Wiki is the reference; it carries the full, platform-specific installation and troubleshooting guides that this chapter deliberately does not duplicate.

3. The mental model

Before turning to the precise definitions of Part II, this chapter gives a high-level picture of the SMS++ framework. Its goal is to build the reader's intuition: what a `Block` is for, what role a `Solver` plays, why two representations of the same model coexist, how a `Block` typically lives across its lifetime, and how the three running examples used throughout the manual fit into this picture. None of what follows is rigorous in the sense of Part II; everything will be made precise there.

3.1 Block: a sub-problem with semantics

The cornerstone of SMS++ is the abstract class `Block`. The intuition behind it is captured by a single sentence: *a `Block` is a fragment of a mathematical model with a well-understood semantic.* "Fragment" because a `Block` is rarely the whole model; it is more often one of several pieces that compose into a larger model. "Semantic" because what a `Block` represents is not a list of variables and constraints in some general-purpose syntax, but a specific *kind* of problem: a min-cost flow, a knapsack, a unit-commitment, a two-stage stochastic program. The kind is encoded by the concrete `:Block` class derived from the abstract `Block`.

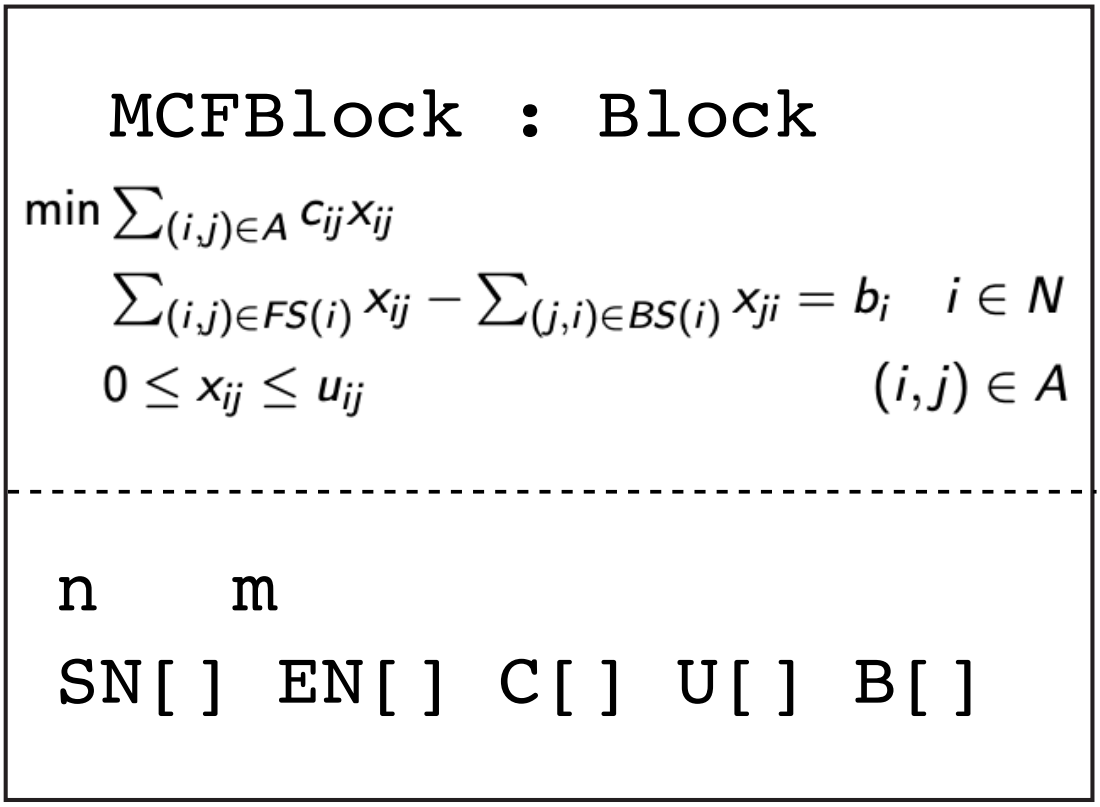


Figure 1: A `Block` carries the semantics of a mathematical sub-problem.

A concrete `:Block` knows its problem class. It knows what the data of an instance looks like (a graph, a table of weights, a network of generation units), how to load it from a text or binary file, how to serialise it back, what makes a candidate solution feasible for it, what the optimal value of the problem is conceptually, and what algorithmic operations (changes to the data, reformulations, copies) are meaningful for it. A user of the

framework who wants to *solve* an instance of that problem class typically does not write a new `:Block`; the existing `:Block` of the relevant class is used and configured. A user who wants to *introduce a new problem class* writes a new `:Block`; this is the topic of Appendix A.

3.2 Solver: anything that can compute on a Block

Once a `Block` carries an instance, the framework needs an algorithm to compute on it. In SMS++ the role is played by `Solver`, an abstract class that represents "anything that can compute on a `Block`". A concrete `:Solver` is one specific algorithm: a simplex implementation for min-cost flow problems, a dynamic-programming procedure for binary knapsack, a state-of-the-art commercial MIP solver, a Lagrangian dual driver, a bundle method, a stochastic dual dynamic programming engine.

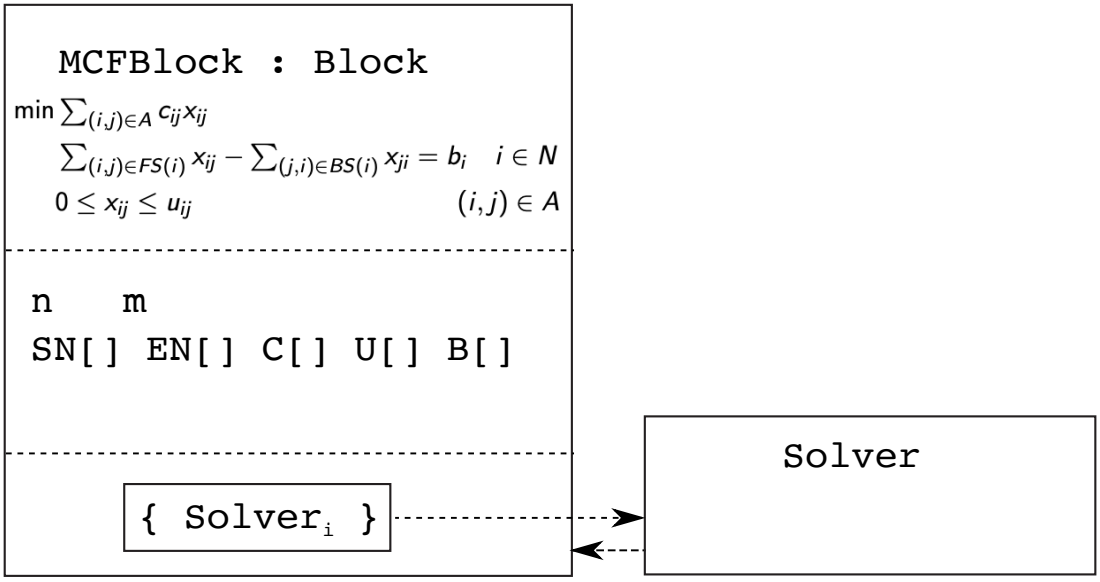


Figure 2: Any number of `:Solver`s may attach to a `Block`; changes flow to them as `Modification`s.

Any number of `:Solver` can be attached to the same `Block` at the same time. A specialised `:Solver` is one written for a specific `:Block` class: it knows the problem and can exploit the structure; the price is that it works only for that one `:Block`. A general-purpose `:Solver` knows nothing about the specific structure but can handle a large family of `:Block` through the abstract representation (Section 3.3). The two coexist: the user picks, at configuration time, which `:Solver` is attached to a given `:Block`. Switching from one to the other is usually a one-line change to a configuration file (Chapter 11), with no change to user code.

Any concrete `:Solver` exposes a uniform interface, regardless of how it works internally. It accepts integer / double / string parameters; it can be told to stop after a time limit, an iteration limit, or a target accuracy; it can be invoked synchronously or asynchronously; when it terminates it reports a return code (optimal, infeasible, unbounded, time limit, iteration limit, error, ...); on demand it yields one or more solutions of the problem, primal and (where it makes sense) dual. The `Solver` interface, in this sense, is the *lingua franca* of SMS++.

3.3 Two representations of the same model

A `Block` represents a mathematical model. SMS++ allows the same `Block` to expose two distinct representations of that model, which coexist and refer to the same underlying problem.

The first is the **physical** representation, sometimes called the *semantic* representation: the problem data in its natural form. For a min-cost flow problem this is a directed graph plus arc costs and capacities plus node deficits, stored as compact arrays. For a knapsack problem it is a capacity, a vector of weights, a vector of profits. For a unit-commitment problem it is a network, a set of generation units each described by its own technological parameters, a demand profile. The physical representation is the form a domain expert would write down on a whiteboard.

The second is the **abstract** representation: an explicit list of `Variable` objects, an explicit list of `Constraint` objects, an explicit `Objective` object — that is, the model in the form a general-purpose solver would

expect. For the same min-cost flow problem this is a vector of flow variables, a vector of flow-conservation row constraints, a linear objective. The abstract representation is what the framework hands over when a general-purpose `:Solver` (an LP solver, a MIP solver) is attached.

```
physical                    abstract
data                        Variable/Constraint/Objective
-------                     ---------------
graph G                     x[i,j] for arc (i,j)
cost C[i,j]                 sum x[i,j] - sum x[j,i] = b[i]
capacity U[i,j]             0 <= x[i,j] <= U[i,j]
deficit b[i]                min sum C[i,j] x[i,j]
```

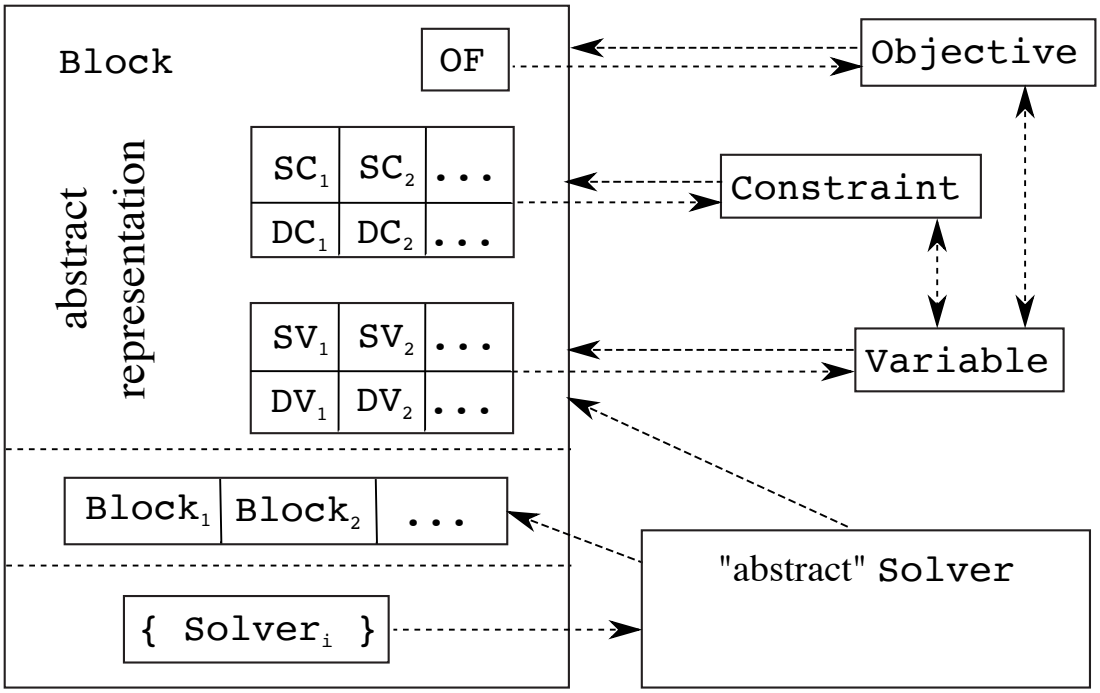


Figure 3: The two faces of a `Block`: the physical (semantic) data and the abstract `Variable` / `Constraint` / `Objective` representation.

Why two representations? Because a specialised `:Solver` does not need the explicit `Variable` and `Constraint` : it knows the problem and can read the physical data directly, much faster and at much lower memory cost. A general-purpose `:Solver`, on the other hand, *does* need the explicit `Variable` and `Constraint`: it does not know what the problem is and has no choice but to consume the model in its abstract form. Having only one of the two would force a compromise; having both is the simplest design that lets the user keep both kinds of `:Solver` on the same `Block` without paying the abstract-representation cost when only a specialised `:Solver` is in use.

The abstract representation is built **on demand**. The `Constraints` and the `Objective` are constructed only when a `:Solver` that needs them is attached. The methods that build them are `generate_abstract_variables()`, `generate_abstract_constraints()`, `generate_dynamic_variables()`, `generate_dynamic_constraints()`, `generate_objective()`. Each accepts a `Configuration` that gates optional parts of the construction. Chapter 7 makes this precise.

> **Status — under development.** In the current implementation the `Variables` of the abstract representation are an exception to the "on demand" rule: they are always materialised, because every `:Solver` reports its solution by *writing into* the `Variables` of the `Block`. This means that even a `:Block` solved exclusively by a specialised `:Solver` that reads only the physical representation still has to construct the abstract `Variables`, just so that the `:Solver` has somewhere to write the solution. The framework framing — "physical and abstract coexist, neither is privileged" — is therefore not, today, a faithful description of the implementation: at present the abstract `Variables` are privileged and unavoidable.
>
> The intended resolution, expected in a future revision, is to let a `:Solver` return a `Solution` object directly and to let the `:Block` *adopt* that `Solution` instead of writing it into the abstract `Variable`

s. Once that mechanism is in place, a `:Block` attached to a specialised `:Solver` only will be able to skip constructing the abstract `Variable`s altogether. This is one of several examples in the manual where the API is not yet fully settled (see also Chapter 16 on `Change`).

Keeping the two representations in sync, when one is modified through one face and a `:Solver` is reading the other, is non-trivial and is the topic of Chapter 8 (the *Janus discipline*).

3.4 The lifecycle of a Block

A `Block` typically goes through five stages along its lifetime: *build*, *configure*, *attach*, *solve*, *modify*. After the first *solve* it loops over *modify* and *solve* zero or more times. The last two stages — *modify* and *re-solve* — are not an afterthought; they are explicit design objectives, and a good deal of the framework's internal machinery exists to make them efficient.

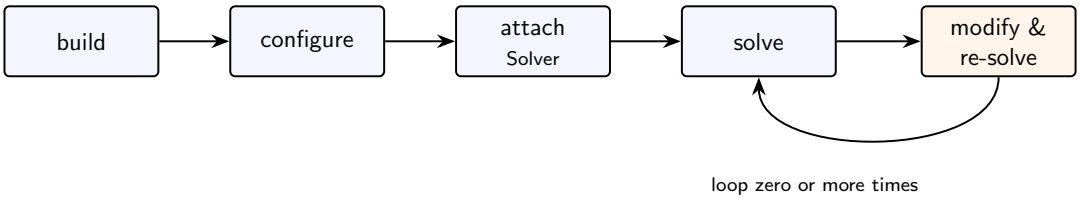


Figure 4: The lifecycle of a `Block`: build, configure, attach a `:Solver`, solve, then loop over modify and re-solve.

- **Build.** The `Block` is constructed and populated from data. The data may come from in-memory arrays (typically passed to a `:Block`-specific `load(...)` overload), from a text file (passed to `load(std::istream&)`), or from a netCDF group (passed to `deserialize(netCDF::NcGroup)`). Of these three routes, only the netCDF one is supported uniformly across the `:Block` catalogue; the in-memory and text-file overloads exist only for those `:Block` classes (such as `MCFBlock` and `MMCFBlock`) that have a historically established input format and for which it made sense to support it. For most other `:Block` classes the canonical route is netCDF (plus, of course, programmatic construction by the application code that owns the `:Block`).
- **Configure.** The optional behaviours of the `Block` (which parts of the abstract representation to generate, with which options; which `:Solver` to attach; which parameters to feed them) are set via a `Configuration` tree that mirrors the `Block` tree. The configuration can be loaded from a text file or from netCDF.
- **Attach.** One or more `:Solver` are registered with the `Block`. Registration is a constant-time operation; the `Solver` does not consume the `Block` data until its `compute()` method is called.
- **Solve.** A registered `:Solver` is asked to `compute()`. It consumes the `Block` data (physical or abstract, as appropriate), runs its algorithm, and produces a return code together with a pool of solutions and a pool of pending `Modification`s emptied on entry.
- **Modify and re-solve.** The user changes the data of the `Block` through one of the documented mutation methods. Each mutation produces zero or more `Modification` objects, which are dispatched to every `:Solver` currently attached to the `Block` or to any of its ancestor `Block`s. The next call to `compute()` on any of them starts with that list of pending `Modification`s; if the `:Solver` is able to reoptimize, it does so.

3.5 The three running examples

Three concrete `:Block` are used throughout the manual to make the concepts tangible. Each is introduced briefly here and revisited in Part II.

`MCFBlock` — a leaf Block with a wrapper Solver

`MCFBlock` represents a linear min-cost flow problem on a directed graph. It is a *leaf* `Block` (no sub-`Block`) with a small, well-defined physical representation: arrays of arc start and end nodes, arc costs and capacities, node deficits, plus a few counters for static / dynamic arcs and nodes. It supports a partly dynamic graph (a static initial set of nodes and arcs, plus up to a configurable maximum number of dynamic nodes and arcs that can be added or deleted at runtime).

`MCFBlock` is paired with the templated specialised `:Solver` `MCFSolver< MCFC >`, which wraps any concrete algorithm from the legacy `MCFClass` project. The template parameter `MCFC` lets the user choose between simplex,

relaxation, cost-scaling and other classical algorithms. This is the SMS++ example of a *wrapper* specialised `:Solver`: the implementation lives in an external library that predates SMS++ by years and continues to be developed in its own right; SMS++ exposes it through the `:Solver` interface.

Through `MCFBlock` the manual will introduce: the basics of a leaf `Block`, the construction of the abstract representation, the two parallel streams of `Modifications`, `is_feasible(useabstract)` and `is_optimal(useabstract)` written on both representations, the trivial copy `R3Block`, and the `MCFSolution` class.

`BinaryKnapsackBlock` — a leaf Block with a native Solver

`BinaryKnapsackBlock` represents a mixed-integer knapsack problem (despite the "Binary" in its name, the boolean integrality vector lets each variable be declared either binary or continuous). It is again a *leaf* `Block`, even smaller than `MCFBlock`: the physical data is a capacity, a vector of weights, a vector of profits, and a boolean integrality vector.

It is paired with a *native* specialised `:Solver`, `DPBinaryKnapsackSolver`, implemented entirely inside SMS++. The implementation is a classical dynamic programming procedure on the integer part of the problem combined with an exact greedy algorithm for the continuous part. The algorithm requires integer weights (any non-integer weight raises an exception); capacities and profits can be `double`. This is the SMS++ example of a *natively-implemented* specialised `:Solver`.

`BinaryKnapsackBlock` is also the reference example of the new beta `Change` family (`BinaryKnapsackBlockChange`, plus the `Rngd` and `Sbst` variants). Through it the manual will introduce: the methods factory, the `Solution` class as a Block-specific object, and the `Change` concept that complements `Modification` (Chapter 16).

`CapacitatedFacilityLocationBlock` — a non-leaf Block with multiple formulations

`CapacitatedFacilityLocationBlock` represents the basic capacitated facility location problem (also known as capacitated warehouse location). It is explicitly described in its own source comments as a "didactic" `Block`, designed to showcase several SMS++ features at once.

`CapacitatedFacilityLocationBlock` supports four formulations of the same problem, selected via a single integer in its `f_static_variables_Configuration`:

- the **Natural Formulation** (StdForm, the standard MILP) has no sub-`Block`, and the abstract representation contains the design variables `Y[i]`, the transportation variables `X[i,j]`, the customer satisfaction constraints, the facility capacity constraints, and a linear objective;
- the **Knapsack Formulation** (KskForm) makes the `Block` grow one sub-`BinaryKnapsackBlock` per facility (each carrying the facility's capacity constraint and the transportation/design variables for that facility); the master `Block` carries the customer satisfaction linking constraints;
- the **Benders Formulation** (BenForm) has only the design variables `Y[i]` plus a single continuous epigraphic variable `v` in the master, and the transportation sub-problem is hidden inside a `BendersBFunction` wrapping an `MCFBlock`. Two sub-variants are available, one with slack arcs in the inner `MCFBlock` (the sub-problem is then always feasible, only optimality cuts are emitted) and one without (feasibility cuts are emitted via the vertical-linearization machinery when the inner sub-problem is infeasible).

In addition, `CapacitatedFacilityLocationBlock` produces a non-trivial `R3Block`: besides the trivial copy, it can produce an `MCFBlock` that represents the *continuous flow relaxation* of CFL. The same construction is used internally by the BenForm formulation; externally it can be used as a fast lower-bound producer.

Through `CapacitatedFacilityLocationBlock` the manual will introduce: sub-`Block` and the recursive flow of `Modifications`, multiple formulations selected by `Configuration`, the non-trivial `R3Block`, `LagBFunction` and `BendersBFunction` as the two `Function`+`Block` hybrids that wrap an inner `Block`, the `LagrangianDualSolver` and the derived `PrimalProximalHeur`, and the user-cut callback pattern of `MILPSolver` for Benders cuts.

3.6 How Part II reuses the three running examples

Part II — *Concepts and Idioms* — introduces one concept per chapter, in an order that ensures every concept is defined before it is used. Each chapter follows the same skeleton (definition, structure, inline example, API outline, idioms) and uses one of the three running examples to instantiate the concept under discussion. The three examples are chosen so that they collectively exercise every major SMS++ concept once; the reader who

has worked through Part II should be ready to write a new `:Block` (Appendix A) or to assemble one of the Recipes in Part III.

A small but useful summary table appears at the end of the manual as back matter; the reader who wants to know which concept is illustrated by which `:Block` and in which chapter can consult it.

4. Block

[Doxy: Block] [Source: SMS++/include/Block.h]

4.1 Concept

The abstract class Block is the cornerstone of SMS++. A Block is "(a fragment of) a mathematical model with a well-understood semantic". Concretely, a Block contains, all of them optional except where noted:

- any number of Variable objects, organised in *groups* (where a group is a single Variable, a std::vector of Variables, a boost::multi_array of Variables with any number of dimensions, a std::vector< std::vector< Variable > > (a bidimensional arrangement whose second dimension is allowed to vary across the first, as opposed to the rectangular shape of a boost::multi_array< Variable , 2 >), or a std::list of Variables in the dynamic case, ...);
- any number of Constraint objects, organised in groups in the same sense;
- exactly one Objective object (when present at all);
- any number of sub-Block objects, recursively, accessed through a vector of pointers v_Block;
- a pointer to a father Block, if any.

The cardinality "any number" includes zero. A Block with no sub-Block is called a *leaf* Block; the running examples MCFBlock and BinaryKnapsackBlock are both leaves. A Block may have a father Block (and is then said to be a sub-Block of it), or no father (and is then said to be a *root* Block). A Block that is a sub-Block of some father is, conceptually, part of the father's model: every Variable of the sub-Block is automatically a Variable of the father; the father's Objective is conceptually the sum of its own Objective and the Objectives of all its sub-Blocks (with appropriate handling of opposite senses; see Chapter 5).

The base class Block provides no problem-specific functionality. What a concrete :Block derived from it adds is, in essence:

- the **physical** representation of the problem instance: the problem data in its natural form, stored as member fields;
- the **abstract** representation of the problem instance: the groups of Variables, Constraints and the Objective that the abstract representation would consist of, generated on demand by the methods generate_abstract_variables(), generate_abstract_constraints(), generate_dynamic_variables(), generate_dynamic_constraints(), generate_objective();
- methods to mutate the physical representation while keeping the abstract one in sync, each of them able to issue the appropriate Modification objects to inform any listening :Solver of the change.

4.2 The identity rule

A design decision that pervades the framework is that the *name* of a Variable, of a Constraint, and of any other object that may be referred to by an algorithm, is its **memory address**. There is no separate identifier; &v is what makes v distinguishable from every other Variable.

The immediate consequence is:

> Once a Variable or a Constraint has been constructed, it cannot be moved to a different memory location.

This rules out, in particular, storing dynamic Variables or dynamic Constraints in a std::vector, because growing the vector may reallocate the underlying storage and silently invalidate every existing reference. Dynamic

`Variable`s and dynamic `Constraint`s must therefore live in a `std::list`. Static `Variable`s and static `Constraint`s, by contrast, are allocated once in a `std::vector` that is never grown or shrunk, and may safely be addressed by index into that vector.

The same rule applies to a `Block`: a `Block` is referenced by pointer (`p_Block = Block*`), and its sub-`Block`s are stored as a `std::vector` of pointers (`v_Block` of type `Vec_Block`). Copying a `Block` produces a different `Block`, not the same one; if two `Block`s must be kept "the same up to data", the `R3Block` mechanism (Chapter 10) is the supported way to do it.

4.3 Static versus dynamic

Three kinds of contents of a `Block` admit a static / dynamic distinction:

Kind	Static	Dynamic
`Variable`	guaranteed to exist throughout the lifetime of the `Block`	may appear and disappear
`Constraint`	guaranteed to exist throughout the lifetime of the `Block`	may appear and disappear
sub-`Block`	always static	not supported

Sub-`Block`s are always static: their number and their concrete type cannot change after the parent `Block` has been built. This restriction does not hurt expressiveness in practice and considerably simplifies the framework's internal logic.

Static contents allow storage by index; dynamic contents allow storage in a `std::list` only, and are addressed by an iterator or by a `Block`-specific name. Dynamic contents are the support for *column generation* (dynamic `Variable`s) and *row generation* (dynamic `Constraint`s).

4.4 Structure of the class

The principal data members of `Block` are (with shortened types for brevity):

- `p_Block f_Block` — pointer to the father `Block` (`nullptr` for a root `Block`);
- `Vec_Block v_Block` — vector of pointers to the sub-`Block`s;
- `std::string f_name` — optional human-readable name, useful for diagnostic prints;
- a list of `Solver*` currently registered with this `Block` (`v_Solver`);
- a flag `f_at` that caches "is there any `:Solver` listening to this `Block` or to any of its ancestor `Block`s?";
- machinery for concurrent access (`lock()`, `read_lock()`, an owner thread id), discussed in Chapter 17;
- a pointer to a `BlockConfig`, the `Configuration` object that controls the optional behaviour of the `Block`.

The principal public methods, grouped by purpose:

- *navigation*: `get_f_Block()`, `set_f_Block(p_Block)`, `get_nested_Block(Index)`, `get_nested_Block(name)`, `get_number_nested_Blocks()`, `get_nested_Block_index(name)`;
- *contents*: `get_number_static_constraints()`, `get_number_static_variables()`, and their dynamic counterparts;
- *abstract-representation construction*: `generate_abstract_variables(Configuration*)`, `generate_abstract_constraints(Configuration*)`, `generate_dynamic_variables(Configuration*)`, `generate_dynamic_constraints(Configuration*)`, `generate_objective(Configuration*)`;
- *Solver attachment*: `register_Solver(Solver*)`, `unregister_Solver(Solver*)`, `unregister_Solvers()`, `replace_Solver(Solver*, ...)`, `get_registered_solvers()`;
- *Modification handling*: `add_Modification(sp_Mod, ChnlName)`, `anyone_there()`, `concerns_Block()`;
- *checking*: `is_feasible(bool useabstract, Configuration*)`, `is_dual_feasible(...)`, `is_optimal(...)`;
- *bounds*: `get_objective_sense()`, `get_valid_upper_bound(bool conditional)`, `get_valid_lower_bound(bool conditional)`;
- *I/O*: `load(std::istream&, char)`, `load(std::string&, char)`, `serialize(netCDF::NcGroup&) const`, `deserialize(const netCDF::NcGroup&)`, `print(std::ostream&, char)`;

- *reformulation*: `get_R3_Block(Configuration*, base, father)`, `map_back_solution(...)`, `map_forward_solution(...)`, `map_forward_Modification(...)`, `map_back_Modification(...)`;
- *configuration*: `set_BlockConfig(BlockConfig*)`, `get_BlockConfig()`.

The full list of public methods, with their precise signatures and their detailed specification, is in Doxy: `Block`.

4.5 Inline example: constructing an `MCFBlock`

The example below builds an `MCFBlock` from in-memory data. It is deliberately minimal: it neither constructs the abstract representation nor attaches a `:Solver`. Both will be added in Chapter 6 and in Recipe R1.

```
#include "MCFBlock.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 // construct a root MCFBlock (no father)
 MCFBlock mcf;

 // a tiny 3-node, 3-arc instance:
 //
 //     (0) --1--> (1) --1--> (2)
 //      |                     ^
 //      +--------- 1 ---------+
 //
 // node deficits b = (-2, 0, +2)
 // arc capacities U = (3, 3, 3); arc costs C = (2, 3, 4)

 MCFBlock::Subset      pSn = { 0, 1, 0 };  // arc starting nodes
 MCFBlock::Subset      pEn = { 1, 2, 2 };  // arc ending nodes
 MCFBlock::Vec_FNumber pU  = { 3.0, 3.0, 3.0 };
 MCFBlock::Vec_CNumber pC  = { 2.0, 3.0, 4.0 };
 MCFBlock::Vec_FNumber pB  = { -2.0, 0.0, +2.0 };

 // note the parameter order: ending nodes BEFORE starting nodes
 mcf.load( 3 /* n */ , 3 /* m */ , pEn , pSn , pU , pC , pB );

 // human-readable summary on stdout (the default verbosity prints the graph)
 mcf.print( std::cout );

 return 0;
}
```

A few things are worth noticing.

- The constructor `MCFBlock()` is the default; it produces an empty `MCFBlock` with no father. The variant `MCFBlock(Block* father)` installs the new `MCFBlock` as a sub-`Block` of `father`.
- The `load(...)` method populates the physical representation. It takes ending nodes (`pEn`) before starting nodes (`pSn`); this is the signature documented in `MCFBlock.h`.
- No call to `generate_abstract_*()` has been made. The abstract representation does not exist yet; it will be created lazily later, only if a `:Solver` that needs it is attached.
- No call to any factory function has been made by the user, even though the class is registered in the `Block` factory. Factory registration is performed by the `:Block` implementer in the `.cpp` file, once and for all, through the macro `SMSpp_insert_in_factory_cpp_1( MCFBlock );` at namespace scope in `MCFBlock.cpp` (the `_1` indicates that the constructor takes one optional argument, the father pointer). Once registered, the class can be instantiated by name through the `Block` factory; see Chapter 18 for the details.

The same `MCFBlock` could equivalently have been built by reading a DIMACS-formatted text file (with `load(std::istream&)`) or a netCDF file (with `deserialize(netCDF::NcGroup&)`); both are documented forms of construction.

4.6 Common idioms

The "nuclear option" `NBModification`

Every form of `load(...)` on an `MCFBlock` (and, by convention, on every concrete `:Block` that supports `load(...)`) replaces the problem instance wholesale. If a `:Solver` is already attached when `load(...)` is invoked, the framework issues a single `NBModification` — the "nuclear option" — that tells the `:Solver` "everything has changed; throw away whatever you have cached and start over". This is the only `Modification` that `load(...)` is allowed to issue, by convention; incremental changes to the data have their own dedicated `Modification` types (Chapter 8).

The "nuclear option" is the safe default; it is meant to be used sparingly, because it prevents any form of reoptimization.

The "anyone listening?" cache `f_at`

A `:Block` does not always need to construct and dispatch a `Modification` for every change it undergoes. If no `:Solver` is registered with the `Block` and no `:Solver` is registered with any of its ancestors, no one will react to a `Modification`, and the work of constructing it is wasted. To avoid the waste, every `Block` maintains a boolean cache `f_at` ("anyone there?") that is true iff some `:Solver` is currently listening to the `Block` or to any ancestor.

The cache is updated automatically by `register_Solver()` / `unregister_Solver()` and propagated to all sub-`Block`s recursively. `:Block` implementers can check it through `anyone_there()` before paying the cost of constructing a `Modification`. The default implementation of `anyone_there()` in the base class returns the cached value; a concrete `:Block` may override `anyone_there()` to always return `true` when the abstract representation is currently constructed, because abstract `Modification`s must be issued anyway in order to keep the two representations in sync (Chapter 8).

The `father` argument of the constructor

A concrete `:Block` constructor typically takes one optional argument, a pointer to a father `Block` (default `nullptr`). Passing a non-null father makes the new `Block` a sub-`Block` of the father; the framework will automatically install the new `Block` in the father's `v_Block`. This is the canonical way to assemble a `Block` tree from the leaves up; it is exemplified by `CapacitatedFacilityLocationBlock` in its Knapsack Formulation, which constructs its `BinaryKnapsackBlock` sub-`Block`s by passing itself as the father (Chapter 12).

With `Block` introduced, the next chapter turns to the three primitives that live *inside* a `Block`: `Variable`, `Constraint`, `Objective`.

5. Variable, Constraint, Objective

[Source: SMS++/include/Variable.h, ColVariable.h, Constraint.h, RowConstraint.h, OneVarConstraint.h, FRowConstraint.h, Objective.h, RealObjective.h, FRealObjective.h, LinearObjective.h]

5.1 Concept

The three primitives that live inside a Block are Variable, Constraint, and Objective. A Block contains, conceptually, any number of Variables and Constraints organised in *groups*, and at most one Objective. Together they constitute the *abstract representation* of the mathematical model carried by the Block (Chapter 7); when a Block is solved by a general-purpose :Solver, this is what the :Solver reads.

All three classes share two architectural traits:

- each instance **belongs to one Block** (its f_Block field points to the owning Block);
- the *name* of an instance is its **memory address**, with the consequence — discussed for Block in §4.2 — that an instance cannot be moved once constructed. Dynamic groups therefore live in std::list; static groups live in std::vector or boost::multi_array whose storage is allocated once and never reallocated.

Constraint and Objective further share two interfaces inherited from common base classes:

- they derive from ThinVarDepInterface: each Constraint and each Objective depends on a specific set of *active* Variables, queryable through that interface;
- they derive from ThinComputeInterface: a Constraint's satisfaction status and an Objective's value must be compute()-d before being read, and computation is expected to be potentially costly (for instance, an Objective given by a multi-dimensional integral, or a Constraint given by the solution of a hard sub-problem).

Variable, by contrast, derives from neither: its "value" is specified by derived classes, not by the base, and it has no notion of computation (only of being fixed or not).

5.2 Variable

The base class Variable makes very few assumptions:

- a Variable belongs to one Block (get_Block());
- a Variable influences a set of *active stuff* (instances of ThinVarDepInterface: Constraints, Objectives, Functions); the set is exposed through v_iterators and queried through is_active(...);
- a Variable can be (temporarily) *fixed* via is_fixed(bool, ModParam); the current state is read by is_fixed() with no arguments.

No method to *read the value* of a Variable exists in the base class, because that requires knowing what kind of value it is.

ColVariable

The overwhelmingly common concrete :Variable is ColVariable, which holds a single real value (VarValue = double). The name is a deliberate reminiscence of the column structure of a linear programming coefficient matrix.

`ColVariable` extends `Variable` in two directions. First, it gives the value: `set_value(VarValue)`, `get_value()`. Second, it can be restricted to a subset of the reals by setting a *type* (`col_var_type` enum); the 16 possible types are

```
kContinuous     any real value
kInteger        any integer value
kNonNegative    any non-negative real value
kNatural        any non-negative integer value
kNonPositive    any non-positive real value
kNegative       any non-positive integer value
kZeroReal       any real value provided it is 0
kZeroInteger    any integer value provided it is 0
kUnitary        any real value in [-1, 1]
kTernary        either -1, 0, or 1
kPosUnitary     any real value in [0, 1]
kBinary         either 0 or 1
kNegUnitary     any real value in [-1, 0]
kNegBinary      either -1 or 0
kZeroRealU      any real value in [-1, 1] provided it is 0
kZeroIntU       any integer value in [-1, 1] provided it is 0
```

Combined with the "fixed / not fixed" state, these cover most of the bound, sign and integrality restrictions found in practical models. Crucially, all of these are *type* restrictions stored inside the `ColVariable` itself, *not* `Constraints` in the `Block`: this saves a substantial amount of memory in models where most variables are, say, simply non-negative or binary.

5.3 Constraint

The base class `Constraint`, beyond what its base classes already provide, supports:

- *relaxation*: `relax(bool, ModParam)` temporarily disables the `Constraint`; `is_relaxed()` reads the current state;
- *feasibility checking*: `feasible()` returns `true` if the `Constraint` is satisfied at the current values of its active `Variables`. This is a pure virtual method; it relies on `compute()` having been called first.

`Constraint` is otherwise abstract: it specifies nothing about *what* it constrains.

`RowConstraint`

`RowConstraint` specialises `Constraint` to the case "*a real-valued left-hand side lies between a lower and an upper bound*", i.e. the canonical row constraint of a linear (or non-linear) program:

$$\text{lhs_bound} \leq \text{LHS} \leq \text{rhs_bound}.$$

Two `RHSValues` store the lower and upper bounds, accessed via `get_lhs()` / `get_rhs()` / `set_lhs(...)` / `set_rhs(...)` / `set_both(...)`. Either bound may be infinite (sentinel `RHSINF`). A single real dual variable is attached, accessed via `get_dual()` and set by `set_dual(...)`. The actual real-valued left-hand side expression is *not* defined in `RowConstraint`; that is left to subclasses.

Two principal subclasses cover the common cases.

`OneVarConstraint`

`OneVarConstraint` is the case "*the real-valued left-hand side is the value of one single `ColVariable`*", i.e. a bound (or pair of bounds) on a single variable. The class ships with several pre-defined refinements: `BoxConstraint`, `LB0Constraint`, `UB0Constraint`, `LBConstraint`, `UBConstraint`, `NNConstraint` (non-negativity, $\geq$ `0`), `NPConstraint` (non-positivity, $\leq$ `0`), `Z0Constraint` (`x` $\in$ `[0,1]`). The most common of these, `LB0Constraint`, encodes "`0` $\leq$ `x` $\leq$ `ub`", and is the variant used by `MCFBlock` for the arc capacity constraints when the per-arc upper bound is not infinite (§4.5 and Chapter 7).

FRowConstraint

FRowConstraint is the case "*the real-valued left-hand side is computed by a* ***Function***", and is the workhorse of essentially every "real" linear or non-linear constraint in the framework. The Function inside is set with set_function(Function*) and retrieved with get_function(). The framework takes ownership of the Function* passed to set_function: passing nullptr releases it.

The Function family (Chapter 13) covers linear, quadratic, and polyhedral cases, plus the two specialised "Block + Function" hybrids LagBFunction and BendersBFunction. For a "row of a linear program" the standard combination is FRowConstraint carrying a Function that evaluates a linear expression in ColVariables, which is exactly what MCFBlock uses for its flow-conservation constraints.

5.4 Objective

The base class Objective adds two facts to its base classes:

- it has a *sense* (int f_sense), either eMin = -1 (minimisation, lower values are better) or eMax = +1 (maximisation). The sense is read by get_sense() and set by set_sense(int, ModParam). The base class assumes the value of the Objective is totally ordered, so that minimising vs maximising is well defined; problems with vector-valued objectives would extend the class.
- a value type is *not* defined in Objective directly, leaving room for multi-objective extensions. A derived class RealObjective (and FRealObjective below) fixes the value type to be an *extended* real (RealObjective::OFValue, the same as the value type of RealFunction), so that $+\infty$ and $-\infty$ are representable to encode infeasibility and unboundedness.

The composition rule for sub-Blocks is also fixed at the Objective level: the Objective of a sub-Block is summed into the Objective of its father Block. This is what gives the recursive Block tree a single, well-defined Objective overall.

FRealObjective

FRealObjective is to Objective what FRowConstraint is to Constraint: the value is computed by a Function. The interface is the same: set_function(Function*) (the framework takes ownership) and get_function(). FRealObjective is the standard choice for any Block whose Objective is best described by a Function, which in practice is "essentially every Block with a non-trivial objective".

A simpler LinearObjective exists for the case where the objective is a plain linear form in ColVariables; it stores the coefficients directly without the Function indirection.

5.5 Inline example: the abstract representation of BinaryKnapsackBlock

A reader who reaches this point may be tempted to construct a Block's abstract representation by hand: instantiate a vector of ColVariables, build an FRowConstraint carrying a linear Function, build an FRealObjective likewise, and connect everything by hand. **That is not how the framework works.** The abstract representation of a concrete :Block is built *by the* ***:Block*** *itself*, on demand, through its generate_abstract_variables(), generate_abstract_constraints() and generate_objective() overrides. User code typically *triggers* these calls but does not *populate* what they produce.

For BinaryKnapsackBlock, the trigger is as small as:

```
#include "BinaryKnapsackBlock.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 // physical data: 4 items, knapsack of capacity 5
 BinaryKnapsackBlock          bkb;
 const std::vector< double > W = { 2.0, 3.0, 4.0, 1.0 };  // weights
 const std::vector< double > P = { 3.0, 4.0, 5.0, 1.0 };  // profits
```

```
 const std::vector< bool >   I = { true, true, true, true };  // integral
 bkb.load( W.size() , /* capacity = */ 5.0 , W , P , I );

 // construct the abstract representation on demand
 bkb.generate_abstract_variables();
 bkb.generate_abstract_constraints();
 bkb.generate_objective();
 // from this point on, bkb carries both the physical and the
 // abstract representation; the two are kept in sync by the
 // mechanisms covered in Chapter 7 and Chapter 8.

 return 0;
}
```

What `BinaryKnapsackBlock` constructs in response — and what the three primitives of this chapter look like in concrete — is documented in the `:Block`'s own source comments and reflected in its protected data members (`BinaryKnapsackBlock.h:940-942`):

- one *static* group of `Variable`s, materialised as `std::vector< ColVariable > v_x` of size n (one per item), of `col_var_type` either `kBinary` (integer items) or `kPosUnitary` (continuous items), according to the integrality vector;
- one *static* group of `Constraint`s, consisting of a single `FRowConstraint f_cnst` whose `Function` is a linear expression $\sum_i W_i x_i$ with right-hand side equal to the capacity C and no lower bound;
- a single `FRealObjective f_obj` carrying a linear `Function` with coefficients P_i, whose sense is `eMax` by default and can be flipped to `eMin` by `bkb.set_objective_sense(false)`.

The three protected fields are accessible to the rest of the framework (and to a derived `:Block`), but they are not part of the public interface of `Block` and should not be reached at by user code; the framework convention is to read the *physical* representation through the `:Block`-specific accessors (`get_Var(i)`, `get_objective_value()`, `get_x(...)`, ...) rather than to walk the abstract groups directly. The mechanism for *iterating* over the abstract groups of an arbitrary `Block` is at present mediated by `boost::any`, an arrangement scheduled for revision in a future release; for the moment, code that needs to look at the abstract representation of a `:Block` it does not know should go through the `:Block`'s own getters.

The deliberately small `AbstractBlock` class (Chapter 1, also discussed in Chapter 4) is the converse case: a `Block` *without* a problem class of its own, whose abstract representation *is* constructed by user code from the outside. `AbstractBlock` is the correct vehicle if the goal is to assemble a `Variable` / `Constraint` / `Objective` arrangement by hand — typically when loading a `.mps` or `.lp` file — and the construction patterns of this section are the canonical way to do so. The interested reader will find a worked example of an `AbstractBlock` built from scratch in Recipe R3 (where the MCF flow relaxation of CFLB is described) and a more systematic treatment in Appendix A, where a fresh `:Block` is developed from scratch.

5.6 API outline

Class	Key methods
`Variable`	`get_Block()`, `is_fixed()`, `is_fixed(bool, ModParam)`, iterators on active stuff
`ColVariable`	`set_value(VarValue)`, `get_value()`, `is_integer(bool, ModParam)`, `is_positive(...)`, `is_unitary(...)`, `get_type()`
`Constraint`	`relax(bool, ModParam)`, `is_relaxed()`, `feasible()`, `compute(bool)`
`RowConstraint`	`set_lhs`, `set_rhs`, `set_both`, `get_lhs`, `get_rhs`, `get_dual`, `set_dual`
`OneVarConstraint`	accepts a single `ColVariable`; the bounds are the only data
`FRowConstraint`	`set_function(Function*)`, `get_function()`; LHS is the `Function`'s value

Class	Key methods
`Objective`	`set_sense(int, ModParam)`, `get_sense()`, `compute(bool)`
`RealObjective`	adds `OFValue` and the `value()` accessor
`FRealObjective`	`set_function(Function*)`, `get_function()`
`LinearObjective`	direct linear-form storage, no `Function*` indirection

For the exhaustive list of public methods, see the corresponding Doxygen page of each class.

5.7 Idioms

Pointer-identity equality. Anywhere a `Variable*` or a `Constraint*` appears, it is compared with `==`; two pointers referring to the same memory address are the same object. This is not just an optimisation: it is the only sense of "equality" the framework recognises. In particular, two `ColVariable`s with the same value are *not* the same variable; copying a `ColVariable` into a different vector slot makes a new, distinct `ColVariable`.

Ownership transfer to wrappers. `FRowConstraint::set_function` and `FRealObjective::set_function` *take ownership* of the `Function*` passed in. The caller must not delete it, must not re-attach it to a second wrapper, and should not keep a pre-existing pointer to it for later use (the wrapper may replace or destroy the function in response to a `Modification`). Passing `nullptr` releases the currently held `Function*` (and deletes it).

Composition of `Objectives` in a `Block` tree. When a sub-`Block` carries an `Objective`, it contributes additively to the `Objective` of its father (and recursively up). A `Block` can have no `Objective` of its own and still expose a non-trivial overall `Objective` made up entirely of those of its sub-`Block`s; this is the standard pattern for "master" `Block`s that only carry linking constraints, as in `CapacitatedFacilityLocationBlock` in the Knapsack Formulation (Chapter 12, Recipe R4).

6. Solver

[Source: SMS++/include/Solver.h, CDASolver.h, ThinComputeInterface.h]

6.1 Concept

A Solver is the abstract base class for any algorithm capable of "computing" on a Block: typically, producing one or more (approximately feasible, approximately optimal) solutions, or proving that no solution exists, or determining that the model is unbounded. The class derives from ThinComputeInterface, which is the SMS++ abstraction of "anything that has a compute() method whose call is expected to be costly".

A Solver is *attached* to one Block, its target. The Solver's responsibility is to solve the *entire* Block, which includes any sub-Block of the target, recursively. What the Solver is unaware of, by contrast, is the part of the Block tree that lies *above* the target: any father Block, any uncles and grand-uncles, any ancestor's Variables and Constraints. (If the target is a root Block no part of the tree lies above it, and the Solver sees the whole model.)

The principal interactions between Block and Solver are four:

- **registration / deregistration**: a Block calls Solver::set_Block(p_Block) (typically via Block::register_Solver(Solver*)) to attach the Solver to itself; deregistration restores set_Block(nullptr). Any number of :Solvers may be attached to the same Block.
- **modification notification**: any change in the target Block or in any of its sub-Blocks, recursively, that occurred after the Solver was attached is delivered as a Modification and enqueued in the Solver's pending list (Chapter 8). The Solver is responsible for consuming the list when it is next invoked.
- **computation**: the user calls Solver::compute(bool changedvars) to ask the Solver to (re-)solve. The return value is an sol_type enum that summarises the outcome (§6.3). The changedvars parameter is *not* about whether the target Block has changed (this is signalled via Modifications in the pending list, see above); it is about whether the *values* of any Variable upon which the computation depends may have been altered between the previous compute() and this one. The relevant Variables are those in the abstract representation of the target Block (and of its sub-Blocks, recursively), plus any Variable belonging to an ancestor Block that appears in a Constraint of the target (and that the target therefore treats as a constant whose current value matters; see Chapter 8). With changedvars = true (the default) the Solver must assume that these values may have changed and re-read them as needed; with changedvars = false the caller is *guaranteeing* that they have not changed since the last call, which allows the Solver to resume the previous computation from where it had stopped, if the state of the search has been preserved. The guarantee is the caller's responsibility; the Solver is not required to verify it.
- **solution retrieval**: after compute() returns, the user calls new_var_solution(...) / get_var_solution(...) to iterate through the primal solutions the Solver has produced. Similarly, a CDASolver ("Convex Duality Aware") exposes new_dual_solution(...) / get_dual_solution(...) for one or more dual solutions.

A few :Solver are *exact*, in the sense that, given unbounded computational resources and no error condition, they will always eventually return one among kOK, kUnbounded, kInfeasible. An exact :Solver is not required to do so under any *bounded* budget: when forced to stop early, an exact :Solver may well return kStopTime, kStopIter or kLowPrecision, exactly like a heuristic one would. A genuinely *heuristic* :Solver differs in that it cannot in principle guarantee optimality even given unbounded resources, and may therefore return kLowPrecision as its terminal status even when no resource limit has been hit.

6.2 Specialised versus general-purpose

Solvers come in two flavours.

A **specialised** :Solver is written for a specific :Block class. It reads the *physical* representation directly: the problem data in the form the :Block stores it (a graph for MCFBlock, a weights / profits / capacity triple for BinaryKnapsackBlock, etc.). A specialised :Solver is typically fast, because it knows the structure and can exploit it; the cost of that speed is that it works only for that one :Block. The running examples include MCFSolver<MCFC> (a wrapper around an MCFClass algorithm) for MCFBlock, and DPBinaryKnapsackSolver (SMS++-native dynamic programming) for BinaryKnapsackBlock.

A **general-purpose** :Solver knows nothing about the specific problem class. It reads the *abstract* representation — the Variables, Constraints, Objective introduced in Chapter 5 — and applies a general algorithm to it. The running example is the MILPSolver family, which constructs a matrix-form MILP from the abstract representation and hands it to one of the major MIP back-ends (CPLEX, Gurobi, HiGHS, SCIP) through a derived class. A general-purpose :Solver is typically slower than a specialised one on the same problem, but it works on any :Block that can expose a suitable abstract representation.

Switching between the two is typically a one-line change in the BlockSolverConfig (Chapter 11). User code that depends only on the Solver interface (calling compute() and reading the solutions) does not need to change.

6.3 The sol_type return enum

Solver::compute() returns a value of type Solver::sol_type, an enum that extends ThinComputeInterface::compute_type with several values specific to optimisation. The enum is divided into four *ranges* by the sentinel values kUnEval, kOK, kError; the range, not just the specific value, carries semantic meaning. The convention is:

- values $\leq$ kUnEval mean *the computation has not finished yet.* kStillRunning is one such value: compute() was re-entered (typically from an event handler) before the previous invocation had returned. kUnEval itself means compute() has not been called at all. These values are not normally returned *by* compute() (which by definition has finished when it returns); they are used internally and observed via event handlers.
- values strictly between kUnEval and kOK (inclusive) mean *the computation succeeded.* kOK ("optimal solution to within the required accuracy") is the prototypical success; other values in this range encode "different kinds of success", such as kUnbounded (the model is provably unbounded) and kInfeasible (the model is provably infeasible). The Solver has, in all of these cases, a *certificate* of its claim; the form of the certificate is problem-class-specific and not exposed by a uniform interface.
- values strictly between kOK and kError mean *the computation was stopped early in a recoverable state.* The principal codes are kStopTime (time limit hit), kStopIter (iteration limit hit), and kLowPrecision (a feasible solution was found, but optimality to within the required accuracy could not be certified). Calling compute() again with a fresh time or iteration budget allows the search to continue from where it stopped; this range is therefore not terminal.
- values $\geq$ kError mean *the computation hit an unrecoverable error and is unlikely to make progress on a subsequent call.* The error need not be "non-numerical": typical cases include numerical failures (a linear system that cannot be factorised, an iterate that diverges, ...) as well as structural failures such as kBlockLocked (the Block could not be acquired, see Chapter 17). The convention is that kError is the *smallest* value in this range and that each concrete :Solver may extend the range past it with :Solver-specific codes that encode the exact kind of error (e.g. "ran out of memory", "callback raised an exception", ...). The user code should therefore treat *any* return value $\geq$ kError as an error, and consult the :Solver-specific documentation only when it wants to distinguish between different kinds.

The principal values defined by Solver::sol_type on top of ThinComputeInterface::compute_type are:

Value	Range	Meaning
kStillRunning	$<$ kUnEval	compute() re-entered before the previous call returned (typically from an event handler).
kUnEval	sentinel	compute() has not been called yet.

Value	Range	Meaning
`kUnbounded`	$\leq$ `kOK`	The model is provably unbounded; the `Solver` can produce feasible solutions of arbitrarily large (or small) objective on demand.
`kInfeasible`	$\leq$ `kOK`	The model is provably infeasible.
`kBothInfeasible`	$\leq$ `kOK`	Both the primal and the dual are infeasible.
`kOK`	sentinel	An optimal solution to within the required accuracy was found.
`kStopTime`	$<$ `kError`	Stopped because `dblMaxTime` was reached; calling `compute()` again resets the timer.
`kStopIter`	$<$ `kError`	Stopped because `intMaxIter` was reached; calling `compute()` again resets the iteration count.
`kLowPrecision`	$<$ `kError`	A feasible solution was found but optimality could not be certified; an exact `:Solver` returns this only when forced to stop early, a heuristic `:Solver` may return it as its terminal status.
`kBlockLocked`	$\geq$ `kError`	Could not acquire the lock on the `Block`.
`kError`	sentinel	Generic unrecoverable error; a concrete `:Solver` typically extends the range with more specific codes.

6.4 Parameters

A `Solver` exposes a uniform parameter interface inherited from `ThinComputeInterface` and extended for the optimisation case. Parameters are indexed by an integer enum and have one of three value types: `int`, `double`, `std::string`. The principal predefined slots are:

- *integer parameters* (`int_par_type_S`):
  - `intMaxIter` — maximum number of "iterations" the `Solver` may execute before returning `kStopIter`; the concept of "iteration" is `:Solver`-specific.
  - `intMaxSol` — maximum number of distinct solutions to keep and report on demand.
  - `intLogVerb` — verbosity level of the log (0 = silent).
- *double parameters* (`dbl_par_type_S`):
  - `dblMaxTime` — maximum wall-clock time before returning `kStopTime`.
  - `dblRelAcc`, `dblAbsAcc` — relative and absolute accuracy for declaring a solution optimal.
  - `dblUpCutOff`, `dblLwCutOff` — upper and lower cutoffs for stopping the algorithm early.
  - `dblRAccSol`, `dblAAccSol`, `dblFAccSol` — accuracies and constraint violations tolerated in any reported primal solution. The `CDASolver` adds the corresponding `*DSol` variants for dual solutions.

A specific `:Solver` typically extends both enums with its own parameters (for instance, `DPBinaryKnapsackSolver` adds `dblReopt`, and `LagrangianDualSolver` adds a number of parameters controlling the inner method). All parameters are read and written through `get_par(idx)` and `set_par(idx, value)`; in addition, the same parameters can be bundled into a `ComputeConfig` object that loads from text files or netCDF (Chapter 11).

6.5 Asynchronous computation

`Solver` exposes `compute_async()` (inherited from `ThinComputeInterface`) which launches the `compute()` in a separate thread and returns a `std::future< int >` from which the caller can later retrieve the `sol_type`. The expectation is that the `:Solver`'s `compute()` is thread-safe with respect to the `Block` it is attached to; the framework provides recursive locking on the `Block` to make this manageable (Chapter 17).

6.6 Feasibility and optimality checks

Three closely-related methods on the `Block` are used to verify the *result* of a `Solver`'s computation: `is_feasible()`, `is_dual_feasible()`, `is_optimal()`. All three take a boolean `useabstract`:

- `useabstract = false` (the default) asks the `Block` to check the current solution against the **physical** representation; this is typically much cheaper.
- `useabstract = true` asks the `Block` to check against the **abstract** representation; this only works if the abstract representation has been built (Chapter 7).

The two answers should agree up to numerical tolerance. They may genuinely disagree only in edge cases (for instance when dynamic `Constraint`s have not yet been generated by the abstract path). A specialised `:Solver` typically calls `Block::is_optimal(false)` after its own `compute()` returns `kOK`, as a final sanity check on the physical representation.

The full discussion of why both paths exist — and of the particular asymmetry caused by the current "always materialised `Variable`s" implementation (already flagged in §3.3 as **Status — under development**) — is the topic of Chapter 7.

6.7 Inline example: attaching an `MCFSolver` to an `MCFBlock`

The example below continues the `MCFBlock` of §4.5: it constructs the same small instance and attaches a specialised `MCFSolver` templated on `MCFSimplex` (the classical primal simplex implementation shipped in the `MCFClass` external library).

```
#include "MCFBlock.h"
#include "MCFSolver.h"
#include "MCFSimplex.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 // construct the MCFBlock and load the instance (as in §4.5)
 MCFBlock mcf;
 MCFBlock::Subset      pSn = { 0, 1, 0 };
 MCFBlock::Subset      pEn = { 1, 2, 2 };
 MCFBlock::Vec_FNumber pU  = { 3.0, 3.0, 3.0 };
 MCFBlock::Vec_CNumber pC  = { 2.0, 3.0, 4.0 };
 MCFBlock::Vec_FNumber pB  = { -2.0, 0.0, +2.0 };
 mcf.load( 3 , 3 , pEn , pSn , pU , pC , pB );

 // construct the specialised Solver and attach it to the MCFBlock
 auto * solver = new MCFSolver< MCFSimplex >();
 mcf.register_Solver( solver );

 // ask the Solver to compute
 const int code = solver->compute( /* changedvars = */ true );

 // interpret the return code
 if( code == Solver::kOK ) {
  // optimal solution found; primal flows have been written into the
  // ColVariable of the MCFBlock by the Solver. Read them back via the
  // MCFBlock's own physical accessors.
  std::cout << "objective = " << mcf.get_objective_value() << '\n';
  MCFBlock::Vec_FNumber x( mcf.get_NArcs() );
  mcf.get_x( x.begin() );
  for( MCFBlock::Index a = 0 ; a < mcf.get_NArcs() ; ++a )
   std::cout << "  flow on arc " << a << " = " << x[ a ] << '\n';
  }
 else if( code == Solver::kInfeasible ) {
  std::cout << "infeasible.\n";
  }
```

```
 else if( code == Solver::kUnbounded ) {
  std::cout << "unbounded.\n";
  }
 else {
  std::cout << "solver returned " << code << ".\n";
  }

 // detach and clean up
 mcf.unregister_Solver( solver , /* deleteold = */ true );
 return 0;
}
```

Four things to notice.

1. The `MCFSolver` is templated on the underlying `MCFClass` algorithm. `MCFSimplex` is one option; `RelaxIV`, `MCFCplex`, `SPTree` and others are also available, each in its own header. The choice fixes the algorithm at compile time.
2. `register_Solver` *does not* take ownership of the `Solver*`; the call only enrols it in the `Block`'s registered-solvers list. Ownership is the caller's. The convenience flag `deleteold = true` on `unregister_Solver` asks the framework to `delete` the `Solver` once it has been detached.
3. The primal solution is written by the `Solver` into the `ColVariable`s of the `Block`. Because `MCFBlock` is here solved by a specialised `Solver` that has not built the abstract representation, this materialisation is logically unnecessary — but it is what currently happens (cf. §3.3, "always materialised"). The recommended way to read the solution from a specialised `:Solver` is via the `Block`'s physical accessors, here `get_x(...)` and `get_objective_value()`, not via iterating over the `ColVariable`s directly.
4. Swapping `MCFSolver< MCFSimplex >` for a generic `:MILPSolver` (say, `CPXMILPSolver`) requires no change to the surrounding code: the user calls `mcf.register_Solver(milp)`, `milp->compute()`, then reads the solution as before. What does change, however, is that `MILPSolver` *needs* the abstract representation, which means `mcf.generate_abstract_variables()`, `mcf.generate_abstract_constraints()`, `mcf.generate_objective()` must have been called first. This is exactly the trade-off Chapter 7 makes precise.

6.8 API outline

The principal public methods of `Solver`, grouped by purpose:

- *attachment*: `set_Block(p_Block)`, `get_Block()`;
- *computation*: `compute(bool changedvars)`, `compute_async()`;
- *parameters*: `set_par(idx, int|double|string)`, `get_par(idx)`, plus `set_ComputeConfig(ComputeConfig*)` for bulk configuration;
- *primal solutions*: `new_var_solution(...)`, `get_var_solution(...)`, `sol_count()`;
- *dual solutions* (on `CDASolver` only): `new_dual_solution(...)`, `get_dual_solution(...)`, `dsol_count()`;
- *bounds and value*: `get_lb()`, `get_ub()`, `get_value()`;
- *events*: `register_event(...)` for user-callbacks invoked periodically by `compute()`;
- *logging*: `set_log(std::ostream*)`, `set_par(intLogVerb, level)`.

The full method-by-method specification, including the precise contract of each parameter and each callback signature, is in Doxy: `Solver` and Doxy: `CDASolver`.

6.9 Idioms

Switching `:Solvers` is a configuration change. User code that depends only on the `Solver` interface — `register_Solver`, `compute`, read the solution from the `Block` — does not need to know whether the `Solver` is specialised or general-purpose. The choice is typically expressed in a `BlockSolverConfig` text file, and changing it is one line. The framework's idiom is *not* to write a different program for each choice of `Solver`.

Reading solutions through the `Block`, not the `Solver`. Once `compute()` returns `kOK`, the framework convention is that the primal solution lives in the `Block` (in its `Variable`s, in its physical accessors, and in its `Solution` objects produced by `get_Solution()`). The `Solver` is the *agent* that put the solution there; it

is *not* the canonical place from which to read it back. This makes specialised and general-purpose `:Solver`s interchangeable from the reader's point of view.

`Solver::new_Solver` for runtime selection. When the choice of `:Solver` is itself a runtime parameter (read from a configuration file), the canonical idiom is

```
Solver * s = Solver::new_Solver( "MCFSolver<MCFSimplex>" );
mcf.register_Solver( s );
```

`new_Solver` consults the `Solver` factory, which any concrete `:Solver` registers itself in by means of a one-line macro at namespace scope in its `.cpp` file (Chapter 18).

7. Physical vs Abstract representation

[Source: SMS++/include/Block.h, MCFBlock/include/MCFBlock.h, BinaryKnapsackBlock/include/BinaryKnapsackBlock.h]

This chapter makes precise what Chapter 3 introduced informally: a Block carries two representations of the same mathematical model and the framework's machinery is designed to let both coexist without forcing the user to pick one. It also closes the "Variables are always materialised" caveat already flagged in §3.3, by stating the current limitation precisely and indicating the direction the API is intended to take.

7.1 Two faces of the same model

A concrete :Block exposes its model along two complementary faces.

The **physical** representation, sometimes called the *semantic* representation, is the problem data in its natural form. The :Block stores it as ordinary C++ data members. For MCFBlock the physical representation is essentially the seven pieces NNodes, NArcs, two arrays of node indices SN[a] and EN[a] that encode the start and end nodes of every arc a, a vector of arc capacities U[a], a vector of arc costs C[a], and a vector of node deficits B[i]. For BinaryKnapsackBlock the physical representation is a capacity f_C, a vector of weights v_W, a vector of profits v_P and a boolean integrality vector v_I. For CapacitatedFacilityLocationBlock the physical representation is a triple of dimensions (f_n_facilities, f_n_customers, f_max_facilities), the demand and capacity vectors, the fixed-cost vector and the transportation-cost matrix.

The **abstract** representation, sometimes called the *syntactic* representation, is the same model expressed as the explicit Variables, Constraints and Objective introduced in Chapter 5: it is the form a general-purpose solver would expect. For MCFBlock the abstract representation consists of one std::vector< ColVariable > of size NArcs for the arc flows, one std::vector< FRowConstraint > of size NStaticNodes (and possibly a std::list< FRowConstraint > for the dynamic nodes) for the flow-conservation constraints, optionally a std::vector< LBOConstraint > of size NStaticArcs (and possibly a std::list< LBOConstraint > for dynamic arcs) for the arc capacity bound constraints, and an FRealObjective carrying a linear Function for the total flow cost. The latter two groups are constructed only when needed: the LBOConstraints are skipped when all capacities are infinite, and the Objective shape can be tuned between "dense" and "sparse" by an option (cf. generate_objective() in §7.3).

The two representations refer to the same model and are *conceptually equivalent*: a feasible solution in one is a feasible solution in the other, with the same objective value, up to numerical tolerance.

7.2 Why two representations

The reason for keeping both faces around is that different :Solvers have different needs.

A *specialised* :Solver written for a specific :Block knows the problem and reads the physical representation directly. The MCFSolver< MCFC > of Chapter 6, for instance, hands the seven arrays of MCFBlock to an underlying MCFClass algorithm; it does not look at the abstract representation at all. The specialised path is typically the fastest one and the cheapest one in memory.

A *general-purpose* :Solver such as :MILPSolver, by contrast, knows nothing about min-cost flow specifically. It reads the abstract representation: it iterates the Variables, builds the constraint matrix from the linear-Function-carrying FRowConstraints, walks the Objective, and hands the resulting MILP to its underlying back-end (CPLEX, Gurobi, HiGHS, SCIP). This path works on any :Block that exposes a suitable abstract representation, at the price of carrying the explicit Variable/Constraint/Objective objects in memory.

Having only the physical face would lock out general-purpose :Solvers; having only the abstract face would forbid the speed gains of specialisation. SMS++ keeps both faces around and lets the user pick, on a per-Solver basis, which one to consume.

7.3 On-demand construction of the abstract representation

The physical representation exists as soon as the :Block has been loaded (cf. §4.5). The abstract representation, by contrast, is *built on demand*: a :Block does not pay the cost of constructing it until a :Solver that needs it is attached. The construction is triggered by five virtual methods that every concrete :Block overrides:

- generate_abstract_variables(Configuration*) constructs the static Variable groups.
- generate_abstract_constraints(Configuration*) constructs the static Constraint groups.
- generate_dynamic_variables(Configuration*) constructs the dynamic Variable lists (typically empty initially).
- generate_dynamic_constraints(Configuration*) constructs the dynamic Constraint lists.
- generate_objective(Configuration*) constructs the Objective.

Each takes a Configuration* that *gates* optional parts of the construction. The :Block may interpret it as it wishes; the canonical pattern is that the Configuration* is a SimpleConfiguration< int > whose f_value is a bit-mask of "which groups to materialise". For instance, MCFBlock interprets the Configuration* of generate_abstract_constraints to decide whether the arc-bound LBOConstraints are skipped (which is allowed only if all arc capacities are infinite, in which case the bound is already encoded in the ColVariables' kNonNegative type); see MCFBlock.h:506-571. CapacitatedFacilityLocationBlock goes further and uses the Configuration* of generate_abstract_variables to select one of the four formulations described in §3.5; see CapacitatedFacilityLocationBlock.h:472-625.

When the same Configuration parameters need to be applied across a whole Block tree, the recursive [C/O/R]BlockConfig family of Chapter 11 is the supported way to do it; it carries the Configuration for generate_abstract_variables, generate_abstract_constraints, generate_dynamic_*, generate_objective, plus a few other slots, and propagates them recursively into the sub-Blocks.

The user-facing rule is simple: if only specialised :Solvers are attached, the abstract representation need not be constructed at all (with the caveat in §7.6 below); as soon as a :Solver that reads the abstract face is attached, the corresponding generate_* methods are called once and the abstract face becomes populated. The two faces are then kept in sync by the mechanisms of Chapter 8.

7.4 The lifetime of the abstract representation

Once constructed, the abstract representation lives for as long as the :Block lives, with two exceptions.

First, a :Block may be *reset* by another load(...) call (or by deserialize(...)), which replaces the problem instance wholesale. The framework convention is that load(...) issues a single NBModification (the "nuclear option" of §4.6) to every listening :Solver, and the abstract representation is rebuilt the next time it is needed. The :Block's generate_* methods are *not* called automatically; they are called when a :Solver that needs them next reads it. This is consistent with the "on demand" rule.

Second, dynamic groups of Variable or Constraint may be added to or removed from the abstract representation at any time through the dedicated add_dynamic_variable, remove_dynamic_variable, add_dynamic_constraint, remove_dynamic_constraint operations. These are the support for column generation and row generation; they issue dedicated Modifications (BlockModAdd / BlockModRmv, see Chapter 8) and are consumed by :Solvers that know how to handle them.

7.5 Feasibility and optimality checks across both faces

Three closely related virtual methods on the Block class encapsulate the duality between the two representations: is_feasible(useabstract, Configuration*), is_dual_feasible(useabstract, Configuration*), is_optimal(useabstract, Configuration*). All three take a boolean useabstract:

- useabstract = false (the default) asks the :Block to check the current candidate solution against the *physical* representation. This path is typically much cheaper: the :Block walks its own arrays directly and applies the problem-specific definition of feasibility. For MCFBlock, is_feasible(false) calls flow_feas

ible(...)` and `bound_feasible(...)`, both of which read the arc-flow values and check them against the physical data (`MCFBlock.h:980-1054`).
- `useabstract = true` asks the `:Block` to check against the *abstract* representation by iterating the `Constraints` and calling `feasible()` on each. This path only works if the abstract representation has been built, and is generally slower; its principal use is by general-purpose `:Solver`s that do not know the physical representation and have no faster alternative.

The two answers must agree up to numerical tolerance under the `Configuration*` parameter that sets the tolerance; the `Configuration*` is documented per `:Block` and typically reads from the `:Block`'s `BlockConfig::f_is_feasible_Configuration`. The two answers may, however, *genuinely* disagree in edge cases where the abstract representation has not been fully synchronised yet — most notably when dynamic `Constraints` have not yet been generated. The `MCFBlock` source comments (`MCFBlock.h:4280-4330`) discuss this in detail and we refer the reader to that source for the precise contract.

`is_optimal` is exactly `is_feasible && is_dual_feasible && complementary_slackness`; it requires *both* the primal and the dual solutions to have been written into the appropriate `Variable`s and `Constraints`, which in turn requires the abstract representation to carry both groups.

7.6 Status — under development: the half-baked `Solution`

The framing of §7.1, "two faces coexist and neither is privileged", is not, at version 0.7.0, a faithful description of the implementation: it is the framing we are working towards.

What is true today is the following:

- `Constraints` and the `Objective` of the abstract representation are *genuinely on demand*: they are constructed only if a `:Solver` that needs them is attached, and the cost of constructing them is paid only when it is needed.
- `Variables` of the abstract representation are *always materialised*, regardless of whether any `:Solver` actually reads them, because every `:Solver` reports its solution by *writing into* the abstract `Variables` of the `Block`. Even an `MCFBlock` attached to an `MCFSolver< MCFSimplex >` that has never asked for the abstract representation must still have a `std::vector< ColVariable >` of size `NArcs` allocated, just so that the `:Solver` has somewhere to deposit the optimal flows.

The asymmetry is visible to any reader of the framework code: it explains why a `:Block`'s `generate_abstract_variables()` is called even when the only `:Solver` registered is specialised, and why a `:Block` whose constructor allocates no `Variable` is nonetheless not the canonical way to build a memory-thin specialised-only `:Block`.

The intended resolution, in a future revision of the API, is to let a `:Solver` return a `Solution` object *directly* and to let the `:Block` *adopt* that `Solution`, instead of writing the solution component-by-component into the abstract `Variables`. Under that scheme a `:Block` attached to a specialised `:Solver` only would be able to skip the `generate_abstract_variables()` step entirely, the `Solver` would hand back a `:Block`-specific `Solution` (such as `MCFSolution`; full discussion in Chapter 9) at the end of `compute()`, and the `Block` would absorb it into its physical representation without intermediation.

The above is one of several places where the SMS++ public API is not yet fully settled at version 0.7.0; the convention adopted by the manual is to flag such places with **Status — under development** and to let the reader know that user code written today should not assume the asymmetry will persist. Chapter 9 returns to this point with the corresponding `Solution`-side discussion.

7.7 Idioms

Build the abstract representation lazily. User code does not need to call `generate_abstract_*()` and `generate_objective()` explicitly; the framework invokes them when the first abstract-using `:Solver` is registered and the corresponding information is requested. Calling them by hand is only useful in two situations: when the user wants the abstract representation to be present for inspection or for the `is_feasible(true)` check, and when the user wants to pass a `Configuration*` that differs from the one carried by the `:Block`'s `BlockConfig`.

Use `is_feasible(false)` for cheap sanity checks. When the solution sitting in the `:Block`'s `Variables` has been put there by a specialised `:Solver`, the physical-path `is_feasible(false)` is both cheaper and more reliable; the abstract path adds nothing beyond what the specialised `:Solver` already knows. Reserve `is_feasi`

`ble(true)` for cases where the abstract representation is the only one available, or for debugging discrepancies between the two.

Treat the abstract `Variables` as the solution channel, not the problem definition. Until the API is reworked along the lines of §7.6, the abstract `Variable`s of a `:Block` are best thought of as "the place where the `:Solver` writes the solution", not as "the syntactic spelling of the problem". When the abstract face of a `:Bloc k` is needed in full (e.g. for an `:MILPSolver` to operate on), the framework constructs it on demand; user code that reads the solution should go through the `:Block`'s physical accessors, not through the `Variable`s.

8. Modification and the Janus discipline

[Source: SMS++/include/Modification.h, Observer.h, MCFBlock/include/MCFBlock.h]

A Block and its abstract representation can both change after a :Solver has been attached to the Block. Chapter 6 introduced the *pending list* of Modifications through which the framework delivers such changes to a :Solver; this chapter makes the machinery precise. It also makes precise the principal idiom by which a :Block keeps its physical and its abstract faces in sync — the **Janus discipline** — and the few ModParam knobs that the user has to dial the behaviour.

8.1 Concept

A Modification is a small immutable object that records a change. It is the notification the framework dispatches when *anything* in a :Block, or in its abstract representation, is modified after a :Solver has been attached. The notification is **lazy** in two distinct senses: it is *issued* lazily by the object that performs the change (the framework does not interrupt the change with synchronous callbacks), and it is *consumed* lazily by the :Solver (which processes its pending list only at the next call to compute()).

Each :Solver that is registered with the Block — or with any ancestor of the Block, recursively — receives a pointer to the Modification, owned by a std::shared_ptr. A Modification is *always* held through shared pointers, and this is precisely why the framework needs no global registry of pending Modifications and no explicit lifetime management: a Modification is automatically deallocated as soon as the last :Solver (or :Block) that has seen it releases its pointer. Each :Solver keeps its own list of the Modifications it still has to react to, and simply drops each entry once it has reacted; when the last interested party has dropped it, the object is gone.

The two basic methods involved are Block::add_Modification(sp_Mod, ChnlName), which routes a fresh Modification to all interested :Solvers, and Solver::add_Modification(sp_Mod, ChnlName), which appends the Modification to the :Solver's pending list. The first is called by the :Block (or by one of its Variable / Constraint / Function instances); the second is called by the framework on the :Solver's behalf.

8.2 The hierarchy of Modifications

The Modification class is the abstract base. Its principal subclasses, all in SMS++/include/Modification.h, are:

- Modification (line 357) — the abstract base. Carries a pointer to the :Block that originated the change (get_Block()), a concerns_Block() flag (default false), and the channel the Modification is sent on. Concrete subclasses encode *what* changed.
- AModification (line 451) — the abstract "abstract" Modification. This is the base class for every Modification issued *by the abstract representation*: a Variable being fixed, a Constraint being relaxed, a Function's coefficient being changed, and so on. AModification is exactly the kind of Modification whose concerns_Block() is true by default, to signal to the receiving :Block that it should react.
- NModification (line 522) — "nuclear" Modification. Used as a base for the few cases where the entire content of an object is replaced.
- NBModification (line 572) — "nuclear Block" Modification: issued by Block::load(...) and by derialize(...) to signal that the entire Block has been re-loaded from scratch and any prior computation must be discarded. This is the "nuclear option" of §4.6.
- GroupModification (line 635) — a GroupModification bundles several Modifications into one logical unit. A receiving :Solver may either react to each sub-Modification in turn or treat the whole group as

a single atomic change. This is the principal way to express that several individual changes have happened together and should be reacted to together.
- `VariableGroupMod` (line 778) — a `GroupModification` that specifically groups `Variable`-related Modifications; rarely used directly by user code, but produced by some `:Block` operations that touch multiple `Variable`s at once.

Each concrete `:Block` typically defines its own `:Modification` subclasses for the changes that its physical representation supports. For `MCFBlock`, the classes are:

- `MCFBlockMod` (`MCFBlock.h:2275`), the base for all `MCFBlock`-specific physical changes;
- `MCFBlockRngdMod` (`MCFBlock.h:2355`), a *range-based* change: a contiguous range of arcs or nodes has had its costs, capacities, or deficits modified, or has been added or removed;
- `MCFBlockSbstMod` (`MCFBlock.h:2406`), a *subset-based* change: an arbitrary subset of arcs or nodes has been similarly modified.

Analogous Rngd / Sbst pairs exist for `BinaryKnapsackBlock` (`BinaryKnapsackBlockMod`, `BinaryKnapsackBlockRngdMod`, `BinaryKnapsackBlockSbstMod`) and for `CapacitatedFacilityLocationBlock` and most other concrete `:Block` classes.

8.3 The two streams

There are two parallel streams of `Modification`s, both arriving at the same list inside each `:Solver`, distinguished by what they say about themselves.

The **physical** stream consists of `Modification`s issued by the `:Block`'s own mutation methods (the `chg_*()`, `fix_x()`, `close_arc()`, `add_arc()`, ... family). These `Modification`s descend directly from `Modification`, *not* from `AModification`; their `concerns_Block()` returns `false`, because the change in the `:Block` has *already happened* by the time the `Modification` is constructed. A specialised `:Solver` consumes these and reacts by recomputing what it can. An `MCFBlock` cost-change, for instance, produces an `MCFBlockRngdMod` with the indices of the changed arcs and the old / new costs (`MCFBlock.h:1665-1693`).

The **abstract** stream consists of `Modification`s issued by the `:Block`'s abstract representation when something inside it changes — when a `Variable` is fixed, when a `Constraint` is relaxed, when a `Function`'s coefficient is updated, when a dynamic `Constraint` is added or removed. These all descend from `AModification`; their `concerns_Block()` returns `true` by default. The `true` value of `concerns_Block()` is precisely the flag that tells the receiving `:Block` "*you* are also expected to react to this `Modification` by mutating your physical representation accordingly", as discussed in §8.4.

A general-purpose `:Solver` such as `:MILPSolver` consumes the abstract stream and is essentially indifferent to the physical one (it does not know what to do with an `MCFBlockRngdMod` and would simply discard it). Conversely, a specialised `:Solver` typically consumes the physical stream and may safely discard the abstract `Modification`s issued by a change that has produced both (because reacting to the same change twice would be incorrect).

8.4 The Janus discipline

A change to a `:Block`'s data may originate from either face:

- The user calls a `:Block`-specific mutator, such as `MCFBlock::chg_cost(NCost, arc, issueMod, issueAMod)`. This changes the physical data, issues an `MCFBlockRngdMod` (the physical `Modification`), and — if `issueAMod` says so — *also* updates the corresponding coefficient in the linear `Function` inside `f_obj`. The update of the abstract representation is itself a change to a `Variable`-dependent `Function` and will normally trigger an abstract `C05FunctionMod` of its own; the framework ensures that the abstract `Modification` carries `concerns_Block() == false` in this scenario, because the `:Block` has obviously already applied the change and need not be told to do so again.
- The user, or a `:Solver`, calls an abstract-side mutator directly, such as `x[ a ].is_fixed( true , eModBlck )` to fix the flow on arc `a` to zero (idiomatic abstract-side way to *close* the arc), or an analogous mutator on a `Function` carrying the abstract `Constraint`/`Objective` data. This changes the abstract representation; the framework constructs the corresponding abstract `Modification` and calls `Block::add_Modification(sp_Mod)` to dispatch it. The `Modification` carries `concerns_Block() == true`. The concrete `:Block`'s override of `add_Modification(...)` is expected to *intercept* it: detect that this is a

kind of change it knows how to translate to its physical representation, apply the corresponding physical change, and issue the corresponding physical `Modification` *before* forwarding the abstract `Modification` to the listening `:Solver`s. The abstract `Modification` is forwarded with `concerns_Block()` rewritten to `false` (it has done its duty of informing the `:Block`).

This is the **Janus discipline**: a `:Block` has two faces and each face is responsible for telling the other when it has changed. The mechanism is symmetric in the user-visible interface (a user can change either face and the other will adapt) but deliberately *not* symmetric internally — each change generates *both* a physical and an abstract `Modification`, so that both specialised and general-purpose `:Solver`s see exactly what they need to see.

The flow seen from the `:Block`

When the change originates on the abstract face, the sequence of events at the `:Block` is precise and worth spelling out, because it is exactly the recipe a `:Block` author has to follow (see also Appendix A) and the canonical override scheme documented at `Block.h:5394-5402`:

1. *Someone modifies the abstract representation* — a `Variable` is fixed, a `Constraint`'s bound is changed, a coefficient of a `Function` carrying an abstract `Constraint`/`Objective` is updated.
2. *An abstract `Modification` is generated*, with `concerns_Block() == true`, and `Block::add_Modification(...)` is invoked with it.
3. *The concrete `:Block` reacts*: its `add_Modification(...)` override detects `concerns_Block() == true`, immediately sets it to `false`, and applies the matching change to its physical representation.
4. *The (now `concerns_Block() == false`) abstract `Modification` is forwarded onwards* — to the `:Block`'s father and to the listening `:Solver`s — but only if there is anyone listening (cf. §8.7).

The reaction in step 3 typically consists of calling one or more of the `:Block`'s own physical `chg_*()` mutators, with the two `ModParam` arguments set as follows:

- `issueMod = eNoBlck` — perform the physical change and issue the physical `Modification`, but *only if there is anyone listening*, and with `concerns_Block() == false` (the abstract side has nothing to react to);
- `issueAMod = eDryRun` — do *not* touch the abstract representation again, because the change there has already happened (it is what triggered this whole sequence); issuing another abstract `Modification` would double-count the change.

In short: an abstract-origin change becomes, inside the `:Block`, a physical change issued with `(eNoBlck, eDryRun)`. A physical-origin change, conversely, is issued with `(eModBlck, eModBlck)` (the defaults) and the abstract side is updated as a genuine second action.

Why the `:Block` sees the abstract `Modification` immediately

A subtle but important property holds:

> the abstract `Modification` reaches its `:Block` *immediately*, before it can be packed into any `GroupModification`.

This matters. If the abstract `Modification` could be buffered — inserted into an open `GroupModification` (see §8.6) and only delivered later, when the group is closed — then by the time the `:Block` finally "saw" it, the abstract representation could have undergone arbitrarily many further changes. The `Modification` might, for instance, refer to a *dynamic* `Variable` that has since been deleted, forcing the `:Block` to carry logic to detect whether that has happened and to cope with it.

The framework rules this out. As documented at `Block.h:5404-5427`, the `:Block`'s `add_Modification(...)` override sees the abstract `Modification` "naked" — *before* the base-class machinery packs it into any `GroupModification`, even when the `Modification` is being sent to a non-zero channel defined in the `:Block`. Moreover, an abstract change must be performed while the `:Block` is `lock()`-ed (Chapter 17), so no other thread can be mutating it concurrently. The combined effect is that

> when a `:Block` processes an abstract `Modification`, the state of its data structures is *exactly* the state in which the change was issued: the change has just happened, and nothing else can have happened in the meantime.

This is a substantial simplification: a *leaf* `:Block` (with no sub-`Block`) never has to handle an abstract `GroupModification` at all, and never has to reason about "what else might have changed since". It only ever has to react to one atomic abstract `Modification` at a time, applied to a data structure that is in the precise state the `Modification` describes. A non-leaf `:Block` has to handle abstract `GroupModification`s only insofar as they refer to changes inside its sub-`Block`s.

Refusing an abstract change

A `:Block` is free to *refuse* an abstract change it does not know how to translate. `MCFBlock`, for instance, does not allow adding or removing arcs through the abstract representation (such an operation would also require changing the structure of the flow-conservation `FRowConstraint`s, which is more complex than the current `add_Modification` implementation supports). The refusal is concrete: the override throws a `std::logic_error` with a descriptive message. The convention is that *failing loud* is the safe default; a `:Block` that wants to support a particular abstract change must do so explicitly.

8.5 The four `ModParam` values

Every mutator that may issue a `Modification` takes one or two `ModParam` arguments — `issueMod` for the physical `Modification`, `issueAMod` for the abstract one — with default value `eModBlck`. The enum `amododification_type` (`Modification.h:908-913`) takes four values:

`ModParam`	Numeric	Semantics
`eDryRun`	0	The mutator must **not** perform the change, and consequently must not issue any `Modification`. Useful when a method that normally changes both the physical and the abstract face is called in a context where the change has already been applied to one face by some other means; passing `eDryRun` to the other face's parameter switches that face off.
`eNoMod`	1	The mutator **performs** the change but issues **no** `Modification`. Should be used only when the user is certain that neither the `:Block` nor any `:Solver` will ever need this information (typical case: setting up a fresh `:Block` from scratch, before any `:Solver` has been attached).
`eNoBlck`	2	The mutator performs the change and issues the `Modification`, *but only if there is anyone listening* (cf. `anyone_there()`); furthermore, `concerns_Block()` is set to `false`, telling the receiving `:Block` (if any) that it must not re-apply the change to its physical representation. This is the value the framework uses internally when forwarding an abstract `Modification` after the `:Block` has already intercepted and applied it.

ModParam	Numeric	Semantics
eModBlck	3	The mutator performs the change and issues the Modification unconditionally, with concerns_Block() set to true. This is the default, "most conservative" value: it ensures that everybody who may need to know is informed, and it makes the abstract Modification carry the flag that asks the :Block to also apply the change to its physical face if needed.

The two parameters issueMod and issueAMod are independent and both default to eModBlck. They can also carry a *channel name* encoded in the upper bits (Observer::make_par(iM, chnl)), which is the mechanism for routing related Modifications to a specific channel so that they can be grouped; channels are the subject of §8.6. In normal use both parameters default to channel 0 and the channel machinery need not be touched.

8.6 Channels and GroupModification

So far every Modification has been treated as if it were dispatched the instant it is issued. That is the default, but it is not the only possibility: SMS++ provides a mechanism to *batch* a set of logically related Modifications and deliver them together, as a single GroupModification. The mechanism is built on **channels** ([Source: Observer.h:303-374, Block.h:5429-5442]).

Opening and closing a channel

Whoever is about to make a sequence of related changes can *open a channel* on a :Block:

```
ChnlName c = blk.open_channel();      // open a fresh channel
//  ... make several related changes, all passing channel c ...
blk.close_channel( c );               // close it: the group is shipped
```

open_channel() (with the default argument 0) creates a fresh GroupModification, gives it a unique name, and returns that name as the ChnlName c. From that moment, any Modification issued *on channel* c is **not** dispatched to the :Solvers and the ancestor :Blocks; instead it is appended to the GroupModification. The accumulated group is dispatched as one object only when close_channel(c) is called.

A Modification is issued "on channel c" by passing c to the ModParam argument of the mutator, encoded via Observer::make_par(issueMod, c). Alternatively, a :Block's *default channel* can be set once with set_default_channel(c), so that subsequent mutators called with a plain eModBlck / eNoBlck are automatically routed to c.

Nesting

The mechanism is recursive. Calling open_channel(c) with the name c of an already-open channel does *not* open an independent channel: it *nests* a new GroupModification inside the one currently associated with c. Subsequent Modifications on c go into the inner group. close_channel(c) then closes only the innermost group and "returns control" to the one immediately above; the accumulated content stays inside the outer group. Only when the *root* GroupModification (the one created by the original open_channel(0)) is finally closed is the whole nested structure dispatched. This allows the construction of arbitrarily tree-nested groups of related Modifications.

open_channel() also accepts an optional pointer to a GroupModification (or to a derived class of it): if provided, that object is "adopted" as the group being opened, which lets the caller attach extra information that a :Solver or :Block may find useful when processing the group.

What it is for

The purpose is to make a `:Solver`'s life easier when several changes are *correlated* and are best reacted to together. The prototypical example is changing all the coefficients of a single "column" of a model: those coefficients live in several different `Constraints` (in the `Functions` carrying them), so changing the column produces one `Modification` per `Constraint`. A `:Solver` that receives them one at a time may redo expensive bookkeeping once per `Modification`; a `:Solver` that receives them as a single `GroupModification` can recognise the pattern and react once, far more efficiently. `BundleSolver`, for instance, exploits `GroupModifications` in exactly this way.

This mechanism is, frankly, **not used as much as it could be**, but it exists and is available to any `:Solver` and any code that mutates a `:Block`.

The cost of batching

The downside of batching is the very property that §8.4 showed to be absent for the abstract `Modification` reaching its own `:Block`: when `Modifications` are buffered into a `GroupModification`, a `:Solver` may end up "seeing" many of them at once, possibly long after the first was issued and after the underlying objects have undergone many further changes. A `:Solver` that consumes `GroupModifications` therefore generally needs more complex logic than one that consumes naked `Modifications`: it must be prepared for the fact that, by the time it processes a buffered `Modification`, the relevant `Variable`/`Constraint` may already have been further changed (or even, for dynamic ones, removed). This is exactly the complication that the "immediate delivery to the originating `:Block`" guarantee of §8.4 spares a `:Block` from — but a `:Solver` that opts into channel batching does not get that guarantee for free, and must handle the accumulated group with the appropriate care.

8.7 The "anyone listening" filter

A `:Block` with no `:Solver` attached and no `:Solver` attached to any ancestor is, by construction, silent: no one is listening and no one will react to any `Modification`. Issuing a `Modification` in that case is wasted work.

The framework caches the boolean answer to "*is anyone listening?*" in the `f_at` field of `Block`, updated by `register_Solver(...)` / `unregister_Solver(...)` and propagated recursively. The query method is `anyone_there()`. Mutators that take `issueMod = eNoBlck` consult the cache before constructing the `Modification`: if `anyone_there()` is `false`, the `Modification` is silently dropped. Mutators called with the default `eModBlck` do not consult the cache: they issue the `Modification` anyway, because the `concerns_Block() == true` flag may still need to reach the `:Block` itself even when no `:Solver` is listening (for instance, to apply the corresponding physical change in the Janus discipline).

A concrete `:Block` may override `anyone_there()` to return `true` unconditionally whenever its abstract representation is constructed (because abstract `Modifications` must then flow regardless of the registered `:Solvers`, to keep the two faces in sync). `MCFBlock` does exactly this; `BinaryKnapsackBlock` also has the analogous override.

8.8 Inline example: a cost change in `MCFBlock`, observed from both faces

The example below changes a cost in an `MCFBlock` via the physical interface, then via the abstract interface, and observes the resulting `Modifications` by attaching a simple "tap" `:Solver` whose only role is to record everything that arrives in its pending list.

```
#include "MCFBlock.h"
#include "FakeSolver.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 // construct and load an MCFBlock (as in §4.5)
 MCFBlock mcf;
 mcf.load( 3 , 3 ,
           MCFBlock::Subset      { 1, 2, 2 } ,   // pEn
```

```
          MCFBlock::Subset      { 0, 1, 0 } ,   // pSn
          MCFBlock::Vec_FNumber { 3.0, 3.0, 3.0 } ,  // pU
          MCFBlock::Vec_CNumber { 2.0, 3.0, 4.0 } ,  // pC
          MCFBlock::Vec_FNumber { -2.0, 0.0, +2.0 } );

 // attach a tap: FakeSolver simply accumulates every Modification
 // it receives without ever doing anything with them. This is the
 // canonical way to observe the Modification stream.
 auto * tap = new FakeSolver();
 mcf.register_Solver( tap );

 // make sure the abstract representation is built, so that the
 // abstract stream will also be visible.
 mcf.generate_abstract_variables();
 mcf.generate_abstract_constraints();
 mcf.generate_objective();

 // ---- physical-side change: lower the cost of arc 0 ----
 mcf.chg_cost( /* NCost = */ 1.0 , /* arc = */ 0 ,
               /* issueMod  = */ eModBlck ,
               /* issueAMod = */ eModBlck );
 // The framework issues:
 //   - one physical MCFBlockRngdMod with rng = [0, 1) and the
 //     new cost vector,
 //   - one abstract CO5FunctionMod (with concerns_Block() == false)
 //     reflecting the change in the linear Function inside f_obj.

 // ---- abstract-side change: close arc 1 by fixing its flow to 0 ----
 // The idiomatic abstract-face counterpart of MCFBlock::close_arc(1)
 // is fixing the corresponding ColVariable to 0. This is a pure
 // abstract change; the framework intercepts it and applies the
 // matching physical operation.
 ColVariable * x1 = mcf.get_Var( /* arc index = */ 1 );
 x1->set_value( 0.0 );
 x1->is_fixed( true , eModBlck );
 // The framework issues:
 //   - one abstract VariableMod with concerns_Block() == true,
 //     which mcf.add_Modification() intercepts;
 //   - mcf then performs the corresponding physical close_arc(1),
 //     issues a physical MCFBlockRngdMod marking the arc as
 //     closed, and forwards the abstract Modification with
 //     concerns_Block() rewritten to false.

 // examine the tap's pending list
 for( auto & mod : tap->get_Modifications() )
  std::cout << *mod << '\n';

 mcf.unregister_Solver( tap , /* deleteold = */ true );
 return 0;
}
```

The salient observations:

1. **Both changes produce two `Modifications`.** The framework guarantees this so that listeners on either face see what they need. A pure-physical listener (an `MCFSolver`) ignores the abstract one; a pure-abstract listener (an `:MILPSolver`) ignores the physical one.
2. **The Janus discipline is invisible from the user's vantage point.** Whether the change came through the physical mutator `chg_cost(...)` or through an abstract-face mutator such as `ColVariable::is_fixed(true, eModBlck)`, both faces end up consistent and both streams are populated.
3. **`concerns_Block()` is `true` only on abstract `Modifications` that have not yet been digested by the originating `:Block`.** After the `:Block`'s `add_Modification(...)` override has run, the flag is reset to `false` before the `Modification` is forwarded to listeners.
4. **`FakeSolver`** is a `:Solver` that does nothing except recording the `Modifications` it receives; the framework provides it as a debugging aid and as a convenience for code that needs to *defer* reacting (e.g. while a

sub-`Block` is being modified in bulk and the changes will be replayed later).

8.9 Idioms

Default to `eModBlck`. The default value of `issueMod` and `issueAMod` is `eModBlck` for a reason: it is the safest. Anything else is an optimisation that should be applied only when the user has thought through what the listeners need.

Use `eNoMod` only when no listener exists yet. The natural place to pass `eNoMod` is during the initial construction of a `:Block`, before any `:Solver` has been registered: the absence of listeners makes the `Modifications` useless anyway. After any `:Solver` has been attached, `eNoMod` becomes dangerous (a `:Solver` that does not learn about the change will produce stale results).

Use `eDryRun` to suppress one face in a coordinated update. The canonical use of `eDryRun` is inside the `:Block`'s own `add_Modification(...)` override: when an incoming abstract `Modification` has applied a change to the abstract face and the `:Block` needs to update the physical face accordingly, it calls the physical mutator with `issueMod = some_value, issueAMod = eDryRun` to say "do the physical change, do not redo the abstract one (it is already done)".

Group related changes. When a single logical operation produces several individual `Modifications`, the canonical idiom is to bundle them into a `GroupModification`. A `:Solver` may then react to the group as a unit, which is often more efficient than reacting to each component in turn.

8.10 Forward reference to `Change`

The current `Modification` design has one notable limitation: `Modifications` carry enough information to *describe* a change but not always enough to *replay* it on a different `:Block`, and they cannot in general be serialised in a form that survives an inter-process boundary. The `Change` concept (Chapter 16) is the framework's response: a `Change` carries the data needed to apply the corresponding change to any copy of a `:Block`, can be `[de]serialize`-d to netCDF as a plain object, and (where the `:Block` chooses to support it) can be *undone* via an automatically-generated inverse `Change`. The intended downstream use is message-passing distributed computation; only a small number of `:Change` classes exist at version 0.7.0 (in particular for `BinaryKnapsackBlock` and `SATBlock`) and the interface is subject to change. **Status — beta.**

9. Solution

[Source: SMS++/include/Solution.h, ColVariableSolution.h, RowConstraintSolution.h, ColRowSolution.h; MCFBlock/include/MCFBlock.h (MCFSolution)]

9.1 Concept

A Solution is an object that stores a solution of a Block — typically the values of the Block's Variables and, where applicable, the dual values attached to its Constraints — so that the solution can be saved, restored, serialised, or combined with other solutions. The cardinal design fact is:

> a Solution is **Block-specific**, not Solver-specific.

The same Block produces the same kind of Solution regardless of which :Solver computed it. A min-cost flow solution is an MCFSolution whether it was found by an MCFSolver< MCFSimplex > or by a general-purpose :MILPSolver; the Solution knows how to read itself out of an MCFBlock and write itself back into one, and it does not care how the values got there.

The two methods at the heart of the interface are:

- read(const Block*) — read the current solution out of the given Block and store it inside this Solution;
- write(Block*) — write the stored solution back into the given Block.

A Solution is normally constructed *by* the Block whose solution it stores, through Block::get_Solution(Configuration*, bool). The Configuration* argument lets the caller request a *specific part* of the solution — only the primal values, only the dual values, only a particular subset of Variables — and this choice is *permanent*: once constructed, the Solution object will only ever store that part. Asking a Solution to read() information it was not configured to hold (or that the Block does not currently have, e.g. dual values when the Constraints have not been generated) is an error and throws.

9.2 Whose Block a Solution belongs to

A Solution is tightly bound to the Block that created it. It is an error — and throws an exception — to read() or write() a Solution against the *wrong* Block. "Wrong" here means more than "wrong type": the safe contract is that a Solution is used only with the very Block that created it, or with a Block that is "identical" to it (for instance, a copy produced as an R3 Block; Chapter 10), or, at the very least, one that is "compatible" — same sizes in the relevant Variable / Constraint groups.

Solutions are *not* meant to be exchanged between distinct Blocks, even of the same type. A :Solution class that does allow such an exchange must say so explicitly in its own documentation, and the exchange must be used with care.

9.3 Serialisation and combination

Beyond read / write, the base Solution interface provides:

- **Serialisation to netCDF**: serialize(netCDF::NcGroup&), serialize(const std::string& filename, bool replace), and the matching deserialize(const netCDF::NcGroup&). A Solution can be written to disk and read back independently of its Block; the Solution factory (Solution::new_Solut

ion(name)) reconstructs the correct :Solution type from the class name stored in the netCDF group, exactly as the Block and Solver factories do for their respective hierarchies (Chapter 18).
- **Cloning**: clone(bool empty) produces a copy of the Solution (or an empty one of the same type and configuration, if empty == true).
- **Linear combination**: scale(double factor) returns a scaled copy and sum(const Solution*, double multiplier) adds a scaled solution into this one. Together they let one form weighted sums — in particular *convex combinations* — of solutions of the same Block. Convex combinations are central to many optimisation techniques (the convexified primal solution produced by a Lagrangian dual, for example; Chapter 14 and Recipe R4), which is why the operation is part of the base interface.

A caveat applies to scale / sum: not every Solution can be meaningfully scaled or summed. A *discrete* Solution (one whose stored values are integral by nature) may refuse to be combined, or may produce a "more general" Solution that no longer carries the integrality. The base-class comments to scale() and sum() (Solution.h:417-469) describe the contract; a :Solution is free to convert itself, on demand, into a "more general" Solution type when a combination would otherwise be impossible.

9.4 The standard abstract Solutions

Three concrete :Solutions in the core library work off the *abstract* representation of a Block and are therefore reusable across any :Block whose abstract representation is of the right shape:

- ColVariableSolution stores the *primal* solution of any Block whose Variables are all ColVariables: it reads and writes their values.
- RowConstraintSolution stores the *dual* solution of any Block whose Constraints are all RowConstraints: it reads and writes their dual values.
- ColRowSolution stores both at once.

These are the "default" Solutions: a Block that has not defined a :Solution of its own, but whose abstract representation fits one of these shapes, can use them directly.

9.5 The Block-specific physical Solutions

A concrete :Block typically defines its own :Solution class that stores the solution in terms of the *physical* representation, which is usually far more compact than the abstract one. The three running examples each ship one:

- MCFSolution (declared in MCFBlock.h:2508) stores the arc flows and, optionally, the node potentials and arc reduced costs — the natural primal and dual data of a min-cost flow problem.
- BinaryKnapsackSolution stores the per-item values and, where meaningful, the dual value of the knapsack constraint.
- CapacitatedFacilityLocationSolution stores the design (y) and transportation (x) values, with a Configuration-selectable choice of which part to keep.

A Block-specific physical Solution reads and writes the Block's physical data directly, without needing the abstract representation to have been constructed. This is the property that makes it the right vehicle for the API change discussed in §9.7.

9.6 Inline example: a CFL solution to netCDF and back

```
#include "CapacitatedFacilityLocationBlock.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 CapacitatedFacilityLocationBlock cfl;
 cfl.load( /* ... a CFL instance ... */ );

 // solve cfl with some Solver (omitted; see Recipe R3) so that a
 // solution sits in cfl ...
```

```
 // ask the Block for a Solution object holding the current solution
 Solution * sol = cfl.get_Solution();   // a CapacitatedFacilityLocationSolution

 // persist it to a netCDF file
 sol->serialize( "cfl-solution.nc4" );

 // ... later, possibly in another run ...
 Solution * sol2 = cfl.get_Solution( nullptr , /* emptys = */ true );
 sol2->deserialize( /* the NcGroup read from "cfl-solution.nc4" */ );

 // restore the stored solution into the Block
 sol2->write( & cfl );

 // a convex combination of two solutions of the SAME Block
 // (here: 0.5 * sol + 0.5 * sol2), useful e.g. for rounding heuristics
 Solution * mix = sol->scale( 0.5 );
 mix->sum( sol2 , 0.5 );

 delete sol; delete sol2; delete mix;
 return 0;
}
```

The points to take away: `get_Solution()` returns a `Solution*` that is in fact a `CapacitatedFacilityLocationSolution` (the return type is the base `Solution*` for the usual reason that the `:Solution` cannot be forward-declared at that point); the `Solution` round-trips through netCDF without any reference to the `Block`; and `scale` / `sum` combine solutions of the *same* `Block` — never of different ones.

9.7 Status — under development: closing the half-baked `Solution`

This chapter is the natural home of the discussion opened in §3.3 and continued in §7.6: the *intended* role of `Solution` in relation to the abstract `Variable`s.

The relevant facts, restated here in full:

- Every `:Solver`, today, reports its result by *writing the solution into the abstract* ***`Variable`s*** of the `Block`. After `compute()` returns `kOK`, the optimal values sit in the `ColVariable`s; the user (or the `Block`) reads them from there, or calls `get_Solution()` to snapshot them into a `Solution` object.
- Because the abstract `Variable`s are the channel through which the solution is communicated, they must *always* be materialised — even for a `Block` solved exclusively by a specialised `:Solver` that reads only the physical representation and would otherwise have no need for them. This is the asymmetry flagged in §7.6: `Constraint`s and `Objective` are genuinely on demand, but `Variable`s are not.

The `Solution` machinery of this chapter is precisely what makes the *intended* fix possible. Under the planned revision of the API, a `:Solver` would, at the end of `compute()`, hand back a `:Block`-specific `Solution` object — an `MCFSolution`, a `BinaryKnapsackSolution`, … — and the `Block` would *adopt* it, absorbing the solution directly into its physical representation. The abstract `Variable`s would then no longer be the obligatory communication channel; a `Block` attached to a specialised `:Solver` only would be able to skip `generate_abstract_variables()` entirely, holding only the physical data and the `:Solution` that carries the result.

The `Block`-specific physical `Solution`s of §9.5 are designed with exactly this in mind: each can read and write the `Block`'s physical data without touching the abstract representation, which is the prerequisite for letting a `Block` be solved, and its solution communicated, with no abstract `Variable`s in sight.

Until that revision lands, the convention to follow in user code is the one stated in §7.7: treat the abstract `Variable`s as the present-day solution channel, read the solution back through the `Block`'s physical accessors or through a `Solution` object obtained from `get_Solution()`, and do not write code that assumes the `Variable`s-as-channel arrangement will persist. This remains one of the clearest signs that the SMS++ public interface is not yet fully settled at version 0.7.0. **Status — under development.**

9.8 API outline

Method	Purpose
`read(const Block*)`	snapshot the current solution of the `Block` into this `Solution`
`write(Block*)`	write the stored solution back into the `Block`
`serialize(NcGroup&)` / `serialize(filename, replace)`	persist to netCDF
`deserialize(NcGroup&)`	restore from netCDF
`clone(bool empty)`	copy (or empty copy) of the `Solution`
`scale(double)`	scaled copy
`sum(const Solution*, double)`	add a scaled `Solution` into this one
`Solution::new_Solution(name)`	factory: construct a `:Solution` by class name

The `Block`-side entry point is `Block::get_Solution(Configuration* solc, bool emptys)`: `solc` selects which part of the solution to keep (permanently), and `emptys == true` prepares the object without immediately reading a solution.

9.9 Idioms

Get the `Solution` from the `Block`, not from the `Solver`. The canonical place to obtain a `Solution` is `Block::get_Solution()`. The `Block` knows which `:Solution` type fits it; the `:Solver` is merely the agent that put the values in place.

Configure once, at construction. The part of the solution a `Solution` stores is fixed when the object is created. To store a different part, create a different `Solution` object with a different `Configuration`; do not expect to reconfigure an existing one.

Combine only solutions of the same `Block`. `scale` and `sum` are defined for solutions of one and the same `Block`. Combining solutions of different `Block`s — even of the same type — is outside the contract and will, at best, throw.

10. R3Block: Reformulation, Relaxation, Restriction

[Source: `SMS++/include/Block.h` (R3 Block methods), `UpdateSolver.h`; `CapacitatedFacilityLocationBlock/include/CapacitatedFacilityLocationBlock.h`]

10.1 Concept

A central design objective of SMS++ is that a model may need to be *transformed* for algorithmic reasons, producing a different `Block` that is related to the original in one of three ways ([Source: `Block.h:67-85`]):

- a **Reformulation** — a different `Block` that encodes a problem whose optimal solutions are optimal also for the original `Block` (the two are equivalent, but one may be easier to work with);
- a **Relaxation** — a different `Block` whose optimal value provides a valid global lower bound (for a minimisation problem; upper bound for maximisation) on the optimal value of the original, while hopefully being easier to solve;
- a **Restriction** — a different `Block` whose feasible region is a strict subset of that of the original, which again may make it easier to solve (at the price of possibly cutting off the true optimum).

The three transformations are collectively called the **R3 Block** of the original `Block` (Reformulation / Relaxation / Restriction). The set of R3 Blocks a given `Block` supports is decided by the `Block` itself; the base `Block` class provides no general R3 Block. A concrete `:Block` that wishes to support one must implement it explicitly — there is no automatic mechanism ("forget 'auto' here", as the slide decks put it).

10.2 Why an R3 Block, and not a copy of the `Variables`

The need for a dedicated mechanism, rather than "just copy the relevant `Variables` and `Constraints`", comes straight from the identity rule of §4.2: the *name* of a `Variable` is its memory address, so a copy of a `Variable` is, necessarily, a *different* `Variable`. An R3 Block therefore lives in its own, fresh set of `Variables` and `Constraints`, entirely disjoint from those of the original. This is exactly what makes an R3 Block useful for algorithmic purposes (branch, fix, relax, solve a separate copy in a separate thread, ...): it is genuinely independent, and operating on it does not perturb the original.

But independence creates a problem: a solution found on the R3 Block lives in the R3 Block's `Variables`, and is of no direct use to a `:Solver` working on the original. The framework's answer is a small family of *mapping* methods that move solution and modification information back and forth between an original `Block` and one of its R3 Blocks.

10.3 The R3 Block API

All of the following are virtual methods of `Block`; a concrete `:Block` overrides the ones it chooses to support and leaves the rest to throw.

- `get_R3_Block(Configuration* r3bc, Block* base, Block* father)` produces an R3 Block. The `Configuration*` selects *which* R3 Block (a `:Block` may offer several); the `base` and `father` parameters are discussed in detail below.

- `map_back_solution(Block* R3B, Configuration* r3bc, Configuration* solc)` takes the solution currently held in the R3 Block `R3B` and writes the corresponding solution into the original `Block`. The `solc` selects which part of the solution to map (primal, dual, both).
- `map_forward_solution(Block* R3B, ...)` is the reverse: it takes the solution of the original `Block` and writes it into the R3 Block.
- `map_forward_Modification(Block* R3B, c_p_Mod mod, ...)` takes a `Modification` that occurred on the original `Block` and applies the corresponding change to the R3 Block, so that the two stay in sync. `map_forward_Modifications(...)` does the same for a whole list.
- `map_back_Modification(Block* R3B, c_p_Mod mod, ...)` is the reverse direction.

In every case the `Configuration* r3bc` passed to a mapping method must be the *same* (or an identical) `Configuration` that was used to produce `R3B` in `get_R3_Block`, so that the `Block` knows which kind of R3 Block it is dealing with.

A `:Block` is, as always, free to support only part of this interface. The base-class default `get_R3_Block` (with a null `Configuration`) recursively produces a copy by asking each sub-`Block` for its own copy, but a leaf `:Block` must implement even the copy itself.

The `base` and `father` parameters

The two trailing parameters of `get_R3_Block` exist to make the construction of an R3 Block composable along a class hierarchy and along a `Block` tree.

`base` lets the construction happen *piecemeal*. Consider a class `B1 : Block` that already provides some R3 Block — say, the copy — and a derived class `B2 : B1`. It is sensible for `B2` to want to offer the same R3 Block, and wasteful for it to re-implement the part that `B1` already knows how to handle. The pattern the `base` parameter supports is: `B2::get_R3_Block` allocates the R3 Block (an object of class `B2`, in the copy case), then calls `B1::get_R3_Block(r3bc, that_object)` to have the `B1`-specific part of the new object managed (copied) by `B1`, while `B2` takes care of only the `B2`-specific part. In other words, `base` is "an existing object that the method should populate, rather than allocate from scratch". A `:Block` that accepts a `base` will have requirements on its concrete type, depending on `r3bc`: for the copy, `base` must be of a class derived from the one whose `get_R3_Block` is being called (any `B1`-or-derived for `B1::get_R3_Block`, so a `B2` qualifies).

`father` is used in the converse situation, when `base` is *not* supplied and the R3 Block therefore has to be allocated inside the method. In that case `father` is the pointer to the father `Block` of the newly created R3 Block, to be passed to its constructor so that the R3 Block is correctly nested in the surrounding tree. (`Block` is abstract and cannot itself be instantiated, which is why the base-class default implementation requires a non-null `base` whenever the `Block` has inner `Block`s: it has no way to allocate the right concrete type otherwise; `Block.h:4761-4775`.)

Dynamic `Variable`s and `Constraint`s in an R3 Block

A subtlety worth stating explicitly concerns *dynamic* `Constraint`s and `Variable`s (§4.3), because their semantics interacts with R3 Blocks in a way that is easy to get wrong.

A dynamic `Constraint` is conceptually "there even when it is not there": it contributes to defining the feasible region of the `Block`, so *every* feasible solution must satisfy *all* of them, whether or not they have been explicitly constructed. The reason not all of them are materialised is purely operational — there can be very many, and only the subset that is useful for algorithmic reasons (the ones a separation procedure has found to be violated, say) is explicitly generated. Likewise, a dynamic `Variable` is "there even when it is not there": every dynamic `Variable` that has not been explicitly generated is assumed to be present in the `Block` at its default value (most often zero), under the assumption that this does not invalidate the feasibility of the explicitly-constructed part of the solution. Only the dynamic `Variable`s actually needed to encode the optimal solution need be materialised.

The consequence for R3 Blocks is that generating dynamic `Variable`s/`Constraint`s may require complex separation or pricing procedures. These are surely available to the *original* `Block`, but may or may not be available to the *R3* `Block`. If they are, the R3 Block may generate dynamic stuff independently and have it "imported back" into the original via `map_back_Modification` (`Block.h:4750-4759`). If they are not, dynamic `Variable`s/`Constraint`s can only be generated on the original `Block` and then, where supported, pushed forward to the R3 Block via `map_forward_Modification`. A `:Block` author who provides an R3 Block over a model with dynamic components must decide, and document, which of the two directions its R3 Block supports — and `map`

`_back_solution` is, accordingly, allowed to do a *best-effort* job when the R3 Block holds dynamic components the original has not (yet) generated.

10.4 The trivial case: the copy

The simplest R3 Block is the **copy** (clone): a new `Block` of the same type, holding the same data, in its own fresh `Variable`s and `Constraint`s. The copy is a Reformulation in the trivial sense (it encodes exactly the same problem) and "should always work" for any reasonable `:Block`. Even so, the copy is not free: because of the identity rule, the copy's `Variable`s are distinct objects, and `map_back_solution` from the copy to the original is a genuine value-by-value transfer, not a pointer aliasing.

For `MCFBlock`, `get_R3_Block(nullptr)` produces a copy (`MCFBlock.h:1096-1105`); `map_back_solution` / `map_forward_solution` move the flows, potentials and reduced costs between the two, with a `solc` selecting "primal only" / "dual only" / "both".

10.5 The non-trivial case: CFL's MCF flow relaxation

The instructive R3 Block is the one offered by `CapacitatedFacilityLocationBlock`: besides the copy, it can produce a **`MCFBlock` that represents the continuous flow relaxation** of the CFL problem (`CapacitatedFacilityLocationBlock.h:1048-1099`). The `Configuration*` selects it:

- `SimpleConfiguration<int>(0)` (or `nullptr`): the copy, another `CapacitatedFacilityLocationBlock`;
- `SimpleConfiguration<int>(1)`: an `MCFBlock` flow relaxation;
- `SimpleConfiguration<int>(2)`: the same flow relaxation, but with extra "artificial" arcs that keep it feasible for every facility-opening pattern.

The flow relaxation is a genuine *Relaxation* in the R3 sense: its optimal value is a valid lower bound on the CFL optimum, and it is much cheaper to solve (a min-cost flow rather than a MILP). The graph it builds has `f_n_facilities + f_n_customers + 1` nodes — one per facility, one per customer, plus a super-source — and `f_n_facilities * (f_n_customers + 1)` arcs: a "facility arc" from the super-source to each facility (capacity = facility capacity, cost = fixed cost / capacity) and a "transportation arc" from each facility to each customer (capacity infinite, cost = unitary transportation cost). When the `(2)` variant is requested, an extra arc from the super-source to each customer, with very large cost, absorbs any unmet demand and so guarantees feasibility.

This same construction does double duty in the framework: it is the user-visible R3 Block described here, *and* it is the internal engine of the "Benders friendly" formulation of CFL (BenForm, §3.5), where the `MCFBlock` is wrapped in a `BendersBFunction` to produce Benders cuts (Chapter 14, Recipe R5). A single, carefully written `get_R3_Block` thus pays off twice.

`CapacitatedFacilityLocationBlock` implements `map_back_solution`, `map_forward_solution` and `map_forward_Modification` for both the copy and the MCF R3 Block (with the documented exception that `map_back_Modification` to the MCF relaxation is not implemented — the relaxation is a one-way target in that respect).

10.6 Keeping an R3 Block in sync: `UpdateSolver`

When an algorithm holds both an original `Block` and one of its R3 Blocks and changes the original, the R3 Block must be updated to match. Doing this by hand — intercepting every `Modification` on the original and calling `map_forward_Modification` — is mechanical and error-prone, so the framework packages it as a `:Solver`: `UpdateSolver`.

`UpdateSolver` is a `Solver` whose only job is to forward `Modification`s. Registered on the original `Block` with a pointer to the R3 Block, it `map_forward`s every `Modification` it receives to the R3 Block; registered on the R3 Block with a pointer to the original, it `map_back`s them instead. Its constructor takes the R3 Block, the `Configuration` used to produce it, an options bit-mask (forward vs back; map vs pass through unchanged; restrict to `Modification`s concerning the `Block` itself vs its sub-`Block`s; restrict by `concerns_Block()`), and the `issuePMod` / `issueAMod` values to pass on to the mapping methods (`UpdateSolver.h:79-129`).

The canonical idiom — used verbatim in the CFL test (Recipe R3) — is: build the original `Block`, produce its R3 Block via `get_R3_Block`, register an `UpdateSolver` on the original pointing at the R3 Block, and from then on simply mutate the original; the R3 Block follows along automatically. The `UpdateSolver` is, in effect, the

live wire that keeps a relaxation faithful to the model it relaxes while the model changes underneath it (as in a slope-scaling or branch-and-bound loop).

10.7 Inline example: a CFL flow relaxation kept in sync

```
#include "CapacitatedFacilityLocationBlock.h"
#include "MCFBlock.h"
#include "MCFSolver.h"
#include "MCFSimplex.h"
#include "UpdateSolver.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 CapacitatedFacilityLocationBlock cfl;
 cfl.load( /* ... a CFL instance ... */ );

 // produce the MCF flow relaxation (R3 Block, "feasible" variant)
 auto r3bc = SimpleConfiguration< int >( 2 );
 Block * R3B = cfl.get_R3_Block( & r3bc );    // an MCFBlock
 auto * mcf  = dynamic_cast< MCFBlock * >( R3B );

 // attach a fast MCF Solver to the relaxation
 auto * mcfs = new MCFSolver< MCFSimplex >();
 mcf->register_Solver( mcfs );

 // keep the relaxation in sync with cfl automatically: every
 // Modification on cfl is map_forward-ed to the MCFBlock
 auto * upd = new UpdateSolver( R3B , & r3bc );
 cfl.register_Solver( upd );

 // ---- a slope-scaling-style iteration ----
 for( int it = 0 ; it < max_iters ; ++it ) {
  mcfs->compute();                        // solve the (cheap) relaxation
  cfl.map_back_solution( R3B , & r3bc ); // bring its solution into cfl
  // ... use the (fractional) relaxation solution to adjust cfl's
  //     facility costs (slope scaling); each chg_cost on cfl is
  //     automatically forwarded to the MCFBlock by `upd` ...
 }

 cfl.unregister_Solver( upd , true );
 mcf->unregister_Solver( mcfs , true );
 delete R3B;
 return 0;
}
```

The salient points: `get_R3_Block` returns an independent `MCFBlock` (its `Variable`s are not those of `cfl`); `map_back_solution` is what brings the relaxation's solution into `cfl`; and the `UpdateSolver` removes the need to manually forward each cost change — a `chg_cost` on `cfl` reaches the `MCFBlock` through `map_forward_Modification` without any explicit call. Recipe R3 develops this into a complete, runnable program and compares it with the alternative of using a `CapacitatedFacilityLocationBlock` R3 Block solved by a `:MILPSolver` or by a `LagrangianDualSolver`.

10.8 Idioms

Pass the same `Configuration` to `get_R3_Block` and to every mapping call. The `Block` uses `r3bc` to know which R3 Block it is dealing with; using a different `r3bc` in `map_back_solution` than in `get_R3_Block` is an error.

Use `UpdateSolver` rather than hand-forwarding. Whenever an R3 Block must track changes to its original (or vice versa), register an `UpdateSolver` instead of intercepting `Modification`s by hand. It is less code and it

gets the `map_forward` / `map_back` direction, the channel handling, and the `concerns_Block()` filtering right.

Remember that R3 Blocks are independent objects. An R3 Block has its own `Variable`s, its own `Constraint`s, its own `Solver`s, and its own lifetime. It must be destroyed explicitly (unless it was created with a `father` that owns it), and a solution read off it means nothing to the original until it has been `map_back`-ed.

A Relaxation is a lower/upper bound, not the answer. The CFL MCF relaxation gives a bound and a (typically fractional) solution, fast; it does not give the CFL optimum. Mapping its solution back is the *start* of a rounding or branching procedure, not the end of one.

11. Configuration

[Source: SMS++/include/Configuration.h, Block.h (BlockConfig), RBlockConfig.h, BlockSolverConfig.h, ThinComputeInterface.h (ComputeConfig)]

11.1 Concept

Almost every behaviour of a Block and of a Solver that is not fixed by the problem instance is controlled by a Configuration object: which parts of the abstract representation to generate and with which options, which :Solvers to attach and with which parameters, which part of a solution to keep, what tolerances to use in the feasibility checks, and so on. A Configuration is a first-class object: it can be constructed programmatically, loaded from a text file, or [de]serialize-d to netCDF, and it mirrors the tree structure of the Block it configures.

The chapters so far have used Configuration objects in passing — to select a CFL formulation (§3.5, §7.3), to gate the construction of MCFBlock's bound constraints (§7.3), to select an R3 Block (§10.5), to pick which part of a Solution to keep (§9.1). This chapter gives the systematic treatment.

11.2 SimpleConfiguration<T>

The simplest concrete Configuration is the template SimpleConfiguration<T>, which wraps a single value of type T in a public field f_value. T is typically int, double, a std::pair<>, or a std::vector<> of such; nesting is allowed, so a SimpleConfiguration< std::vector< Configuration * > > is itself a perfectly good Configuration carrying a list of sub-configurations.

SimpleConfiguration<T> is the workhorse for the many places where a :Block needs "one number" to gate a decision. The four formulations of CapacitatedFacilityLocationBlock are selected by a SimpleConfiguration< int > whose f_value is the bit-mask wf (§3.5); the sparsity option of MCFBlock's objective is a SimpleConfiguration< double >; the R3 Block selector of §10.5 is a SimpleConfiguration< int >. Each :Block documents the exact type and meaning it expects.

Two container instantiations deserve special mention because they recur as the way to configure a *whole* ***Block*** *tree* at once:

- SimpleConfiguration< std::vector< Configuration * > > — a *positional* list: the i-th element configures the i-th sub-Block (this is, for instance, the form get_R3_Block accepts to pass a distinct sub-configuration to each inner Block, §10.3). Its drawback is that the caller must know the exact position of each Block in the tree.
- SimpleConfiguration< std::map< std::string , Configuration * > > — a *non-positional*, by-name alternative that is far more convenient and is used extensively in the tools/ and tests/ directories for **meta-configuration**: "apply this Block[Solver]Config to *every* Block of a given type", keyed by the class name. It removes the need to know where in the tree the Blocks of that type reside, and is typically sufficient, since it is rare for two sub-Blocks of the same type within a given parent to require *different* Block[Solver]Configs. This by-type meta-configuration is the idiom to reach for when configuring a large or programmatically-built tree.

11.3 BlockConfig and the [C/O/R] family

A BlockConfig gathers, in one object, every Configuration slot that a Block exposes. Its fields (Block.h:8516-8525) are:

- `f_static_variables_Configuration` — passed to `generate_abstract_variables()`;
- `f_static_constraints_Configuration` — passed to `generate_abstract_constraints()`;
- `f_dynamic_variables_Configuration` — passed to `generate_dynamic_variables()`;
- `f_dynamic_constraints_Configuration` — passed to `generate_dynamic_constraints()`;
- `f_objective_Configuration` — passed to `generate_objective()`;
- `f_is_feasible_Configuration` — the tolerance / options used by `is_feasible()`;
- `f_is_optimal_Configuration` — the tolerance / options used by `is_optimal()`;
- `f_solution_Configuration` — which part of the solution `get_Solution()` keeps;
- `f_extra_Configuration` — a `:Block`-specific catch-all (used, for instance, by `CapacitatedFacilityLocationBlock` in BenForm to configure the hidden `BendersBFunction`; §11.6).

A `BlockConfig` carries a `f_diff` flag indicating whether it is a *differential* configuration — one that changes only the slots it explicitly sets, leaving the others untouched — as opposed to a complete one that resets every slot.

The nine slots above are what the *base* `BlockConfig` carries, and they configure *one* `Block`, the one the object is applied to. The `[C/O/R]` family of derived classes adds two further, distinct capabilities on top of that base.

The first is recursion. The second is the configuration of the `ComputeConfig` of the `Objective` and of individual `Constraints`. This second capability is worth dwelling on, because it is easy to confuse with the named slots above. Recall (§5.1) that `Objective` and `Constraint` both derive from `ThinComputeInterface`: their value / satisfaction must be `compute()`-d, and that computation may in principle need its own parameters — i.e. a `ComputeConfig` (§11.5). This is rare, if it happens at all, in the current `:Block` catalogue, because the `Objectives` and `Constraints` in use are cheap to evaluate; but it *can* arise for a `Constraint` or `Objective` whose value is itself the result of an expensive sub-computation, and the framework provides for it. The named slot `f_objective_Configuration` gates the *generation* of the `Objective`; the `O` handler, by contrast, carries the *`ComputeConfig`* the `Objective` will use when it is `compute()`-d. The two are different things at different points of the lifecycle.

Three internal handlers (`RBlockConfig.h:100-779`) implement the three capabilities, and the concrete classes are the non-empty combinations of the corresponding letters (always written in the fixed order O, C, R):

- **O** — `OHandler`: carries one `ComputeConfig` for the `Block`'s single `Objective` (`set_Config_Objective(...)`).
- **C** — `CHandler`: carries a *list* of `ComputeConfigs`, one per selected `Constraint`. Because a `Block` may have many `Constraints` in many groups, each entry must *address* the `Constraint` it configures, by a pair `(constraint_group_id, constraint_index)` — where `constraint_group_id` is either the name or the index of the group, and `constraint_index` is the position of the `Constraint` within that group (the `Block::ConstraintID` convention; `RBlockConfig.h:374-418`). A `C`-flavoured config thus says, in effect, "give *this* `ComputeConfig` to the `Constraint` at index `j` of group `g`".
- **R** — `RHandler`: applies the whole configuration recursively to the sub-`Blocks` as well.

Class	O	C	R	Adds
`OBlockConfig`	•			`ComputeConfig` of the `Objective`, this `Block`
`CBlockConfig`		•		`ComputeConfigs` of selected `Constraints`, this `Block`
`RBlockConfig`			•	recursion of the base slots into sub-`Blocks`
`OCBlockConfig`	•	•		both `Objective` and `Constraint` `ComputeConfigs`
`ORBlockConfig`	•		•	`Objective` `ComputeConfig`, recursively
`CRBlockConfig`		•	•	`Constraint` `ComputeConfigs`, recursively
`OCRBlockConfig`	•	•	•	both, recursively

Every one of these is *also* a `BlockConfig`, so it carries the nine named slots in addition to the handler-specific data; the prefix only says which extra capabilities it brings.

The practical upshot: to apply one configuration to a `Block` and all its sub-`Block`s in one operation, use the `R`-flavoured variant; to touch only this `Block`, use the non-`R` one; to act only on the objective, only on the constraints, or on both, pick the `O`, `C`, or `OC` prefix accordingly.

11.4 `BlockSolverConfig` and `RBlockSolverConfig`

While a `BlockConfig` configures the *model*, a `BlockSolverConfig` configures the *solving*: it specifies which `:Solver`(s) are registered with a `Block` and, for each, the `ComputeConfig` (the solver parameters, §11.5) to apply. Applying a `BlockSolverConfig` to a `Block` registers the listed `:Solver`s; clearing it unregisters them. This is the object that makes "switching solver is a one-line change" literally true: the choice of `:Solver` is a line in a text file read into a `BlockSolverConfig`.

`RBlockSolverConfig` is the recursive variant: it carries, in addition to the `:Solver`s for the `Block` itself, a list of `BlockSolverConfig`s to be applied to the sub-`Block`s, recursively. This is how a whole `Block` tree gets its `:Solver`s attached in one operation — for instance, a `LagrangianDualSolver` on the master and a `DPBinaryKnapsackSolver` on each leaf sub-`Block` of CFL/KskForm (Recipe R4).

11.5 `ComputeConfig`

A `ComputeConfig` configures any `ThinComputeInterface` — so any `Solver`, but also any `Function` or other computing object. It bundles the integer, double and string parameters introduced in §6.4 (`intMaxIter`, `dblMaxTime`, `dblRelAcc`, ...) into one serialisable object, plus an optional "extra" `Configuration` for object-specific settings. A `BlockSolverConfig` holds one `ComputeConfig` per `:Solver` it manages; `Solver::set_ComputeConfig(ComputeConfig*)` applies one directly.

11.6 The `f_extra_Configuration` escape hatch

Most of a `Block`'s configurable behaviour fits the named slots of §11.3. For the cases that do not, `BlockConfig` carries `f_extra_Configuration`, a slot whose meaning is entirely `:Block`-specific.

The instructive use is `CapacitatedFacilityLocationBlock` in BenForm. There, the transportation sub-problem is held in a `BendersBFunction` wrapping a hidden `MCFBlock` that is *not* a sub-`Block` of the CFL `Block` (it has `father == nullptr`). The recursive `[C/O/R]BlockConfig` and `RBlockSolverConfig` machinery therefore cannot reach it. The configuration of that hidden `BendersBFunction` and of its inner `MCFBlock` — including the `BlockSolverConfig` that attaches a `:Solver` to the inner `MCFBlock`, which BenForm requires — is passed through `f_extra_Configuration`, in the precise format documented at `CapacitatedFacilityLocationBlock.h:756-818` (a `SimpleConfiguration< std::pair< Configuration*, Configuration* > >` whose first element is the R3-Block configuration of the inner `MCFBlock` and whose second is the `ComputeConfig` of the `BendersBFunction`). This is exactly the kind of situation `f_extra_Configuration` exists for: a configurable component that is logically part of the `Block` but structurally outside its sub-`Block` tree.

11.7 Loading and serialising

A `Configuration` (of any of the above kinds) can be:

- constructed programmatically, by setting its fields directly;
- loaded from a text file with `Configuration::load(std::istream&)` — the form used by the configuration files shipped in the `tests/` directories (the `BPar*.txt`, `BSPar*.txt` files of the CFL tests, Recipe R3 / R4 / R5);
- `[de]serialize`-d to / from a netCDF group, like every other first-class SMS++ object.

The factory `Configuration::new_Configuration(...)` reconstructs the right concrete `:Configuration` from a class name, which is what lets a text or netCDF file specify, say, an `RBlockSolverConfig` without the reading code knowing the type at compile time (Chapter 18).

11.8 Inline example: selecting a CFL formulation and a solver

```cpp
#include "CapacitatedFacilityLocationBlock.h"
#include "BlockSolverConfig.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 CapacitatedFacilityLocationBlock cfl;
 cfl.load( /* ... a CFL instance ... */ );

 // configure the MODEL: select the Knapsack Formulation (wf = 1)
 // by setting f_static_variables_Configuration on a BlockConfig
 auto * bc = new BlockConfig;
 bc->f_static_variables_Configuration = new SimpleConfiguration< int >( 1 );
 cfl.set_BlockConfig( bc );   // the Block takes ownership

 // configure the SOLVING: attach a LagrangianDualSolver, reading
 // the chosen solver and its parameters from a text file
 auto * bsc = new BlockSolverConfig;
 std::ifstream cfg( "BSPar-LDS.txt" );
 bsc->load( cfg );
 bsc->apply( & cfl );         // registers (and constructs) the Solver(s)
 bsc->clear();                // empty the config but keep it for cleanup

 // ... solve, read solution ...

 // cleanup: a clear()-ed (hence empty, non-differential) config,
 // re-applied, unregisters and deletes the Solver(s) it had created
 bsc->apply( & cfl );
 delete bsc;
 return 0;
}
```

The two halves are independent: `set_BlockConfig` decides *what model* `cfl` exposes (here, KskForm with its `BinaryKnapsackBlock` sub-`Block`s), while the `BlockSolverConfig` decides *how it is solved* (here, by whatever `:Solver BSPar-LDS.txt` names). Changing the formulation is a one-line change to the `SimpleConfiguration< int>` value; changing the solver is a change to the text file. Recipe R3 shows the same `CFL_test` executable producing three completely different solution strategies purely by swapping the configuration files.

11.9 Idioms

Separate model configuration from solver configuration. `BlockConfig` answers "what model"; `BlockSolverConfig` answers "how solved". Keep them in distinct objects (and distinct files); it is what makes the formulation and the solver independently swappable.

Reach for the `R` variant to configure a tree. When the same choice should propagate to sub-`Block`s, use `RBlockConfig` / `RBlockSolverConfig` rather than walking the tree by hand.

Use `f_extra_Configuration` only for what the named slots cannot express. It is the documented escape hatch for components that are logically part of the `Block` but structurally outside its sub-`Block` tree (the BenForm hidden `BendersBFunction` being the canonical case); it should not be used to smuggle in configuration that a named slot already covers.

Let configuration come from files. The framework's intended workflow is that formulations and solver stacks live in text or netCDF configuration files, read at runtime, not hard-coded. The `tests/` directories are the reference examples of this style.

12. Sub-Block and the recursive flow of Modifications

[Source: `SMS++/include/Block.h`; `CapacitatedFacilityLocationBlock/include/CapacitatedFacilityLocationBlock.h`, `BinaryKnapsackBlock/include/BinaryKnapsackBlock.h`]

Chapter 4 introduced the `Block` tree in the abstract; Chapter 5 fixed the composition rule for `Objective`s; Chapter 8 described how `Modification`s travel. This chapter draws those threads together into the single picture of a *recursive `Block` tree* and makes explicit the consequences for data, for solving, and for concurrency.

12.1 The recursive Block tree

A `Block` may contain any number of sub-`Block`s, each of which may in turn contain its own sub-`Block`s, to any depth. The structure is a tree: every `Block` has at most one father (the root has none), and the sub-`Block`s of a given `Block` are held in its `v_Block` vector of pointers (§4.4).

The defining restriction (§4.3) is that **sub-`Block`s are always static**: their number and their concrete types are fixed once the parent `Block` has been built and cannot change afterwards. There is no "dynamic sub-`Block`" analogous to dynamic `Variable`s or `Constraint`s. This restriction costs little in expressiveness — a model whose component count varies is usually better expressed with a fixed set of sub-`Block`s some of which are "switched off" — and it considerably simplifies the framework's bookkeeping, in particular the locking and the `Modification` routing described below.

12.2 How a model is split across the tree

The point of the tree is that a structured model is *decomposed* across it: each sub-`Block` carries the part of the model that is local to it, and the parent carries only what *links* its sub-`Block`s together.

Two composition rules, already stated in earlier chapters, make the tree add up to a single coherent model:

- **`Variables`** of a sub-`Block` are, conceptually, also `Variable`s of the parent (§4.1).
- **`Objectives`** sum up the tree (§5.4): the overall objective of a `Block` is its own `Objective` plus the `Objective`s of all its sub-`Block`s, recursively. A parent that carries no `Objective` of its own still has a well-defined overall objective, made up entirely of those of its sub-`Block`s.

What may, at first, look like a third rule — "a `Constraint` lives in the parent and links the `Variable`s of its children" — is in fact only the *most common pattern*, not a constraint imposed by the framework. The actual rule is more permissive:

> a `Constraint` defined anywhere in the `Block` tree may reference `Variable`s defined anywhere in the tree.

Compositionality makes one *expect* the references to follow inclusion — `Variable`s of two different sub-`Block`s coupled by a `Constraint` placed in their common parent — and a well-designed `:Block` library will largely follow that expectation, because it keeps each `Block` understandable in isolation. But it is not a hard requirement. A `Constraint` in the parent may legitimately involve `Variable`s of the parent together with `Variable`s of a child; and a `Constraint` *in a child* may even reference a `Variable` of the parent. The latter happens, for instance, with the `Gamma` variable in the dual formulation of `PolyhedralFunctionBlock`, and with the `DCNetworkBlock`s nested inside a `DesignNetworkBlock`. Such cross-references require specific support — the sub-`Block` must

expose a *setter* for the "external" `Variable` so that the parent (or whoever builds the model) can wire it in — but they are entirely possible and sometimes the natural way to express a problem.

The data of the problem is split accordingly: data that belongs to one component lives in the corresponding sub-`Block`; data that expresses the coupling lives wherever the modeller chose to place the linking `Constraints` — most often, but not necessarily, in the common parent.

12.3 The recursive flow of Modifications

A `:Solver` attached to a `Block` is responsible for solving the *entire* sub-tree rooted at that `Block` (§6.1). It follows that such a `:Solver` must be told about changes occurring *anywhere* in that sub-tree, not only in the `Block` it is directly attached to.

This is exactly what the framework does: a `Modification` issued by a `Block` travels up the chain of its ancestors, recursively up to the root, **and is delivered to all the `:Solvers` registered with any `Block` along that chain** — the issuing `Block` itself and every one of its ancestors. A change deep in a leaf sub-`Block` therefore reaches a `:Solver` attached to the root, because the root is an ancestor of the leaf, and equally any `:Solver` attached to any intermediate `Block` between the two. The "anyone listening" cache `f_at` (§8.7) records, at each `Block`, whether any ancestor has a `:Solver` registered, so that a `Block` whose sub-tree no one is solving does not waste effort constructing `Modifications`.

Conversely, a `:Solver` attached to a sub-`Block` is *not* told about changes happening in the parent or in sibling sub-`Blocks` (they are above or outside its sub-tree, §6.1). This is what makes it sound to attach a specialised `:Solver` to a single leaf sub-`Block` while a different `:Solver` solves the whole tree from the root: each sees exactly the changes relevant to its own responsibility.

12.4 Locking propagates down the tree

Concurrency control (Chapter 17) is also recursive. Acquiring a write lock on a `Block` with `lock()` automatically locks all of its sub-`Blocks`, recursively (`lock_sub_block`); the analogous `read_lock()` / `read_lock_sub_block` acquire shared read locks down the tree. The rationale is the same as for `Modification` routing: a `:Solver` operating on a `Block` operates on its whole sub-tree, so to obtain exclusive access it must lock the whole sub-tree, not just the root of it.

This recursive locking is also what underpins the immediacy guarantee of §8.4: an abstract change to a `Block` is performed while the `Block` is locked, hence while its entire sub-tree is locked, so no concurrent mutation can be interleaved between the change and the `Block`'s reaction to it.

12.5 Inline example: CFL in the Knapsack Formulation

The Knapsack Formulation (KskForm) of `CapacitatedFacilityLocationBlock` is the canonical small example of a non-trivial tree. Selected by setting `f_static_variables_Configuration` to `SimpleConfiguration<int>(1)` (§11.8), it makes the CFL `Block` grow `f_n_facilities` sub-`Blocks`, each a `BinaryKnapsackBlock`, one per facility. The split is exactly as §12.2 describes:

- each sub-`BinaryKnapsackBlock` carries the capacity constraint of one facility and that facility's transportation/design variables (the per-facility knapsack);
- the parent `CapacitatedFacilityLocationBlock` carries only the customer-satisfaction *linking* constraints $\sum_i X_{ij} = 1$, which couple `Variables` living in different sub-`Blocks`;
- the overall objective is the sum of the per-facility knapsack objectives (the parent carries no objective of its own in this formulation).

```
#include "CapacitatedFacilityLocationBlock.h"
#include "BinaryKnapsackBlock.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 CapacitatedFacilityLocationBlock cfl;
```

```
 cfl.load( /* ... a CFL instance ... */ );

 // select the Knapsack Formulation: the Block will grow one
 // BinaryKnapsackBlock sub-Block per facility
 auto * bc = new BlockConfig;
 bc->f_static_variables_Configuration = new SimpleConfiguration< int >( 1 );
 cfl.set_BlockConfig( bc );
 cfl.generate_abstract_variables();     // builds the sub-Block tree
 cfl.generate_abstract_constraints();   // builds the linking constraints
 cfl.generate_objective();

 // the tree is now populated: navigate it
 const auto nf = cfl.get_number_nested_Blocks();   // == f_n_facilities
 for( Block::Index i = 0 ; i < nf ; ++i ) {
  auto * bk = dynamic_cast< BinaryKnapsackBlock * >( cfl.get_nested_Block( i ) );
  // bk is the knapsack sub-Block for facility i
  }

 // a change in a leaf sub-Block reaches a Solver attached at the root:
 // (suppose a Solver has been registered on cfl)
 auto * bk0 = dynamic_cast< BinaryKnapsackBlock * >( cfl.get_nested_Block( 0 ) );
 bk0->chg_weight( /* new weight */ 5.0 , /* item */ 2 , eModBlck );
 // the BinaryKnapsackBlockMod issued by bk0 travels up to cfl and is
 // delivered to every Solver registered on cfl (the root of bk0's tree)

 return 0;
}
```

The example is exactly the configuration that Recipe R4 solves with a `LagrangianDualSolver`: the solver attached to the root `cfl` dualises the linking constraints, leaving the per-facility knapsacks as independent sub-problems, each solvable by a `DPBinaryKnapsackSolver` attached to its sub-`Block`. The recursive `Modification` flow is what keeps that arrangement correct: a change to a facility's data, wherever it is made, reaches the root solver that needs to know about it.

12.6 Idioms

Prefer coupling in the parent and locality in the children — but know that this is a guideline, not a rule. As §12.2 made precise, the framework imposes *no* restriction on where a `Constraint` lives relative to the `Variable`s it references: any `Constraint` may reference any `Variable`, anywhere in the tree. The general rule is, in effect, that there is no rule. What there *is* is a strong expectation, born of logical composability: a sub-`Block` that is self-contained — referencing external `Variable`s only through explicit, supported setters, and only where the model genuinely requires it — stays understandable in isolation and reusable across contexts, whereas one that "reaches sideways" into a sibling for no compelling reason makes both `Block`s harder to reason about and to reuse. Place linking `Constraint`s where they make the decomposition clearest; that is usually, but not always, the common parent.

Attach the tree-solving `:Solver` at the root of the sub-tree it must solve. Because `Modification`s flow upward, a `:Solver` that must account for the whole model is attached at the root; a specialised `:Solver` for one component is attached at that component's sub-`Block`. The two coexist precisely because each is told only about the changes within its own responsibility.

Do not expect dynamic sub-`Block`s. If the number of components must vary, model the maximum set of sub-`Block`s once and switch components on and off through their data (fixing `Variable`s, relaxing `Constraint`s), rather than trying to add or remove sub-`Block`s at runtime.

13. The Function family

[Source: `SMS++/include/Function.h`, `C05Function.h`, `C15Function.h`, `DQuadFunction.h`, `QuadFunction.h`, `PolyhedralFunction.h`, `PolyhedralFunctionBlock.h`]

`Function`s are the objects that supply the *value* inside an `FRowConstraint` (its left-hand side) and inside an `FRealObjective` (its value); they also stand on their own as the mathematical content of the two `Block`-and-`Function` hybrids of Chapter 14. This chapter describes the hierarchy and the one idea that gives it its algorithmic power: the *linearization pools* of `C05Function`.

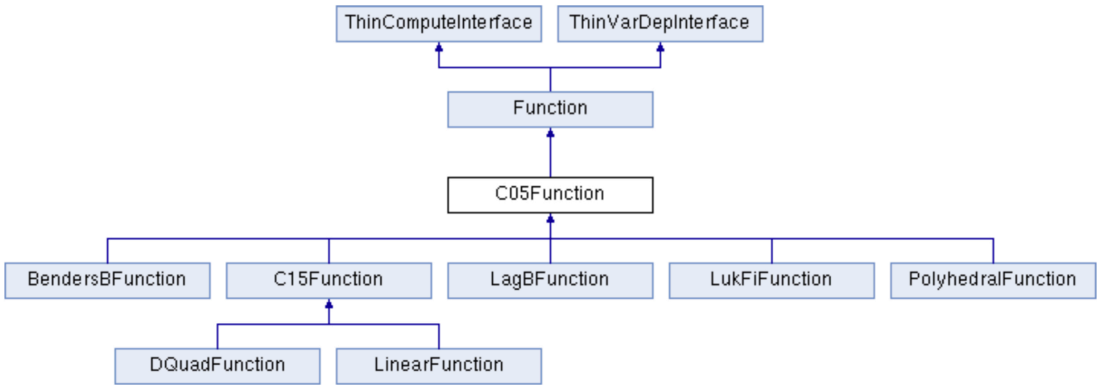


Figure 5: The `Function` hierarchy: from the base `Function` through `C05Function` (first-order information) and `C15Function` (second-order information) down to the leaf implementations.

13.1 The base `Function`

A `Function` is a real-valued function of a set of *active* `Variable`s. Like `Constraint` and `Objective`, it derives from both `ThinVarDepInterface` (it depends on a set of `Variable`s) and `ThinComputeInterface` (it must be `compute()`-d, possibly at significant cost). Its value type is `FunctionValue` (a `double`). The base class exposes only *values*: `compute()` evaluates the function at the current values of its active `Variable`s, and accessor methods read the result. First- and second-order information, and any specific algebraic form, are left to derived classes.

Crucially, the base `Function` also *advertises* a number of mathematical properties that a `:Solver` can rely on to choose its algorithm. These are exposed through a family of `is_*()` predicates (`Function.h:567-615`):

- `is_convex()` and `is_concave()` — whether the function is convex / concave (both default to `false`; a function that is neither simply returns `false` to both);
- `is_linear()` — defined as `is_convex() && is_concave()`;
- `is_lower_semicontinuous()`, `is_upper_semicontinuous()`, and `is_continuous()` (true when both of the former hold).

A bundle method, for example, will check `is_convex()` (or `is_concave()`) before treating the linearizations as sub/supergradients; a `:Solver` that requires a smooth objective will check `is_continuous()`. The base class returns the most conservative answers; a concrete `:Function` overrides them to declare the structure it actually has.

Two further facts about the base class are worth keeping in mind.

First, a `Function` reports its `Modifications` to *one* `Observer` (Chapter 8) — typically the `FRowConstraint`, `FRealObjective`, or `Block` that contains it. A `Function` can also be evaluated *approximately*: a caller may accept lower and/or upper estimates of the value, provided they are accurate enough, which is what makes inexact bundle and Benders methods possible.

Second — a subtle but important rule — a `Function` does **not** register itself with its active `Variables`. A free-floating `Function` has no way of knowing whether it is a part of some `Constraint` / `Objective` or not, so the responsibility of registering with the `Variables` falls on the `ThinVarDepInterface` that *uses* the `Function` (the `FRowConstraint`, `FRealObjective`, ...). When a `Variable` is added to or removed from the `Function`, the `Function` issues a `FunctionModVars Modification`, which the containing object observes and reacts to by registering / unregistering itself with that `Variable` (`Function.h:90-119`). The practical consequence for a user is that `Functions` are always handed to a carrier (`set_function(...)`, §5.3) that takes care of this; one does not normally hold a bare `Function` and expect it to maintain its own `Variable` dependencies.

13.2 `C05Function`: first-order information and linearization pools

`C05Function` extends `Function` with *first-order* information: not necessarily continuous, in the form of **linearizations**. Assuming the active `Variables` are single reals (`ColVariables`), a linearization is an affine function $L(x) = gx + \alpha$ defined by a real n-vector g and a scalar α; geometrically it is an object in the graph space $\mathbb{R}^{n+1}$ that, for a convex function, *supports the graph from below* (a subgradient inequality), and for a concave one from above.

The reason this matters so much in SMS++ is that, for many structured functions, computing the value already produces the linearization as a by-product. The canonical case is a Lagrangian function $f(x) = \max\{cu + x(b - Au) \mid u \in U\}$: any optimal u^* of the inner problem yields both $f(x) = cu^* + x(b - Au^*)$ and the linearization $(g, \alpha) = (b - Au^*, cu^*)$, which supports the graph of f everywhere (`C05Function.h:96-133`). This is exactly the mechanism that `LagBFunction` exploits (Chapter 14).

`C05Function` organises linearizations into two **pools**:

- the **local pool** is tied to the last point x at which `compute()` was called, and is automatically cleared when `compute()` is next called at a different point. It is meant to hold the linearizations "significant at x" — for a convex function, the ε-subgradients at x; for a concave one, the ε-supergradients; for a smooth one, the gradient. One evaluation may produce several linearizations (multiple ε-optimal inner solutions), and one linearization may be valid at many points — both situations the pool accommodates.
- the **global pool** persists across `compute()` calls and is the basis of *reoptimization*: it accumulates linearizations that remain valid as the point moves, so that a method which re-solves the function at many nearby points (a bundle method, a cutting-plane method) can reuse previously computed linearizations rather than recomputing them. The global pool supports operations such as taking a convex combination of its entries, and marking an "important linearization" (the one active at an optimum).

The linearization machinery is supported by a family of `C05FunctionMod Modifications` (the abstract-stream `Modifications` of Chapter 8) that signal, in fine-grained ways, how a change in the function's data affects its value and its pools — the subject of the next section.

13.3 Modifications, directionality, and reoptimization

The `Function` family carries an unusually rich vocabulary for describing *how* a function has changed, and it does so for a single, deliberate reason: to let a `:Solver` **reoptimize** — reuse the work it has already done — instead of starting over. This emphasis on reoptimization is one of the more distinctive aspects of SMS++; the `Function Modifications` are where it is most visible, and the design goes well beyond what one usually finds in comparable settings.

Directionality of a value change: `FunctionMod`

The base `FunctionMod` describes a change to the *value* of a function, and its single payload — the extended real returned by `shift()` — encodes not just the magnitude but the *predictability and direction* of the change (`Function.h:798-841`). Four cases:

- a **finite, non-NaN** `shift()`: the value changed by *exactly* that amount at *every* point — the most informative case, "the constant term of the function changed". A `:Solver` can update every cached value

and every linearization by the same additive shift, with no recomputation.
- $+\infty$ (`INFshift`): the value changed "unpredictably but monotonically **upwards**" — every value is now $\geq$ what it was. The exact new values are unknown, but the *direction* is, which is enough for a `:Solver` maintaining bounds to keep a valid lower or upper bound.
- $-\infty$ ($-$`INFshift`): symmetrically, "unpredictably but monotonically **downwards**".
- **NaN** (`NaNshift`): the value changed unpredictably, in no particular direction — the "nuclear `Modification` for a `Function`": everything previously known about its value is unreliable (the set of active `Variable`s, however, is unchanged; changes to that have their own `Modification`).

The point of distinguishing "up", "down", "none/exact" and "any" is precisely reoptimization: the more a `:Solver` knows about the *direction* of a change, the more of its previous work it can keep. A monotone-up change preserves lower bounds; an exact shift preserves everything up to a translation; only the NaN case forces a fresh start.

Quasi-additivity: `FunctionModVars`

When the change is the *addition or removal of active `Variables`*, the relevant `Modification` is a `FunctionModVars`, which carries the affected `Variable`s and, again through a `shift()` value, declares whether the change is **quasi-additive** (`Function.h:1008-1059`). Adding `Variable`s y to a function $f_{\text{old}}(x)$ is quasi-additive if

$$f(x, 0) = f_{\text{old}}(x) + \text{shift}() \quad \text{for all } x,$$

i.e. if fixing the new `Variable`s to their default value (0) recovers the old function up to a constant shift; removal is defined symmetrically. Quasi-additivity is what lets a `:Solver` reuse previously computed *values* across a change in the variable set: the old values are still meaningful, shifted by a known constant, at the points where the added variables are zero. (A NaN `shift()` signals that the change is *not* quasi-additive and the old values cannot be trusted.)

Strong quasi-additivity: `C05FunctionModVars`

For a `C05Function`, reusing *values* is not enough; one would like to reuse *linearizations* — the contents of the global pool — as well. This requires a stronger property. The `C05FunctionModVarsAddd` `Modification` signals that a variable-set change is **strongly quasi-additive** (which in practice usually means the function is convex or concave; `C05Function.h:2259-2285`). When it is, the linearizations already in the global pool remain valid (suitably interpreted) after the change, so a bundle / cutting-plane `:Solver` can keep its whole model. When a change is only ordinarily quasi-additive — a base `FunctionModVarsAddd` is issued, *not* the `C05` one — a `:Solver` must treat the first-order information computed at points where the added variables were non-zero as invalid, even though the *values* may still be reusable. The framework therefore lets a `C05Function` declare exactly how much of the previous first-order work survives a change, down to this fine distinction.

This graduated vocabulary — exact shift vs monotone direction vs unpredictable; quasi-additive vs strongly quasi-additive — is the machinery that makes SMS++'s reoptimization possible in practice, and Chapter 14 shows it doing real work: `LagBFunction` maps the `Modification`s coming from its inner `Block` into precisely these `FunctionMod*` types, so that a `BundleSolver` driving the Lagrangian dual can reoptimize across changes to the inner problem.

13.4 `C15Function`: second-order information

`C15Function` extends `C05Function` with *second-order* information — (partial) Hessians — for the cases where a method can exploit curvature. It is used far less often than `C05Function` in the current catalogue, because the structure-exploiting methods that dominate SMS++ applications (bundle, Benders, SDDP) are first-order, but it is there for the second-order methods that need it.

13.5 The "easy" leaf Functions

For functions that have a simple closed algebraic form, evaluating them and producing their (trivial) linearizations costs essentially nothing, and SMS++ provides concrete leaf classes with no overhead:

- a linear `Function` — a plain linear form $\sum_i a_i x_i$ in `ColVariables` — is the recipient inside an `FRowConstraint` that encodes a "row of a linear program" or inside an `FRealObjective` that encodes a linear objective; this is what `MCFBlock`, `BinaryKnapsackBlock` and the natural formulation of CFL use for their constraints and objectives (Chapters 5, 7).
- `DQuadFunction` is a *separable* quadratic form (a sum of per-variable quadratic terms); `QuadFunction` is a general quadratic form.
- `PolyhedralFunction` is a function defined as the pointwise maximum (or minimum) of a finite set of affine pieces — i.e., a function that *is* its own set of linearizations. It is a `C05Function` whose linearizations are exactly its defining pieces, which makes it the natural recipient of cuts generated externally; the companion `PolyhedralFunctionBlock` wraps one as a `Block`. `PolyhedralFunction` is, for instance, where the Benders cuts produced inside an `SDDPBlock` are accumulated (Chapter 14 mentions this in passing).

13.6 Where the family points next

The two most consequential members of the family are not "leaf" functions at all but the hybrids that are *simultaneously* a `Block` and a `C05Function`: `LagBFunction` and `BendersBFunction`. Each wraps an inner `Block` and turns it into a `C05Function` whose evaluation is the solution of that inner `Block`, and whose linearizations are produced from the inner `Block`'s primal / dual solutions — exactly the Lagrangian-function mechanism sketched in §13.2, made into a reusable component. They are the subject of Chapter 14, and the engines behind Recipes R4 (Lagrangian) and R5 (Benders).

13.7 Idioms

Hand a `Function` to a carrier; do not free-float it. Because a `Function` does not register with its `Variables`, the supported way to use one is to give it to an `FRowConstraint`, an `FRealObjective`, or one of the `Block`+`Function` hybrids, which takes ownership and maintains the `Variable` dependencies. A bare `Function` held by user code is an advanced use that requires managing the registration by hand.

Reach for `C05Function` when first-order structure exists. If a function's value comes with a linearization "for free" (a Lagrangian dual, a value function, a polyhedral epigraph), modelling it as a `C05Function` lets every bundle / cutting-plane / Benders `:Solver` exploit the linearization pools. Modelling it as a plain `Function` throws that information away.

Let the pools do the reoptimization. When a function is re-evaluated at a sequence of nearby points, the global pool is what makes the sequence cheap: a `:Solver` should add to and draw from it rather than recomputing linearizations from scratch. This is the mechanism that `BundleSolver` relies on.

14. LagBFunction and BendersBFunction

[Source: `SMS++/include/LagBFunction.h`, `BendersBFunction.h`, `LagrangianDualSolver/include/LagrangianDualSolver.h`; `CapacitatedFacilityLocationBlock/include/CapacitatedFacilityLocationBlock.h`]

Two members of the `Function` family are special enough to deserve their own chapter. Each is *simultaneously* a `Block` and a `C05Function`: it wraps a single inner `Block` and presents that inner `Block`, evaluated, as a first-order function of some outer `Variable`s. They are the components that turn the abstract idea "the value of a function is the optimum of a sub-problem" (§13.2) into reusable building blocks, and they are the engines behind Lagrangian decomposition (Recipe R4) and Benders decomposition (Recipe R5).

14.1 Two hybrids, one pattern

`LagBFunction` and `BendersBFunction` both derive from *both* `Block` and `C05Function`. As a `Block`, each holds exactly one sub-`Block`, the "base" `Block` (B) that represents the sub-problem. As a `C05Function`, each is a first-order function of a set of *active* `Variable`s — call them the *outer* `Variable`s — that are **not** `Variable`s of the hybrid `Block` itself (they belong to whatever outer `Block` defines them).

The pattern both follow is the same:

- to **evaluate** the function at a given value of the outer `Variable`s, the hybrid sets up (B) accordingly and calls `compute()` on a generic `:Solver` attached to (B) — "almost just calling the inner `Solver`'s `compute()`";
- the **linearization** of the function at that point is read off the inner `Block`'s primal and/or dual solution, exactly as in the Lagrangian example of §13.2;
- the hybrid maintains **pools of primal / dual `Solution`s** of (B), one per linearization, so that the surrounding cutting-plane / bundle / Benders method can reoptimize;
- any **change in** (B) is translated into the corresponding change of function value and pool entries, and surfaced as a `C05FunctionMod` (Chapter 8, Chapter 13) to whoever is solving the outer problem.

Because the inner `Block` may be "almost any" `Block`, and is solved by "almost any" `:Solver`, these two classes give a generic way to build the two most important decomposition functions over *arbitrary* structured sub-problems.

14.2 `LagBFunction`: the Lagrangian function of a `Block`

`LagBFunction` represents the Lagrangian function of its inner `Block` (B) with respect to a chosen set of *linear* terms. Concretely, given an inner problem

$$(B) \quad \max\{f(x) \mid x \in X\}$$

and a vector of pairs $\langle y_i, g_i(x)\rangle$ — each g_i a linear form in the same `Variable`s x as (B), each y_i a Lagrangian multiplier — `LagBFunction` is the function

$$\varphi(y) = \max\{f(x) + \sum_i y_i g_i(x) \mid x \in X\}$$

The g_i are meant to be (a part of) the *complicating constraints* of some original problem (O), relaxed in a Lagrangian fashion; the multipliers y_i are the outer `Variable`s of the `LagBFunction` — `ColVariable`s that the `LagBFunction` does *not* own (they are "conceptually fixed" while (B) is solved). φ is convex in y (concave if (B) is a maximisation, as written), being a pointwise optimum of affine functions of y; at a point y, an (approximately) optimal inner solution $x(y)$ gives the (sub/super)gradient $g(x(y)) = [g_i(x(y))]_i$ — the linearization. Hence `LagBFunction` is, in general, a non-smooth `C05Function` (`LagBFunction.h:77-147`).

`LagBFunction` automates the whole apparatus: turning (B) into φ, evaluating it by calling a `:Solver` on (B), maintaining the local and global linearization pools, and keeping φ in sync as (B) changes. It is not supposed to have any `Variable` or `Constraint` of its own beyond those of (B).

Reoptimization: mapping inner `Modification`s to `FunctionMod*`

`LagBFunction` is where the reoptimization vocabulary of §13.3 earns its keep. When the inner `Block` (B) changes — a cost in its objective, a bound in one of its constraints, a `Variable` added or removed — a `Modification` is issued by (B), and `LagBFunction` *translates* it into the corresponding `FunctionMod*` for φ: an exact-shift `FunctionMod` when the change shifts φ's value by a known constant, a monotone ($+/-\infty$) one when only the direction is known, a quasi-additive or strongly quasi-additive `FunctionModVars` when the change adds or removes inner `Variable`s. Because the translation is faithful, a `BundleSolver` driving the Lagrangian dual can react to a change in the *inner* problem by reoptimizing — keeping the parts of its bundle (the global pool) that remain valid — rather than rebuilding the dual from scratch. `LagBFunction` uses this mechanism extensively, and it is one of the concrete pay-offs of SMS++'s focus on reoptimization: a change deep inside a decomposed model propagates outward as a precisely typed `FunctionMod`, and the outer `:Solver` does the least work the change allows.

The `LagrangianDualSolver`

A user rarely constructs a `LagBFunction` by hand. The usual route is the `LagrangianDualSolver`, a `CDASolver` that solves the Lagrangian dual of a "generic" `Block` (B) with block-diagonal structure: no `Variable`s and no `Objective` of its own, at least one sub-`Block`, and all of its `Constraint`s being the *linear linking constraints* between the sub-`Block`s. Given such a (B), `LagrangianDualSolver` *stealthily* builds a hidden `LagrangianDualBlock`: for each sub-`Block` (B_i) of (B) it constructs a `LagBFunction` wrapping (B_i), dualising the linking constraints, and attaches the user-chosen inner `:Solver` to drive the dual (a `BundleSolver`, typically). The original (B) still "believes" the (B_i) are its own sub-`Block`s; the framework keeps both views consistent through the `Modification` mechanism (`LagrangianDualSolver.h:55-130`). The output is a *dual* bound and a *convexified* primal solution — the convex combination of inner solutions corresponding to the optimal multipliers, which is "better" than any single inner solution in the precise sense of Lagrangian duality.

The running example. CFL in the Knapsack Formulation (Chapter 12) is exactly a `Block` of this shape: a master carrying only the customer-satisfaction linking constraints, over `f_n_facilities` `BinaryKnapsackBlock` sub-`Block`s. Attaching a `LagrangianDualSolver` to it dualises the linking constraints, leaving one Lagrangian knapsack per facility — each a `LagBFunction` over a `BinaryKnapsackBlock`, solved by a `DPBinaryKnapsackSolver`. Recipe R4 develops this end to end, and shows how the derived `PrimalProximalHeur` turns the convexified primal solution into a feasible one.

14.3 `BendersBFunction`: the value function of a `Block`

`BendersBFunction` represents the *value function* of its inner `Block` (B) as a function of outer `Variable`s that enter the right- and/or left-hand sides of (B)'s constraints. Concretely, (B) has the form

$$(B) \quad \min\{c(y) \mid w \le E(y) \le z,\ y \in Y\}$$

and one is interested in how its optimum varies as w and z are shifted by an affine function of the outer `Variable`s x. The shift is expressed by a matrix A and a vector b: the mapping $M(x) = Ax + b$ gives, component by component, the value that the left- or right-hand side of a designated `RowConstraint` of (B) must take. The `BendersBFunction` is then

$$\varphi(x) = \min\{c(y) \mid (g - Fx) \le E(y) \le (h - Fx),\ y \in Y\}$$

i.e. the value of (B) once its constraint sides have been set to $M(x)$. The outer `Variables` x are the active `Variables` of the `BendersBFunction`; each corresponds to one column of A (`BendersBFunction.h:64-141`). Evaluating $\varphi(x)$ is one solve of (B); the dual solution of (B) gives the linearization (a *Benders optimality cut*), and when (B) is infeasible at a given x the Farkas ray gives a *vertical* linearization (a *Benders feasibility cut*) — both handled by the `C05Function` linearization machinery.

Like `LagBFunction`, `BendersBFunction` has no `Variable` or `Constraint` of its own beyond those of (B), evaluates via a generic `:Solver` on (B), and maintains the linearization pools.

It also maps changes in its inner `Block` (B) into the `FunctionMod*` types of §13.3, with the same reoptimization intent described for `LagBFunction` above. **At version 0.7.0, however, the `BendersBFunction` translation is more limited than `LagBFunction`'s**: fewer kinds of inner change are mapped to the fine-grained, directionally-typed `FunctionMod`s that would let an outer `:Solver` reoptimize maximally, and more of them fall back to a coarser "value changed" signal. This is an implementation gap, not a design one, and it is expected to be narrowed in future revisions; the underlying mechanism is the same.

The running example. CFL in the Benders Formulation (§3.5) is built on exactly this: the master carries the design `Variables` y and a single epigraphic `Variable` v; the transportation sub-problem $\varphi(y)$ is a hidden `MCFBlock` (the very flow relaxation of Chapter 10) wrapped in a `BendersBFunction`. As the master MILP visits design points $\hat{y}$, a user-cut / lazy callback evaluates the `BendersBFunction` — one MCF solve — and emits either an optimality cut (from the dual) or a feasibility cut (from the Farkas ray, in the feasibility-cuts sub-variant). Recipe R5 develops this end to end.

`BendersBFunction` is also the component that produces the cuts in an `SDDPBlock`: there, the Benders cuts generated at each stage are accumulated into a `PolyhedralFunction` (§13.4) representing the recourse-cost approximation of the following stage.

The Benders master. Since version 0.7.0 the enclosing Benders master is built by `BendersDecompositionSolver`, in the way `LagrangianDualSolver` builds the Lagrangian one. The complicating `Variables` x and the subproblems are those named by the parameters `vintMasterVars` and `vintMasterBlock`, since the same `Block` admits, in general, many Benders reformulations. In each subproblem the `:Solver` finds the rows that are linear in x, and it wraps the subproblem in a `BendersBFunction`; the master is then solved in one of two regimes, selected by `int_BDSlv_Regime`. In the *convex* regime the value functions are `C05Function` terms of the master `Objective`, and a `:CDASolver` of the bundle family (e.g., `BundleSolver`, or `ParallelBundleSolver` if the subproblems are to be evaluated in parallel) drives a stabilized cutting-plane method. In the *MILP* regime, instead, the master is a MILP with one epigraph `Variable` per subproblem (or a single one), the cuts enter it as dynamic `Constraints`, and the master is solved again after each round of cuts. A `TwoStageStochasticBlock` provides the required form through `get_Benders_form()`, and the `-k` option of `smspp_tssb_solver` uses it. Also, the arrangement of Recipe R5, in which the cuts are emitted by the user-cut callback of the master `:MILPSolver` driving `generate_dynamic_constraints()`, remains available; it may be the more natural one when the cuts are to be seen by the branch-and-cut of the master while it proceeds.

14.4 Why "almost any `Block`, almost any `Solver`"

The power of these two hybrids is that the inner `Block` (B) is not constrained to be of any particular type, and the `:Solver` that evaluates it is not constrained to be of any particular type either. A `LagBFunction` can wrap a `BinaryKnapsackBlock` solved by dynamic programming, a `UCBlock` solved by a `:MILPSolver`, or a `Block` that is itself decomposed and solved by a nested `LagrangianDualSolver`. A `BendersBFunction` can wrap an `MCFBlock` solved by a fast network simplex, or a far more complex sub-problem solved by whatever fits. This is what lets SMS++ express *multi-level, heterogeneous, nested* decompositions — Benders outside, SDDP in the middle, Lagrange inside, dynamic programming at the leaves — by composing these components, each indifferent to what the others are.

14.5 Idioms

Do not build a `LagBFunction` by hand if a `LagrangianDualSolver` will do. For a `Block` with the block-diagonal "linking constraints over independent sub-`Block`s" shape, attach a `LagrangianDualSolver` and let it construct the `LagBFunction`s and the hidden `LagrangianDualBlock`. Constructing `LagBFunction`s manually is for cases that fall outside that shape.

Configure the inner `Solver`, not just the outer one. Both hybrids evaluate their inner `Block` with a `:Solver` that must be attached and configured. For a `BendersBFunction` whose inner `Block` is "hidden" (as in CFL/BenForm), the inner `:Solver` is configured through the `f_extra_Configuration` of the outer `Block` (§11.6); forgetting it is the most common setup mistake.

Expect a bound and a convexified / fractional solution, not the integer optimum. A `LagBFunction`-based dual gives a bound and a convexified primal solution; a `BendersBFunction`-based master gives a bound and the master's solution. Turning either into a feasible integer solution is the job of a subsequent heuristic (`PrimalProximalHeur`, rounding, branching), not of the hybrid itself.

15. The methods factory

[Source: `SMS++/include/Block.h` (methods factory), `BinaryKnapsackBlock/include/BinaryKnapsackBlock.h`]

The `Configuration` machinery of Chapter 11 can tell a `Block` *which* parts of its representation to build and *which* `:Solver` to attach, but it cannot, by itself, reach inside a `:Block` and *change its data* — change a weight, a cost, a capacity — without knowing the concrete `:Block` type at compile time. The **methods factory** is the mechanism that closes this gap. It is what lets a generic driver (a configuration file, a scenario generator, a stochastic-block realiser) invoke a data-changing member function of an unknown `:Block` by *name*.

15.1 The problem it solves

A `:Block`'s data-changing methods live in its *specialised* interface: `MCFBlock::chg_cost(...)`, `BinaryKnapsackBlock::chg_weight(...)`, `CapacitatedFacilityLocationBlock::...`. Calling any of them requires a pointer of the concrete type and a compile-time dependency on that type. But much framework code is deliberately *type-agnostic*: a `StochasticBlock` that realises scenario data into "some inner `Block`" (Chapter 1) does not know, and should not need to know, whether that inner `Block` is a `UCBlock`, a `CapacitatedFacilityLocationBlock`, or something written next year. It only knows that "the data to change is called such-and- such, and here are the new values".

The methods factory provides exactly this indirection: a (bi-directional) map from a *name* — a `std::string` readable at runtime, e.g. from a configuration file — to a *pointer to a function that changes a `Block`*. With it, type-agnostic code can retrieve the function by name and call it on a base `Block*`, without ever naming the concrete `:Block` class.

15.2 Adapter functions over a base `Block*`

The functions stored in the factory do **not** point directly to `:Block` member functions, because a member function pointer is tied to the concrete class and would defeat the purpose. Instead, each entry is an *adapter* function whose first parameter is a base `Block*`; the adapter `static_cast`s the pointer to the concrete `:Block` and forwards to the real method (`Block.h:5799-5855`). The framework can build these adapters automatically for member functions that already have the right shape, or a user can write one by hand to bridge any existing `:Block` method (even one that changes a single item at a time) to a factory-compatible signature.

The generic adapter signature carries, after the data arguments, the familiar two `ModParam`s (`issuePMod`, `issueAMod`) so that the factory-driven change participates in the Janus discipline of Chapter 8 exactly as a direct call would.

15.3 The six standard signature families

To make the factory useful, the data-changing functions need a small, shared vocabulary of signatures, so that the factory is densely enough populated to be worth consulting. SMS++ predefines six "method-set" signature families (`Block.h:665-680`), parameterised by two independent choices:

- the **data type** carried: none, `MF_dbl_it` (a const iterator into a `std::vector<double>`), or `MF_int_it` (into a `std::vector<int>`);
- the **slice form** addressing *which* elements change: a `Range` (a contiguous `[start, stop)` interval) or a `Subset` (an arbitrary set of indices, with a `bool` flag saying whether it is already ordered).

The six resulting types are:

Type	Arguments encoded
`MS_rngd`	`Range`
`MS_sbst`	`Subset &&, bool`
`MS_dbl_rngd`	`MF_dbl_it, Range`
`MS_dbl_sbst`	`MF_dbl_it, Subset &&, bool`
`MS_int_rngd`	`MF_int_it, Range`
`MS_int_sbst`	`MF_int_it, Subset &&, bool`

A function in the `MS_dbl_rngd` family thus has the shape "`(Block*, MF_dbl_it data, Range slice, ModParam, ModParam)`": "apply the `double` values starting at `data` to the contiguous range `slice` of the relevant data structure". This is precisely the shape of `BinaryKnapsackBlock::chg_weights(c_dblVec_it, Range, ModParam, ModParam)`, which `BinaryKnapsackBlock` registers into the factory under the name `"BinaryKnapsackBlock::chg_weights"` (`BinaryKnapsackBlock.h:1027`). It registers `chg_weights` and `chg_profits` in both the `MS_dbl_rngd` and the `MS_dbl_sbst` families, and `chg_capacity` in `MS_dbl_rngd`. `MCFBlock` does the same for its own data — `chg_costs`, `chg_ucaps`, `chg_dfcts` in the `MS_dbl_*` families and `close_arcs` / `open_arcs` in the `MS_*` ones (`MCFBlock.h:2131-2189`). The framework nudges `:Block` authors to give their data-changing methods one of these six shapes, precisely so they slot into the factory for free.

15.4 `register_method`, `get_method`, `get_method_name`

The three operations on the factory are:

- `register_method(name, function)` — associate `name` with `function` in the factory selected by the function's type; a later registration under the same name replaces the earlier one.
- `get_method(name)` — retrieve the function registered under `name` (in the factory of the requested type), to be called on a `Block*`.
- `get_method_name(function)` — the reverse lookup (the map is bi-directional).

Registration is conventionally done once, in the `:Block`'s protected `static_initialization()` function, so that the `:Block`'s data-changing methods are available in the factory as soon as the class is loaded. The methods are nonetheless *public*, so any code with access to a `:Block` may register additional adapter functions — useful when the `:Block` author did not anticipate a particular form of bulk change, or simply did not get around to registering one (`Block.h:5857-5874`).

15.5 Inline example: changing weights by name

```
#include "BinaryKnapsackBlock.h"

using namespace SMSpp_di_unipi_it;

// type-agnostic code: it only has a base Block* and a string
void apply_new_doubles( Block * blk , const std::string & method_name ,
                        const std::vector< double > & values ,
                        Block::Range slice )
{
 // fetch the adapter registered under `method_name` in the
 // MS_dbl_rngd factory; no knowledge of the concrete :Block type
 auto fun = blk->get_method< Block::MS_dbl_rngd >( method_name );
 if( ! fun )
  throw( std::invalid_argument( "no such method: " + method_name ) );

 // call it on the Block: this changes the Block's data and issues
 // the appropriate Modification(s), just like a direct call would
 fun( blk , values.begin() , slice , eModBlck , eModBlck );
}
```

```
int main()
{
 BinaryKnapsackBlock bkb;
 bkb.load( /* ... */ );

 // change the weights of items [0,3) without naming BinaryKnapsackBlock
 Block * anon = & bkb;          // seen only as a base Block*
 apply_new_doubles( anon , "BinaryKnapsackBlock::chg_weights" ,
                    { 5.0 , 6.0 , 7.0 } , Block::Range( 0 , 3 ) );
 return 0;
}
```

The function `apply_new_doubles` has no compile-time dependency on `BinaryKnapsackBlock`: it works for *any* `:Block` that has registered an `MS_dbl_rngd` method under the given name. Passing `"MCFBlock::chg_costs"` instead, with an `MCFBlock`, would change arc costs through the very same function. The string follows the recommended naming convention (`ClassName::method_name`) and is exactly what a configuration file or a scenario generator would carry — this is, indeed, how `StochasticBlock` pushes scenario data into an inner `:Block` without a compile-time dependency on its type.

15.6 Relation to the class-name factories

The methods factory is one of two distinct factory mechanisms in SMS++, and they should not be confused:

- the **methods factory** of this chapter maps a *method name* to a *function that changes an existing `Block`*;
- the **class factories** of Chapter 18 map a *class name* to a *constructor*, so that a `Block`, `Solver`, `Configuration`, `Solution` or `Change` of a type named in a file can be *created* without compile-time knowledge of the type (`Block::new_Block(name)`, `Solver::new_Solver(name)`, ...).

Both rest on the same idea — defer a type decision to a runtime string — but one *constructs objects* while the other *mutates an existing one*. The `StochasticBlock` realiser mentioned in §15.1 uses the methods factory to push scenario data into an inner `Block` it was handed; it would use the class factory if it had to *build* that inner `Block` from a serialised description.

15.7 The dark side: registration versus the linker

SMS++'s insistence on selecting classes and methods at runtime from a string has a genuine *dark side*, worth stating plainly because it bites newcomers.

The registration of a method (or of a class, in the factories of §15.6 and Chapter 18) is performed by code that runs at program *initialization* — typically a static initializer in the translation unit of the `:Block`. Crucially, that translation unit contains *no visible call* from the rest of the program: the only thing the program does is later ask the factory for `"BinaryKnapsackBlock::chg_weights"` by string. From the compiler's and linker's point of view, nothing in the program references that translation unit's symbols at all — so an aggressive linker may conclude that the unit is *unused* and drop it from the executable. When that happens, the static initializer never runs, the registration never occurs, and the program fails at runtime with "such-and-such not present in the factory", even though the code is right there in the source tree.

This is not a bug in SMS++; it is an inherent tension between "resolve everything at runtime by name" and "let the linker remove what is statically unreachable". Avoiding it requires some build-time care — forcing the linker to retain the relevant translation units even though it sees no reference to them. The mechanics (the linker flags and the SMS++ registration macros that make them work) are the subject of Chapter 18; here we only flag the phenomenon, so that a reader who hits an inexplicable "not found in factory" error knows that the cause is almost always a translation unit the linker has optimised away, not a missing registration in the source.

15.8 Idioms

Give data-changing methods one of the six standard shapes. A `:Block` author who writes `chg_*()` methods in the `MS[_dbl/int]_[rngd/sbst]` shapes gets factory registration almost for free and makes the `:Block` usable by all the type-agnostic machinery (scenario generation, meta-configuration, distributed drivers). A method with an idiosyncratic signature can still be registered, but only through a hand-written adapter.

Register in `static_initialization()`. The conventional home for `register_method` calls is the `:Block`'s `static_initialization()`, run once when the class is loaded. Registering elsewhere is allowed (the methods are public) but should be the exception, for cases the author did not foresee.

Use the recommended `ClassName::method` naming. Factory names are arbitrary strings, but following the `ClassName::method_name` convention avoids collisions and makes a configuration file self-documenting.

16. Change

> **Status — beta.** The `Change` concept is in `develop` but its interface and semantics may change in incompatible ways. Only a small number of `:Change` classes exist at version 0.7.0 (notably for `BinaryKnapsackBlock` and `SATBlock`). Code written against it today should expect to be revisited.

[Source: `SMS++/include/Change.h`, `BinaryKnapsackBlock/include/BinaryKnapsackBlock.h`]

16.1 Concept

A `Change` is, like a `Modification`, an object that describes a change to a `Block`. The two are duals of each other in *time* (`Change.h:55-84`):

- a `Modification` (Chapter 8) *notifies* of a change that **has already occurred** in a `Block`, so that listening `:Solver`s can react;
- a `Change` *represents* a change that **may be applied later**, carrying all the data needed to perform it.

This difference in orientation has a concrete consequence: a `Change` must contain *all the new data* of the change, because it has to be able to *enact* the change, not merely describe one that some other code already made. A `Modification` can be a thin notification ("cost of arc 3 changed"); a `Change` must be a self-contained instruction ("set the cost of arc 3 to 7.0").

The single defining operation is

```
Change * apply( Block * block ,
                bool doUndo = false ,
                ModParam issueMod = eNoBlck ,
                ModParam issueAMod = eNoBlck );
```

which applies the change to `block` (issuing `Modification`s in the usual way, governed by the two `ModParam`s), and — if `doUndo` is `true` — returns an **UndoChange**: a freshly minted `:Change` that, applied to `block`, would restore it to the state it was in before (`Change.h:404-416`).

16.2 Abstract and specific `Change`s

Like `Modification`s and `Block`s, `Change`s come in two flavours:

- an **abstract** `:Change` operates on the abstract representation and therefore applies to *any* `Block` (it fixes a `Variable`, relaxes a `Constraint`, ...);
- a **specific** `:Change` is written for a particular `:Block` and uses that `:Block`'s specialised interface to change its physical representation directly.

A specific `:Change` can support at most the changes that its target `:Block` supports. It may deliberately support *fewer*, for a reason peculiar to `Change`: producing the UndoChange of an arbitrary change can be complicated or impossible, so a `:Change` that wants to guarantee an undo may restrict itself to the subset of changes it can reliably invert (`Change.h:74-84`).

16.3 Why `Change` exists

Serialisation and distribution

If a `Change` only ever applied a change in the same process that created it, it would add little over calling the `:Block`'s `chg_*()` methods directly. Its reason for existing is that, unlike a `Modification`,

> a `Change` is a *plain, self-contained object* that can be `[de]serialize`-d to and from netCDF independently of any `Block`.

`Change` carries a class-name factory (`Change::new_Change(name)`, `Change::deserialize(NcGroup)`) of exactly the same kind as `Block`, `Solver` and `Solution` (Chapter 18), so a `:Change` read from a netCDF group is reconstructed as the right concrete type and then `apply()`-ed. This makes a `Change` something a `Modification` can never be: a portable, storable, *transmissible* instruction. The intended downstream use is **message-passing distributed computation** — a process computes a `Change`, serialises it, ships it to another process holding a copy of the `Block`, which deserialises and `apply()`s it. The `Change` mechanism is, in this sense, the groundwork for the distributed `:Solver`s that the project plans but does not yet ship.

Support for generic search schemes

A second, equally important reason for `Change` to exist is that it makes it possible to write search algorithms in terms of abstract **moves** that are *independent of the `:Block` they are applied to*. A branch-and-bound, a branch-and-X (B&X) scheme, a large-neighbourhood search, a local-search metaheuristic — all of these are, at heart, "apply a move, explore, then undo or commit". If the move is a `Change`, the *search driver* need know nothing about the concrete `:Block`: it applies `Change`s and rolls them back, and the same driver works for any problem class that supplies the appropriate `:Change`s. The undo facility (§16.1) is what makes the roll-back free: an automatically-produced UndoChange restores the previous state wherever the `:Change` supports it.

The B&X case is the most telling, and it shows why the `:Change` abstraction is more than a convenience. The idea is that the `Change`s encode the **branching rule**, and that they are *produced by the `:Solver` that solves the relaxation*. Because that `:Solver` may be a *specialised* one, it can construct the branching `Change`s tailored not only to the structure of the problem but also to its own ability to **reoptimize efficiently** after the branch — choosing moves that its warm-start machinery (Chapter 13's reoptimization vocabulary) can absorb cheaply. The search driver, meanwhile, stays completely generic: it receives `Change`s from the relaxation `:Solver`, applies them to descend the tree, and uses the UndoChanges to backtrack, without any problem-specific code of its own. This decoupling — a generic B&X driver on one side, a problem-and-`Solver`-specific source of branching `Change`s on the other — is one of the more ambitious uses the `Change` concept is meant to enable, and it is **currently under development**.

16.4 The reference example: `BinaryKnapsackBlock`'s `Change`s

`BinaryKnapsackBlock` is the reference implementation of the `Change` family at version 0.7.0. It ships three specific `:Change` classes:

- `BinaryKnapsackBlockChange` — a "whole-object" change;
- `BinaryKnapsackBlockRngdChange` — a *range*-based change (a contiguous interval of items);
- `BinaryKnapsackBlockSbstChange` — a *subset*-based change (an arbitrary set of items),

mirroring, on the `Change` side, the `Rngd` / `Sbst` distinction that the `Modification` family draws (Chapter 8) and that the methods factory draws (Chapter 15). Each carries the new data (weights, profits, capacity, fixings) for the items it covers, can be serialised to netCDF, applied to a `BinaryKnapsackBlock`, and — within the subset it supports — produce its UndoChange.

```
#include "BinaryKnapsackBlock.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 BinaryKnapsackBlock bkb;
 bkb.load( /* ... */ );
```

```
 // build a Change that sets new weights on items [0,3)
 // (constructed directly here; in a distributed setting it would
 //  instead be deserialized from a netCDF group shipped by another
 //  process via Change::deserialize(...))
 Change * chg = /* a BinaryKnapsackBlockRngdChange carrying the new
                   weights for items 0..2 */ ;

 // apply it, asking for the inverse so we can roll back later
 Change * undo = chg->apply( & bkb , /* doUndo = */ true );

 // ... explore the modified Block ...

 // restore the original state
 undo->apply( & bkb );

 delete chg;
 delete undo;
 return 0;
}
```

16.5 `Change` versus `Modification`: which to use

For ordinary, in-process model edits — the bread-and-butter "change a cost, re-solve" of Chapters 8 and 10 — the `Modification` mechanism, driven by the `:Block`'s `chg_*()` methods, remains the right tool, and is the one the rest of this manual uses. `Change` is the right tool when one needs one of the things a `Modification` cannot provide:

- to **serialise** an edit and apply it elsewhere or later;
- to **transmit** an edit across a process boundary (the distributed-`:Solver` use case);
- to **undo** an edit cheaply via an automatically-generated inverse.

Because the concept is **beta** and implemented for only a few `:Blocks`, a `:Block` author should regard adding a `Change` family as an *optional* and forward-looking step (Appendix C), not a requirement; and a user should reach for `Change` only when one of the three needs above is actually present.

17. Parallel and asynchronous computation

[Source: `SMS++/include/Block.h` (locking), `ThinComputeInterface.h` (`compute_async`, `State`), `Solver.h`; `BundleSolver/include/ParallelBundleSolver.h`]

SMS++ is designed for *coarse-grained* parallelism: the unit of concurrent work is the solution of a `Block` (or sub-`Block`), not a fine-grained data-parallel loop. This chapter collects the mechanisms that make such parallelism safe and efficient — locking, asynchronous evaluation, and solver state — and is candid about what is not yet handled.

17.1 Locking a `Block`

Because several threads may want to read or modify the same `Block` tree, SMS++ provides a locking protocol on `Block`:

- `lock(owner)` acquires an *exclusive* (write) lock on behalf of `owner`, a `const void *` that identifies the locking entity (often, but not necessarily, derived from the thread). The lock is **recursive over the tree**: locking a `Block` automatically locks all of its sub-`Block`s, recursively (`lock_sub_block`), because an entity that holds a `Block` must hold its whole sub-tree to operate on it safely (Chapter 12).
- `unlock(owner)` releases it (and the sub-tree).
- `read_lock()` / `read_unlock()` acquire and release a *shared* (read) lock; any number of concurrent readers are allowed, as long as no writer holds the `Block`.

The lock records an **owner** rather than relying on a plain `std::mutex`, for a specific reason: the same entity may need to re-enter a `Block` it already holds (a `Solver` recursing into a sub-`Block`, say), and a plain mutex would deadlock on re-entry. Recording the owner lets the protocol distinguish "someone else holds this" from "I already hold this".

The one rule the user must respect is **direction**: lock attempts must always proceed *top-down* the tree, because locks naturally travel downward and a bottom-up attempt can deadlock against a top-down one. The corollary is convenient: *to lock any set of `Block`s safely, lock their nearest common ancestor* — one of the contenders is then guaranteed to succeed (`Block.h:1700-1714`).

> **Status — under development.** Write starvation is not yet handled: a steady stream of readers can, in principle, keep a writer waiting indefinitely. Applications that mix frequent reads with occasional writes should be aware of this.

This same recursive locking underpins the immediacy guarantee of §8.4: an abstract change is applied while the `Block` (hence its whole sub-tree) is locked, so no concurrent mutation can interleave between a change and the `Block`'s reaction to it.

Lending an ID

The recursive lock raises a problem the moment a `:Solver` *delegates*. Suppose a `:Solver` has `lock()`-ed a `Block` to perform an atomic computation, and in the course of it wants a *sub-`Solver`* to work on a part of the tree — a sub-`Block`, say. The sub-`Solver` needs to `lock()` that sub-`Block`; but the sub-`Block` is *already* locked, as part of the sub-tree the outer `:Solver` holds. The outer `:Solver` cannot simply release it either: doing so would risk another entity grabbing it before the outer `:Solver`'s atomic computation is finished.

The framework resolves this by letting the outer `:Solver` **lend its owner ID** to the inner `:Solver`(s). A lock attempt made with the lent ID is recognised as coming from the entity that already holds the lock — so it succeeds (re-entrantly) instead of blocking — rather than as contention from a different owner. The outer `:Solver` thus delegates work on a part of the tree it holds, without releasing the lock and without deadlocking against itself.

The mechanism only solves the *locking* side of delegation. It is the lending `:Solver`'s responsibility to ensure, by other means, that the inner `:Solver`(s) it has lent the ID to actually behave — that they confine themselves to the part of the tree they were delegated, do not corrupt the outer computation's invariants, and release what they should. The lent ID grants access; it does not grant good behaviour.

17.2 Asynchronous computation

Any `ThinComputeInterface` — so any `Solver`, but also any `Function` such as a `LagBFunction` or `BendersBFunction` — can be evaluated asynchronously. `compute_async(changedvars)` is a thin wrapper that launches `compute()` in a separate task and returns a `std::future< int >` on which the caller can later wait for the `sol_type` result (`ThinComputeInterface.h:1271`). This is the primitive on top of which coarse-grained parallel solution is built: a driver can fire off the `compute_async()` of several independent sub-problems and collect their results as the futures resolve.

For this to be safe, a `:Solver`'s `compute()` must be *thread-safe* with respect to the `Block` it solves; the recursive locking of §17.1 — including the lent-ID mechanism above — is what makes that achievable when a `:Solver` delegates to inner `:Solver`s on sub-`Block`s.

17.3 Solver `State` and checkpointing

A `ThinComputeInterface` may expose a `State` object — a snapshot of its internal algorithmic state — through `get_State()` / `put_State()` (`ThinComputeInterface.h:1980-2018`). A `State` can be serialised to netCDF, stored, and later restored into a (compatible) `:Solver`, which serves three purposes:

- **checkpointing**: a long-running solve can save its `State` periodically and resume from it after an interruption;
- **reoptimization across invocations / processes**: a `State` captured from one `:Solver` can warm-start another, which is especially valuable in the distributed and SDDP settings where the same kind of sub-problem is solved at many nodes;
- **restoring the *same* `:Solver` to an earlier condition.** A `State` is useful even within a single `:Solver`, when that `:Solver` is invoked under different conditions in an order that is not topological. The prototypical case is a branch-and-X search with a non-topological visit order: the driver solves a node with a `:Solver`, then jumps to an unrelated part of the tree, then comes back to explore one of that node's children. To resume the children efficiently, one wants the `:Solver` put back into the condition it was in *right after the node's children were generated* — for an LP relaxation `:Solver`, for instance, holding the optimal basis at that node, so the child re-solves by a few dual-simplex pivots rather than from scratch. Capturing the node's `State` when it is first solved and re-installing it when a child is later visited is exactly what makes this possible.

A `:Solver` with no meaningful internal state simply returns `nullptr` from `get_State()`; the mechanism then costs nothing.

17.4 Inline example: a parallel Lagrangian dual

The Knapsack Formulation of CFL (Chapters 12, 14) decomposes into one independent Lagrangian knapsack per facility; evaluating the Lagrangian function at a given multiplier vector means solving all of those knapsacks, which are independent and therefore parallelisable. The `ParallelBundleSolver` — the parallel variant of `BundleSolver` — exploits exactly this: it evaluates the per-facility `LagBFunction`s concurrently (through their `compute_async()`), gathers the linearizations, and drives the bundle method as usual.

From the user's point of view, switching from serial to parallel solution of the Lagrangian dual is, once again, a *configuration change*: the `BlockSolverConfig` (Chapter 11) names `ParallelBundleSolver` instead of `BundleSolver`, and sets the number of threads. The decomposition structure (Chapter 12), the `LagBFunction`s (Chapter 14), the recursive locking (§17.1) and the asynchronous evaluation (§17.2) do the rest; the surrounding

user code does not change. This is the payoff the energy-system results in the slide decks rely on: thirty-seven scenarios solved on as many processes, each with a parallel bundle method inside.

17.5 Idioms

Lock the nearest common ancestor. To operate atomically on a set of `Block`s, do not lock them one by one (risking a deadlock on ordering); lock their nearest common ancestor, which locks the whole sub-tree and is guaranteed deadlock-free against other top-down lockers.

Let the framework lend the owner identity. When a `:Solver` recurses into sub-`Block`s, rely on the framework's "lent ID" mechanism rather than inventing a new owner per level; this is what keeps the recursive locks re-entrant.

Return `nullptr` from `get_State()` unless state is real. Only implement `State` for a `:Solver` that genuinely benefits from checkpointing or warm-starting; otherwise the default `nullptr` keeps the machinery free.

Remember that the formulation determines the available parallelism. Once a parallel `:Solver` is applicable, selecting it is indeed just a `BlockSolverConfig` change. But *whether* it is applicable is decided earlier, by the *form* of the `Block`: the formulation chosen for a `:Block` determines which `:Solver`s can be attached to it, and hence what parallelism is available. CFL is the clear illustration — only the Knapsack Formulation exposes the block-diagonal structure that `LagrangianDualSolver` (and thus `ParallelBundleSolver`) requires; the Standard and Benders formulations do not, and no configuration switch can make a parallel bundle method apply to them. So the genuine lever for parallelism is often the *formulation*, chosen via the `BlockConfig` (Chapter 11), with the parallel `:Solver` selection in the `BlockSolverConfig` being the easy second step that the formulation has made possible.

18. Factories and netCDF serialisation

[Source: `SMS++/include/SMSTypedefs.h` (factory macros), `Block.h`, `Configuration.h`, `Solution.h`, `Change.h`]

Two cross-cutting mechanisms have been referred to throughout the manual without being treated head-on: the *class-name factories* that let an object be created from a string, and the *netCDF serialisation* that lets a whole `Block` tree be written to and read from disk. This chapter covers both, and discharges the promise of §15.7 about the linker.

18.1 The class-name factories

Five of the framework's base classes carry a *factory*: a static map from a class name (a `std::string`) to a constructor for that class. The entry points are:

- `Block::new_Block(name, father)`
- `Solver::new_Solver(name)`
- `Configuration::new_Configuration(...)`
- `Solution::new_Solution(name)`
- `Change::new_Change(name)`

Each returns a freshly constructed object of the concrete type named by the string, as a pointer to the base class. This is what makes it possible to read "the inner `Block` is a `UCBlock`", or "attach a `CPXMILPSolver`", or "this serialised object is a `BinaryKnapsackBlockRngdChange`" from a configuration file or a netCDF group and *build the right object* without the reading code having any compile-time dependency on the concrete type. It is the companion of the *methods* factory of Chapter 15: that one *mutates* an existing object by method name, this one *creates* objects by class name.

Registering a class

A concrete class joins its factory through two macros (`SMSTypedefs.h:299-377`):

- in the class declaration (private part of the `.h`): `SMSpp_insert_in_factory_h;`
- in *exactly one* `.cpp` (usually the class's own): `SMSpp_insert_in_factory_cpp_0( ClassName );` if the base-class constructor takes no argument, or `SMSpp_insert_in_factory_cpp_1( ClassName );` if it takes one (e.g. a `:Block`, whose constructor takes the father `Block*` — this is why `MCFBlock.cpp` uses the `_1` form, §4.5). *Template* classes use the `_t` variants (`SMSpp_insert_in_factory_cpp_0_t` / `_1_t`), which add the `template<>` specialisation the compiler requires; this is exactly the case of the `SimpleConfiguration<T>` family of Chapter 11, whose instantiations (e.g. `SimpleConfiguration< int >`) are registered with a `_t` macro. If the class name contains commas (a template instantiation with more than one parameter), it is wrapped in extra parentheses so the preprocessor does not read them as macro-argument separators.

The `_h` macro defines a tiny private `_init` class with a single static member `_initializer`; the `_cpp` macro defines that member. When the program starts, the `_initializer`'s constructor runs and does two things: it registers the class in the factory of its base, **and** it calls the class's `static_initialization()` method — which is exactly where the methods-factory registrations of Chapter 15 belong. So the two factory systems are wired up by the same one-time static initialisation.

18.2 The dark side, discharged: factories versus the linker

This is the mechanism §15.7 warned about, now in full.

The registration code lives inside the static `_initializer` of a translation unit, and *nothing in the rest of the program references that translation unit by symbol* — the program only ever asks the factory for the class by string. An aggressive linker, seeing no references into the unit (especially when it sits in a dynamic library), may drop it from the executable. The `_initializer` then never runs, and the program fails at runtime with

```
terminate called after throwing an instance of 'std::invalid_argument'
  what():  XXXX not present in YYYY factory
```

even though the class is present in the source (`SMSTypedefs.h:324-358`). The cause is *never* a missing registration in the source; it is a translation unit the linker has optimised away.

There are three remedies, in increasing order of preference:

1. **Force an object into the executable.** Instantiate an object of the missing type somewhere reachable from `main()`. This requires `#include`-ing the type's header and is therefore the least clean option.
2. **Disable the linker optimisation.** Pass the linker flag that keeps unreferenced units: `--no-as-needed` for `g++` (as `-Wl,--no-as-needed` when linking through the compiler driver), `-all_load` for `clang++`. Note that the default behaviour varies across toolchains and even across OS distributions of the *same* toolchain, and that these flags can have unwanted side effects (they retain *everything*).
3. **Use `SMSpp_ensure_load( ClassName )`.** Placed at the top of a `.cpp` (with the class's header `#include`-d), this macro does the "dirty but cheap" work of guaranteeing that `ClassName` is registered, targeting just that one class rather than retaining every unit. It carries a small, non-zero runtime cost and is meant as the targeted last resort.

The phenomenon is not a defect of SMS++ but the price of resolving types by name at runtime; the manual flags it because the symptom ("not present in factory") is mystifying until one knows the cause.

18.3 Why netCDF

SMS++ serialises `Block`s, `Configuration`s, `Solution`s, `State`s and `Change`s to netCDF, a binary, self-describing, hierarchical file format. The choice fits the framework's structure for two reasons:

- netCDF files are organised into nested **groups**, which map directly onto the nested structure of a `Block` tree: a `Block` serialises into a group, its sub-`Block`s into nested sub-groups, recursively. The on-disk shape mirrors the in-memory shape.
- netCDF stores **native binary arrays** efficiently, which suits the large numeric data (cost vectors, constraint matrices, scenario trees) of the problems SMS++ targets, far better than a text format such as JSON would.

A practical bonus is tooling: because netCDF is a widely adopted standard rather than a SMS++ invention, a SMS++ serialised file can be inspected with general-purpose, third-party tools, no SMS++-specific viewer required. The standard `ncdump` utility turns any such file into readable text; `ncview` visualises array data; and Panoply — an open-source graphical viewer available on all platforms, developed by NASA's Goddard Institute for Space Studies — offers a richer interactive exploration of the file's groups and arrays.

18.4 The dark side of netCDF: groups are not free

netCDF buys the structural fit of §18.3 at a price that users must know about: **creating a file with a very large number of groups is inefficient — in fact, very inefficient.** netCDF groups are designed to be a moderate-cardinality organising device, not a container one instantiates by the thousand; the cost of writing a file grows badly with the number of groups in it.

The naïve, "canonical" serialisation of a deeply or widely nested `Block` tree can fall into exactly this trap. The standard example is the `Solution` of a `UCBlock`: it contains, among other things, the `Solution` of a `NetworkBlock` *for each time instant*, and the canonical encoding would put each `NetworkBlockSolution` in its own group. But the number of time instants can be large — 8760 for the hours of a year — and writing 8760 (or many more) groups per solution makes solution files unbearably slow to produce.

To cope with this, the `Solution` classes involved (specifically, the `NetworkBlockSolution` used as components of a `UCBlockSolution`) implement a *bespoke* encoding that **accumulates the information of all those sub-`Solution`s into a single `netCDF::NcGroup`** rather than giving each its own, trading the clean one-object-per-group correspondence for a tolerable write cost.

It is important to be clear about the scope of this remedy:

> there is **no general, framework-wide mechanism** for collapsing many sub-objects into one group. The accumulation just described is a hand-written solution local to those specific `:Solution` classes, not a facility that any `:Block` or `:Solution` can simply switch on.

Whether a general mechanism could be provided — and what shape it would take — is an open question. So the point for a user (and especially for a `:Block`/`:Solution` author) is twofold: first, that the inefficiency is real and can surface as a surprisingly slow serialisation when a high-cardinality `:Block` or `:Solution` is written in the naïve one-group-per-object way; and second, that there is at present no built-in cure to reach for — a high-cardinality `:Solution` that needs to avoid the cost must, like `NetworkBlockSolution`, implement its own accumulating encoding by hand. This is an inherent limitation of the chosen file format rather than of SMS++'s design, but it currently has to be addressed case by case.

18.5 The serialisation contract

Serialisation is symmetric and recursive. On the writing side, `serialize(netCDF::NcGroup&)` writes the object into the given group; a `Block`'s `serialize` writes its own data and then calls `serialize` on each sub-`Block` into a nested group, so the whole tree is written by one top-level call. There are convenience overloads that take a filename or an `NcFile` and create the group structure for you.

On the reading side, the contract has two levels. The *static* `deserialize(netCDF::NcGroup&)` reads the mandatory string attribute `"type"` from the group, passes it to the factory (`new_Block`, `new_Configuration`, ...) to construct an empty object of the right concrete type, and then calls that object's *virtual* `deserialize` to fill it in. This is why the factory and the serialisation are inseparable: deserialising an object of a type not known at compile time *requires* the class-name factory of §18.1 to turn the stored `"type"` string into the right object.

The same `"type"`-tag-plus-factory pattern is what lets a `Configuration` file name an `RBlockSolverConfig` (Chapter 11), a serialised solution name a `CapacitatedFacilityLocationSolution` (Chapter 9), and a shipped `Change` name a `BinaryKnapsackBlockRngdChange` (Chapter 16): in every case the reader constructs the right concrete object from the stored class name, with no compile-time knowledge of the type.

18.6 Idioms

Register every concrete class in its factory. A `:Block`, `:Solver`, `:Configuration`, `:Solution` or `:Change` that is meant to be created from a string (and almost all are) must carry the `SMSpp_insert_in_factory_h` / `_cpp_k` macro pair. Forgetting it makes the class invisible to `new_X(name)`.

When a class is "not present in factory", suspect the linker first. The error almost never means a missing registration in the source; it means the registering translation unit was optimised away. Reach for `SMSpp_ensure_load(ClassName)` (with the header included) as the targeted fix, or the linker flags of §18.2 as the blunt one.

Serialise the tree, not the pieces. Because `serialize` / `deserialize` recurse, a whole `Block` tree (with its `Configurations` and `Solutions`) round-trips through a single top-level call. Build on that rather than serialising each sub-`Block` separately.

Use `ncdump` to debug serialised files. When a serialised `Block` does not deserialise as expected, `ncdump` on the file is the fastest way to see what was actually written, including the all-important `"type"` attributes.

Recipe R1 — Solving a Min-Cost Flow instance

> **Counterpart in the source tree:** `MCFBlock/test/test.cpp`. The program below is a didactic distillation of that tester; the tester itself is the runnable, fully-checked version.

Goal

Take a small directed graph with arc costs, arc capacities and node deficits; build an `MCFBlock`; solve it with the specialised `MCFSolver`; read back the optimal flows and the objective value; then swap the specialised solver for a general-purpose `:MILPSolver` without changing the surrounding logic. This is the "hello, world" of SMS++: the smallest complete program that builds a `Block`, attaches a `Solver`, and reads a solution.

Concepts used

- `Block` construction and `load(...)` — Chapter 4.
- `Solver` attachment, `compute()`, `sol_type` — Chapter 6.
- Physical vs abstract representation (why the `MILPSolver` variant needs `generate_*`) — Chapter 7.
- Reading the solution through the `Block`'s physical accessors — Chapter 9.
- (Variation) selecting the solver through a `BlockSolverConfig` — Chapter 11.

The code

```
#include <iostream>

#include "MCFBlock.h"
#include "MCFSolver.h"
#include "MCFSimplex.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 // ---- build the MCFBlock ----------------------------------------
 // a 3-node, 3-arc instance:
 //
 //     (0) --arc0,c=2--> (1) --arc1,c=3--> (2)
 //      \                                  ^
 //       `------------ arc2,c=4 -----------'
 //
 // node deficits b = (-2, 0, +2): node 0 produces 2 units, node 2
 // consumes 2; all arcs have capacity 3.

 MCFBlock mcf;                                  // a root MCFBlock

 MCFBlock::Subset      pSn = { 0 , 1 , 0 };     // arc starting nodes
 MCFBlock::Subset      pEn = { 1 , 2 , 2 };     // arc ending nodes
```

```
 MCFBlock::Vec_FNumber pU  = { 3.0 , 3.0 , 3.0 };  // arc capacities
 MCFBlock::Vec_CNumber pC  = { 2.0 , 3.0 , 4.0 };  // arc costs
 MCFBlock::Vec_FNumber pB  = { -2.0 , 0.0 , +2.0 };  // node deficits

 // NOTE: load() takes ending nodes (pEn) before starting nodes (pSn)
 mcf.load( 3 , 3 , pEn , pSn , pU , pC , pB );

 // ---- attach a specialised Solver and solve ---------------------
 auto * solver = new MCFSolver< MCFSimplex >();
 mcf.register_Solver( solver );

 const int status = solver->compute();

 // ---- read the solution -----------------------------------------
 if( status == Solver::kOK ) {
  std::cout << "objective = " << mcf.get_objective_value() << '\n';

  MCFBlock::Vec_FNumber x( mcf.get_NArcs() );
  mcf.get_x( x.begin() );                          // optimal flows
  for( MCFBlock::Index a = 0 ; a < mcf.get_NArcs() ; ++a )
   std::cout << "  flow on arc " << a << " = " << x[ a ] << '\n';

  MCFBlock::Vec_CNumber pi( mcf.get_NNodes() );
  mcf.get_pi( pi.begin() );                        // node potentials (dual)
  for( MCFBlock::Index n = 0 ; n < mcf.get_NNodes() ; ++n )
   std::cout << "  potential of node " << n << " = " << pi[ n ] << '\n';
  }
 else if( status == Solver::kInfeasible )
  std::cout << "infeasible\n";
 else if( status == Solver::kUnbounded )
  std::cout << "unbounded\n";
 else
  std::cout << "solver returned status " << status << '\n';

 // ---- clean up --------------------------------------------------
 mcf.unregister_Solver( solver , /* deleteold = */ true );
 return 0;
}
```

Walk-through

- The five arrays are the *physical* representation of the min-cost flow instance (Chapter 7): start/end node of each arc, arc capacities, arc costs, node deficits. `load(...)` copies them into the `MCFBlock` (Chapter 4); recall its idiosyncratic parameter order, ending nodes before starting nodes.
- `new MCFSolver< MCFSimplex >()` constructs the specialised solver, templated on the classical primal-simplex MCFClass algorithm; `register_Solver` enrols it on the `MCFBlock` (Chapter 6). Registration does *not* transfer ownership of the pointer; the `deleteold = true` flag on `unregister_Solver` is what asks the framework to `delete` it at the end.
- `solver->compute()` runs the algorithm and returns an `sol_type` (Chapter 6, §6.3); `kOK` means an optimal solution was found.
- The solution is read *through the `MCFBlock`*, not through the solver (the convention of §6.9 and Chapter 9): `get_objective_value()`, `get_x(...)` for the primal flows, and `get_pi(...)` for the dual node potentials. `get_rc(...)` would give the arc reduced costs; together the three are the natural primal–dual data of a min-cost flow, exactly what an `MCFSolution` would store (§9.5).
- Note that this program never calls `generate_abstract_*()`: the specialised `MCFSolver` reads the physical representation directly and needs no abstract `Variable`s or `Constraint`s. (The flows are nonetheless deposited into the always-materialised abstract `Variable`s as well — the §7.6 "half-baked" caveat — but the recommended way to read them is the physical accessor used here.)

Expected output

On this instance the cheapest way to move 2 units from node 0 to node 2 is the direct arc 2 (unit cost 4, total 8), which is cheaper than the two-arc path 0→1→2 (unit cost 2+3 = 5, total 10). So:

```
objective = 8
  flow on arc 0 = 0
  flow on arc 1 = 0
  flow on arc 2 = 2
  potential of node 0 = ...
  potential of node 1 = ...
  potential of node 2 = ...
```

The flows and the objective are determined; the exact node potentials depend on the dual solution the algorithm reports (several are optimal), so they are shown elided here.

Variations

Swap to a general-purpose `:MILPSolver`. A min-cost flow is a linear program, so it can also be solved by any `:MILPSolver` (e.g. `CPXMILPSolver`, `SCIPMILPSolver`, `HiGHSMILPSolver`). The *surrounding logic does not change* — construct the solver, `register_Solver`, `compute()`, read the solution through the `MCFBlock` — but a general-purpose solver reads the *abstract* representation, which must therefore be built first:

```
mcf.generate_abstract_variables();
mcf.generate_abstract_constraints();
mcf.generate_objective();
auto * milp = new SCIPMILPSolver();    // or CPX/HiGHS/GRB
mcf.register_Solver( milp );
milp->compute();
// ... read mcf.get_objective_value(), mcf.get_x(...) exactly as before
```

The optimal value is the same (8); the solver is slower on this trivial instance but works on any `:Block` that can expose an abstract representation. This is the swap promised in §6.2 and §6.9.

Select the solver from a configuration file. Rather than hard-coding the choice of solver in the program, the idiomatic SMS++ way is to read a `BlockSolverConfig` from a text file (Chapter 11) and `apply()` it to the `MCFBlock`. The same executable then solves with `MCFSolver`, `CPXMILPSolver`, or any other registered `:Solver` purely by editing the configuration file — no recompilation. Recipe R3 uses exactly this pattern to solve a CFL problem three different ways with one executable.

Reoptimize after a data change. Change an arc cost with `mcf.chg_cost(NewCost, arc)` (Chapter 8) and call `compute()` again: the `MCFSolver` reoptimizes from its previous state rather than starting over. This is the first taste of the reoptimization theme that Recipes R2 and R4 develop further.

```
mcf.chg_cost( 1.0 , 2 );    // arc 2 becomes cheaper still
solver->compute();          // warm restart
```

Recipe R2 — Reoptimizing a Binary Knapsack

> **Counterpart in the source tree:** `tests/BinaryKnapsackBlock/test.cpp`, which exercises the same `chg_*` reoptimization paths (and cross-checks the result against a second solver).

Goal

Solve a small knapsack with the specialised `DPBinaryKnapsackSolver`; then change the data *incrementally* and re-solve, letting the solver *reoptimize* from its previous state rather than starting over. This is the smallest illustration of SMS++'s pervasive reoptimization theme (Chapters 8 and 13).

Concepts used

- `Block` construction and `load(...)` — Chapter 4.
- A *native* specialised `Solver` — Chapter 6.
- `Modification` from a physical `chg_*()` change — Chapter 8.
- Reoptimization, and what a solver may reuse across a change — Chapter 13.
- (Variation) the `Change` family — Chapter 16.

The code

```
#include <iostream>

#include "BinaryKnapsackBlock.h"
#include "DPBinaryKnapsackSolver.h"

using namespace SMSpp_di_unipi_it;

static void solve_and_report( BinaryKnapsackBlock & bkb , Solver * s )
{
 if( s->compute() != Solver::kOK ) { std::cout << "not solved\n"; return; }
 std::cout << "objective = " << bkb.get_objective_value() << " ; x = (";
 for( BinaryKnapsackBlock::Index i = 0 ; i < bkb.get_NItems() ; ++i )
  std::cout << ( i ? "," : "" ) << bkb.get_x( i );
 std::cout << ")\n";
}

int main()
{
 // 4 items, capacity 5; weights must be integer for the DP solver
 BinaryKnapsackBlock bkb;
 const std::vector< double > W = { 2.0 , 3.0 , 4.0 , 1.0 };  // weights
 const std::vector< double > P = { 3.0 , 4.0 , 5.0 , 1.0 };  // profits
 bkb.load( 4 , /* capacity = */ 5.0 , W , P );   // maximization by default

 auto * solver = new DPBinaryKnapsackSolver();
```

```
 bkb.register_Solver( solver );

 // ---- first solve ----
 solve_and_report( bkb , solver );

 // ---- incremental change: item 2 becomes much more profitable ----
 bkb.chg_profit( 8.0 , /* item = */ 2 );   // physical change -> Modification

 // ---- re-solve: the solver reoptimizes from its previous state ----
 solve_and_report( bkb , solver );

 bkb.unregister_Solver( solver , /* deleteold = */ true );
 return 0;
}
```

Walk-through

- `load(4, 5.0, W, P)` sets the physical data of the knapsack; the default sense is maximisation (`set_objective_sense(false)` would switch it to minimisation). The `DPBinaryKnapsackSolver` requires the *weights* to be integer — they are — while capacity and profits may be real.
- `DPBinaryKnapsackSolver` is a *native* SMS++ specialised solver (Chapter 6): a dynamic-programming recursion over the items, no external dependency. It reads the physical representation, so — as in R1 — no `generate_abstract_*()` call is needed, and the solution is read back through the `Block`'s accessors `get_objective_value()` / `get_x(i)`.
- `chg_profit(8.0, 2)` changes the profit of item 2 in the physical representation and issues the corresponding `BinaryKnapsackBlockMod` (Chapter 8). Because the solver is registered, it receives the `Modification` in its pending list.
- The second `compute()` consumes that one `Modification` and **reoptimizes**: a single-profit change does not invalidate the whole DP table, and the solver reuses what it can rather than rebuilding from scratch. The `DPBinaryKnapsackSolver` exposes a `dblReopt` parameter (`DPBinaryKnapsackSolver.h`) governing how aggressively it reoptimizes. This is the recipe-scale instance of the reoptimization theme that, at scale, lets a `BundleSolver` reuse its bundle across changes to an inner `Block` (Chapter 13, Recipe R4).

Expected output

The first solve: with weights `(2,3,4,1)`, profits `(3,4,5,1)`, capacity 5, the best subset is items {0,1} (weight 2+3 = 5, profit 3+4 = 7). After raising item 2's profit to 8, the best subset becomes {2,3} (weight 4+1 = 5, profit 8+1 = 9):

```
objective = 7 ; x = (1,1,0,0)
objective = 9 ; x = (0,0,1,1)
```

Variations

Change several items at once, by range or subset. `chg_profits(it, Range)` and `chg_profits(it, Subset&&, bool)` (and the analogous `chg_weights`) change a contiguous interval or an arbitrary subset of items in one call, issuing a single `Rngd` / `Sbst` `Modification` that the solver can react to as a unit (Chapter 8). `chg_capacity(newC)` changes the knapsack capacity.

Apply the change as a `Change` (beta) instead. Because `BinaryKnapsackBlock` ships a `Change` family (Chapter 16), the same edit can be expressed as a `BinaryKnapsackBlockRngdChange`, `apply()`-ed with `doUndo = true` to obtain the inverse, the modified block explored, and the change rolled back with the returned UndoChange. This is the right tool when the edit must be *serialised*, *transmitted*, or *undone* — not for an ordinary in-process re-solve, for which the `chg_*()` path above is simpler. **Status — beta.**

Change the data by name, through the methods factory. `BinaryKnapsackBlock` registers `chg_weights`, `chg_profits` and `chg_capacity` in the methods factory (Chapter 15), so the same edit made above through the concrete type can also be issued by *string name* on a base `Block*`:

```
auto fun = bkb.get_method< Block::MS_dbl_rngd >(
                            "BinaryKnapsackBlock::chg_profits" );
fun( & bkb , std::vector< double >{ 8.0 }.begin() ,
     Block::Range( 2 , 3 ) , eModBlck , eModBlck );
```

This is exactly how type-agnostic machinery (a `StochasticBlock` scenario realiser, a meta-configuration driver) changes a `BinaryKnapsackBlock`'s data without a compile-time dependency on its type.

Recipe R3 — CFL three ways: cuts / MCF relaxation / Lagrangian

> **Counterpart in the source tree:** `tests/CapacitatedFacilityLocation/` — one executable (`CFL_test`) and three configuration folders, `cuts/`, `MCF/`, `LD/`, each holding the same set of files with different contents. This recipe is a reading of that tester.

Goal

Solve the *same* Capacitated Facility Location instance in three completely different ways — a strengthened MILP, a fast min-cost flow relaxation, and a Lagrangian dual — **without recompiling**, by editing only the configuration files. This is the recipe that shows, at full strength, the SMS++ separation of *what the model is* from *how it is solved*: one program, three solution strategies, selected entirely by configuration.

Concepts used

- `R3Block`: producing a relaxation and keeping it in sync — Chapter 10 (and `UpdateSolver`).
- `Configuration`: `BlockConfig` to pick the formulation, `BlockSolverConfig` to pick the solver — Chapter 11.
- Sub-`Block` decomposition (the KskForm tree) — Chapter 12.
- `LagBFunction` / `LagrangianDualSolver` — Chapter 14.

The pattern

The driver is configuration-agnostic. In skeleton form (the `CFL_test` of the source tree, condensed):

```
// B1 is the original CFL problem, read from the instance file
auto * B1 = dynamic_cast< CapacitatedFacilityLocationBlock * >(
              Block::deserialize( instance_file ) );

// B2 is an R3Block of B1; WHICH one is read from R3BCfg.txt
Configuration * r3bc = /* deserialized from R3BCfg.txt */ ;
Block * B2 = B1->get_R3_Block( r3bc );

// configure the two Blocks (formulation, options) from BPar1/BPar2.txt
B1->set_BlockConfig( /* from BPar1.txt */ );
B2->set_BlockConfig( /* from BPar2.txt */ );

// attach a Solver to each, from BSPar1/BSPar2.txt
/* BlockSolverConfig from BSPar1.txt */ ->apply( B1 );
/* BlockSolverConfig from BSPar2.txt */ ->apply( B2 );

// keep B2 in sync with B1 automatically (Chapter 10)
B1->register_Solver( new UpdateSolver( B2 , r3bc ) );

if( niter == 0 ) {                  // just solve B2 and compare
 B2->get_registered_solvers().front()->compute();
```

```
}
else {                              // a slope-scaling loop
 for( Index h = 0 ; h < niter ; ++h ) {
  B2->get_registered_solvers().front()->compute();  // solve the relaxation
  B1->map_back_solution( B2 , r3bc );                // bring solution to B1
  // ... use the (fractional) solution to nudge B1's facility costs;
  //     each chg is forwarded to B2 by the UpdateSolver ...
 }
}
```

The whole behaviour is decided by four configuration files — `R3BCfg.txt` (which R3Block), `BPar2.txt` (which formulation / options for it), `BSPar2.txt` (which solver) — and it is *those*, not the code, that differ between the three folders.

The three configurations

	`cuts/`	`MCF/`	`LD/`
`R3BCfg.txt` (R3Block)	`0` — a copy `CFLB`	`1` — an `MCFBlock` flow relaxation	`0` — a copy `CFLB`
`BPar2.txt` (formulation)	Natural (StdForm, `wf=0`) with strong forcing cuts enabled	(none — `MCFBlock` has no formulation choice)	Knapsack (KskForm, `wf=1`)
`BSPar2.txt` (solver)	`CPXMILPSolver`	`MCFSolver<MCFCplex>`	`LagrangianDualSolver` (inner `BundleSolver` on the per-facility knapsacks)
what it computes	the bound of the natural formulation strengthened by the dynamically separated $x_{ij} \leq y_i$ cuts	a weak but very cheap lower bound, from the continuous flow relaxation	the Lagrangian bound (in theory, *equal* to the LP+cuts bound) and a convexified primal solution

The three sit at different points of the bound-quality / cost trade-off:

- **`cuts/`** — a `:MILPSolver` on the natural formulation, with the strong forcing constraints $x_{ij} \leq y_i$ separated on demand (Chapter 12 mentioned this dynamic-constraint group). A strong bound, at the highest per-solve cost.
- **`MCF/`** — the `MCFBlock` flow relaxation of §10.5, solved by a network-specialised `MCFSolver`. The weakest bound and the most fractional solution, but obtained extremely fast.
- **`LD/`** — the Knapsack formulation solved by `LagrangianDualSolver`: the master's customer-satisfaction constraints are dualised, leaving one `LagBFunction` per facility (over its `BinaryKnapsackBlock` sub-`Block`), driven by a `BundleSolver` (Chapters 12, 14, Recipe R4). A strong bound and a good convexified solution, at a cost between the other two.

A theoretical point worth making explicit: the bound that `cuts/` and `LD/` compute is, *in theory, exactly the same*. Dualising the customer-satisfaction constraints leaves a per-facility knapsack subproblem; the Lagrangian dual bound therefore equals the value of the natural LP *strengthened by the convex hull of those knapsacks* — which is precisely what the strong forcing cuts $x_{ij} \leq y_i$ approach. This is the standard Lagrangian / Dantzig–Wolfe equivalence, and it means `cuts/` and `LD/` are two routes to one and the same bound, differing in *how* they reach it (branch-and-cut on a MILP engine versus a bundle method on the dual), not in the bound itself. The `MCF/` flow relaxation is the genuinely *weaker* one.

Walk-through

- `B1` is the genuine CFL problem; `B2` is whatever `R3BCfg.txt` asks `get_R3_Block` to produce — *a copy of `B1`* in the `cuts/` and `LD/` cases (so that `B2` can be configured into a different *formulation* and solved without disturbing `B1`), or an *`MCFBlock` flow relaxation* in the `MCF/` case (§10.5).

- The two `set_BlockConfig` calls decide the *formulation*: in `cuts/`, `B2` is the natural MILP with the strong cuts enabled; in `LD/`, `B2` is the Knapsack formulation, which grows the per-facility `BinaryKnapsackBlock` sub-`Block` tree of Chapter 12; in `MCF/`, `B2` is already an `MCFBlock` and needs no formulation choice.
- The two `BlockSolverConfig`s decide the *solver*. Switching the whole strategy is a matter of pointing the driver at a different folder.
- The `UpdateSolver` registered on `B1` is what makes the slope-scaling loop work: when the loop nudges `B1` 's facility costs, the change is `map_forward`-ed to `B2` automatically (Chapter 10), so the relaxation stays faithful to the (modified) problem without any explicit re-synchronisation.
- `map_back_solution(B2, r3bc)` brings `B2`'s (relaxation) solution back into `B1`, where the heuristic reads it.

Expected behaviour

This recipe is about *qualitative* contrast, not a single number. On a typical CFL instance one observes:

- `MCF/` returns the lowest (weakest) lower bound, fastest, with the most fractional transportation solution;
- `cuts/` and `LD/` return the *same* bound in theory (the Lagrangian / Dantzig–Wolfe equivalence above), `LD/` also producing a markedly less fractional convexified solution;
- both are far above the `MCF/` bound.

On why `cuts/` and `LD/` may nonetheless print *different* numbers. A reader running the tester will often see the `cuts/` bound come out slightly *better* than the `LD/` one, which seems to contradict the equivalence just stated. The discrepancy is not in the mathematics but in what the MIP engine does: with the default `intRelax IntVars == 0`, `cuts/` declares the problem to the back-end *as a MILP*, and the back-end (CPLEX, say) applies its MIP presolve / bound-strengthening at the root node, which can tighten the reported bound beyond the pure LP-with-cuts value. That extra strengthening, not a difference in the underlying relaxation, is what makes the two numbers differ.

To recover the equivalence *exactly*, set `intRelaxIntVars == 2` in the `CPXMILPSolver`'s `ComputeConfig` (in `cuts/BSPar2.txt`): in that mode (`MILPSolver.h`, §6.4) the back-end solves the problem *as a pure LP* — no MIP presolve — and `MILPSolver` itself drives the cut-separation loop, calling `generate_dynamic_constraints()` after each LP solve. With MIP preprocessing thus switched off, the `cuts/` bound coincides with the `LD/` Lagrangian bound, as the theory predicts.

The exact numbers depend on the instance and the installed MIP back-end; the point that survives every instance is that **the same executable produced all three** by configuration alone.

Variations

Swap the MIP back-end. In `cuts/BSPar2.txt`, replacing `CPXMILPSolver` with `SCIPMILPSolver`, `GRBMILPSolver` or `HiGHSMILPSolver` (one line) changes the underlying MIP engine with no other change — the open-source HiGHS being the choice that needs no commercial licence.

Run the slope-scaling heuristic. Passing a non-zero `niter` turns the "solve once and compare" path into the slope-scaling loop sketched above: solve the relaxation, map the solution back, adjust `B1`'s costs, repeat. With the `MCF/` or `LD/` relaxation this produces a feasible CFL solution (an *upper* bound) to set against the lower bound, all from the one driver.

Go to Lagrangian + heuristic, or to Benders. Recipe R4 takes the `LD/` configuration further, adding `PrimalProximalHeur` to turn the convexified solution into a feasible one; Recipe R5 solves the *same* CFL by the Benders formulation instead. Both are reached, again, by changing configuration rather than code.

Recipe R4 — CFL via Lagrangian decomposition with `PrimalProximalHeur`

> **Counterpart in the source tree:** `tests/CapacitatedFacilityLocation/LD/` (the `LD` configuration folder of the `CFL_test` executable of Recipe R3).

Goal

Solve a Capacitated Facility Location problem by *Lagrangian decomposition*: put the problem in its Knapsack formulation, dualise the customer-satisfaction constraints with a `LagrangianDualSolver`, and obtain a strong lower bound together with a convexified primal solution. Then turn that convexified (fractional) solution into a *feasible* one by swapping in `PrimalProximalHeur` — a one-line configuration change. This recipe shows the decomposition machinery of Chapters 12 and 14 doing real work.

Concepts used

- Sub-`Block` decomposition (the KskForm tree) — Chapter 12.
- `LagBFunction` and `LagrangianDualSolver` — Chapter 14.
- `BundleSolver` and the reoptimization vocabulary — Chapter 13.
- `Configuration` (the formulation and solver choice) — Chapter 11.

What the configuration sets up

The driver is the same `CFL_test` of Recipe R3; only the `LD/` configuration files matter here. They arrange:

- `BPar2.txt` puts the (R3-copy) CFL `Block` in the **Knapsack formulation** (`SimpleConfiguration<int>` value `1`). As Chapter 12 described, this grows one `BinaryKnapsackBlock` sub-`Block` per facility, leaving only the customer-satisfaction linking constraints $\sum_i X_{ij} = 1$ in the master.
- `BSPar2.txt` attaches a single solver named **`LagrangianDualSolver`**, with a (large) `ComputeConfig` whose parameters configure the inner `BundleSolver` that drives the dual (norm-based stopping, bundle size per component, master- problem solver, the `m1`/`m2`/`m3` serious/null-step thresholds, ...).

What `LagrangianDualSolver` does with this (Chapter 14, §14.2): it stealthily builds a hidden `LagrangianDualBlock`, wraps each per-facility `BinaryKnapsackBlock` in a `LagBFunction` that dualises the linking constraints, and runs the inner `BundleSolver` over the sum of those `LagBFunctions`. Each `LagBFunction` is evaluated by solving its inner knapsack — by a `DPBinaryKnapsackSolver` or an `:MILPSolver`, as configured on the leaf — and the resulting per-facility solutions are combined into the *convexified* primal solution that the dual returns alongside the bound.

```
          LagrangianDualBlock (hidden, built by LagrangianDualSolver)
            |   BundleSolver drives the dual over the sum below
            +-- LagBFunction_0  --> BinaryKnapsackBlock (facility 0)  --> DP solver
            +-- LagBFunction_1  --> BinaryKnapsackBlock (facility 1)  --> DP solver
            +-- ...
master CFL Block: customer-satisfaction linking constraints (dualised)
```

The setup, in code

In a program (rather than via the configuration files) the same arrangement is:

```
#include "CapacitatedFacilityLocationBlock.h"
#include "LagrangianDualSolver.h"
#include "BlockSolverConfig.h"

using namespace SMSpp_di_unipi_it;

int main()
{
 CapacitatedFacilityLocationBlock cfl;
 cfl.load( /* ... */ );

 // Knapsack formulation: grows one BinaryKnapsackBlock per facility.
 // We also tell generate_abstract_constraints() to build ONLY the
 // customer-satisfaction (demand) constraints at the master level (the
 // "wc" bit 0), because the facility-capacity constraints are the
 // knapsack constraints living inside the per-facility BinaryKnapsackBlock
 // sub-Blocks and must not be duplicated at the master.
 auto * bc = new BlockConfig;
 bc->f_static_variables_Configuration   = new SimpleConfiguration< int >( 1 );
 bc->f_static_constraints_Configuration = new SimpleConfiguration< int >( 1 );
 cfl.set_BlockConfig( bc );
 cfl.generate_abstract_variables();      // builds the sub-Block tree
 cfl.generate_abstract_constraints();    // only the "sat" linking constraints
 cfl.generate_objective();

 // attach a LagrangianDualSolver (its ComputeConfig, naming the
 // inner BundleSolver and the leaf knapsack solver, is read from a file)
 auto * bsc = new BlockSolverConfig;
 std::ifstream cfg( "BSPar-LDS.txt" ); bsc->load( cfg );
 bsc->apply( & cfl );

 cfl.get_registered_solvers().front()->compute();   // solve the dual

 std::cout << "Lagrangian bound = " << cfl.get_valid_lower_bound() << '\n';
 // the convexified primal solution now sits in cfl's variables

 bsc->clear(); bsc->apply( & cfl ); delete bsc;      // cleanup (cf. §11.8)
 return 0;
}
```

Walk-through

- The `BlockConfig` selects KskForm (§11.8); the `generate_abstract_*` calls build the per-facility `BinaryKnapsackBlock` tree (§12.5). The `f_static_constraints_Configuration` of value `1` is what makes `generate_abstract_constraints()` build *only* the customer-satisfaction demand constraints in the master (the `wc` bit-mask of §11.3 / §7.3): the facility-capacity constraints are not generated at the master at all, because in KskForm they are the knapsack constraints of the `BinaryKnapsackBlock` sub-`Block`s. Omitting this — letting `generate_abstract_constraints()` build everything — would wrongly duplicate the capacity constraints at the master level.
- `LagrangianDualSolver`, once attached, is a `CDASolver` (Chapter 6): `compute()` solves the Lagrangian dual, and the bound is read with `get_valid_lower_bound()`, the convexified primal solution from the `Block`'s variables.
- The reoptimization theme of Chapter 13 is what makes the inner loop efficient: as the `BundleSolver` moves the multipliers, the changes to each inner `BinaryKnapsackBlock` are mapped by its `LagBFunction` into the appropriate `FunctionMod*`, so the bundle is reoptimized rather than rebuilt (§14.2).

From a bound to a feasible solution: `PrimalProximalHeur`

`LagrangianDualSolver` returns a bound and a *convexified* primal solution, which is generally **fractional** — not a feasible CFL solution. To obtain a feasible one, `PrimalProximalHeur` is a *drop-in derived solver* (§14.2): it derives from `LagrangianDualSolver`, so it sets up the same dual, and adds the Lagrangian-based primal-proximal heuristic on top — compute the convexified solution, round it, add a quadratic penalty $M\|x - \lceil x \rfloor\|$ to push subsequent dual solutions towards that rounding, and iterate. Switching to it is a one-line change in `BSPar2.txt`: replace `LagrangianDualSolver` with `PrimalProximalHeur`. The new solver exposes two extra parameters, `dbl_penaltyFactor` (the M) and `intMaxIterLD` (the number of inner Lagrangian-dual iterations), in addition to all of `LagrangianDualSolver`'s.

Expected behaviour

On a typical CFL instance:

- `LagrangianDualSolver` returns a *strong* lower bound — equal, in theory, to the LP-with-strong-cuts bound of Recipe R3's `cuts/` (the Dantzig–Wolfe equivalence noted there) — together with a convexified primal solution that is markedly *less fractional* than the `MCF/` relaxation's, but still not integer-feasible in general.
- `PrimalProximalHeur` returns the same bound plus a *feasible* (integer, rounded) primal solution, i.e. a valid *upper* bound, so the gap between the two can be reported. The quality of the feasible solution depends on the penalty factor M, which is the heuristic's main tuning knob.

Variations

Choose the leaf solver. The per-facility knapsacks can be solved by `DPBinaryKnapsackSolver` (fast, exact for integer weights) or by an `:MILPSolver`; this is a configuration choice on the leaf `Blocks`, not a change to the decomposition.

Go parallel. Replacing `BundleSolver` with `ParallelBundleSolver` in the inner configuration evaluates the per-facility `LagBFunctions` concurrently (Chapter 17, §17.4) — the decomposition into independent knapsacks is exactly what makes this sound.

Tune the heuristic. `dbl_penaltyFactor` trades feasibility pressure against fidelity to the convexified solution; too small an M may not round to anything feasible, too large an M pins the solution to a possibly poor rounding. This is the heuristic's known sensitivity, flagged honestly in §14.2.

Recipe R5 — CFL via Benders cuts with a user-cut callback

> **Counterpart in the source tree:** `tests/CapacitatedFacilityLocation/Ben/` (the `Ben` configuration folder of the `CFL_test` executable of Recipe R3).

Goal

Solve a Capacitated Facility Location problem by *Benders decomposition*: put it in its Benders formulation, where the master keeps only the design variables and an epigraphic variable, and the transportation sub-problem is a hidden `MCFBlock` wrapped in a `BendersBFunction`. An `:MILPSolver` solves the master, and its user-cut callback emits Benders optimality (and, in the feasibility-cuts variant, feasibility) cuts on demand. This recipe shows the `BendersBFunction` of Chapter 14 and the dynamic-constraint generation of Chapter 7 working together.

Concepts used

- The MCF flow relaxation as the inner sub-problem — Chapter 10, §10.5.
- `BendersBFunction` — Chapter 14, §14.3.
- On-demand `generate_dynamic_constraints()` — Chapter 7, §7.3.
- `Configuration`, including `f_extra_Configuration` for the hidden inner `Block` — Chapter 11, §11.6.

What the configuration sets up

Again the driver is `CFL_test`; the `Ben/` files arrange:

- `BPar2.txt` puts the CFL `Block` in the **Benders formulation** (`SimpleConfiguration<int>` value `3`, the *feasibility-cuts* variant; value `2` selects the slack-arcs variant). The master abstract representation then has only the design variables y_i, a single epigraphic variable v, the bound $v \geq \mathrm{LB}(v)$, and a *dynamic* group of Benders cuts; the transportation sub-problem is held in a hidden `BendersBFunction` wrapping an `MCFBlock` (the very flow relaxation of §10.5), built by `build_BendersBFunction()` and *not* a sub-`Block` of the CFL `Block`.
- That hidden `BendersBFunction` and its inner `MCFBlock` cannot be reached by the recursive configuration machinery (the inner `Block` has no father), so they are configured through `f_extra_Configuration` (§11.6): in `Ben/BPar2.txt` it is a `SimpleConfiguration<std::pair<Configuration*,Configuration*>>` whose first element carries the R3-Block / slack-scaling parameters and whose second is the `ComputeConfig` of the `BendersBFunction` (which in turn configures, and attaches a `:Solver` to, the inner `MCFBlock`).
- `BSPar2.txt` attaches the master solver — a `GRBMILPSolver` (Gurobi) in the shipped configuration; any `:MILPSolver` works.

How a solve proceeds

1. The `:MILPSolver` works on the master MIP over (y, v).
2. Whenever it has a candidate design $\hat{y}$, its user-cut / lazy callback fires and calls the CFL `Block`'s `generate_dynamic_constraints()` (§7.3).

3. That evaluates the `BendersBFunction` at $\hat{y}$ — *one solve of the hidden* `MCFBlock` — to obtain the transportation cost $\varphi(\hat{y})$ and a linearization (§14.3).
4. A Benders cut is added to the master's dynamic-constraint group: an *optimality* cut from the `MCFBlock`'s dual when $\hat{y}$ is feasible for the transportation problem, or — in the feasibility-cuts variant — a *feasibility* cut from the Farkas ray when it is not.
5. The master continues, now constrained by the new cut, until no violated cut is found and the incumbent is optimal.

The whole loop is driven by the master `:MILPSolver`; SMS++ supplies the `BendersBFunction` evaluation and the cut, the `:MILPSolver` supplies the branch-and-cut and the callback hook.

Walk-through

- The Benders formulation is selected exactly as the other CFL formulations are — a single integer in `f_static_variables_Configuration` (§11.8) — value `2` or `3` choosing the slack-arcs or feasibility-cuts variant.
- The hidden `MCFBlock` is the same flow relaxation that Recipe R3's `MCF/` configuration uses directly (§10.5); here it is *internal* to the `BendersBFunction` rather than a user-visible R3Block, the double duty noted in §10.5.
- The cut emission rides on the existing dynamic-constraint mechanism (§7.3): no special Benders plumbing in the driver, just a `generate_dynamic_constraints()` that the `:MILPSolver`'s callback calls.

`BendersDecompositionSolver`

In this recipe the cuts are emitted by the user-cut callback of the master `:MILPSolver`, and therefore no Benders-specific `:Solver` is needed. Since version 0.7.0 the master can also be built automatically by `BendersDecompositionSolver` (§14.3), the Benders counterpart of `LagrangianDualSolver` (Recipe R4). On the CFL this is exercised by `tests/CapacitatedFacilityLocationBlock/test_bds.cpp`, which writes the instance as a two-level `AbstractBlock` (the design `Variable`s at the root, the transportation subproblems below) and compares the value given by both regimes with that of the monolithic solve of the same model. The two arrangements differ, however, in where the cuts go. With the callback they enter the branch-and-cut of the master while it proceeds; with `BendersDecompositionSolver` the master is usually solved again after each round of cuts (MILP regime), or stabilized by a bundle method (convex regime).

Expected behaviour

The Benders master converges to the *same optimum* as the direct MILP of Recipe R3's `cuts/` (it is the same problem, decomposed), solving a sequence of cheap `MCFBlock` transportation problems instead of carrying the transportation variables in the master. Whether this is faster than the monolithic MILP depends on the instance: Benders pays off when the design space is small relative to the transportation one (few facilities, many customers), which is the regime CFL often inhabits.

Variations

Slack-arcs versus feasibility-cuts. Value `2` builds the inner `MCFBlock` with big-M slack arcs, so $\varphi(\hat{y})$ is finite for *every* design and only optimality cuts are emitted; value `3` builds it without slack arcs, so an infeasible $\hat{y}$ yields a feasibility cut from the Farkas ray (§3.5, §14.3). The feasibility-cuts variant preserves infeasibility detection and is, on the shipped test instances, both faster and at least as robust under modifications; the slack-arcs variant is the choice when a finite φ is preferred over an explicit infeasibility report.

Swap the master MIP back-end. As in Recipe R3, the `GRBMILPSolver` in `BSPar2.txt` can be replaced by `CPXMILPSolver`, `SCIPMILPSolver` or `HiGHSMILPSolver` with no other change.

Configure the inner MCF solver. The `:Solver` that evaluates the hidden `MCFBlock` is set through the `ComputeConfig` carried in `f_extra_Configuration` (§11.6); forgetting to provide it is the most common BenForm setup error, since BenForm *requires* a `:Solver` attached to the inner `Block`.

Appendix A — Writing a new `:Block`

This appendix walks through the construction of a new `:Block` from scratch. The running example is a deliberately small `BinPackingBlock`, chosen because the Bin Packing problem is just rich enough to exercise every essential step and small enough to fit in an appendix.

> **Note.** `BinPackingBlock` is a *pedagogical* example written for this manual; it is **not** part of the SMS++ source tree. The code fragments are illustrative — realistic SMS++ C++, but not a production class. The aim is to show the *process* of writing a minimum-viable `:Block`, i.e. the steps A.1–A.7 below. The optional pieces (a custom `:Solution`, an `R3Block`, a `:Change` family) are sketched in §A.8 but not developed.
>
> **Tip.** This appendix covers the *code*: the class itself. For the *repository* around it — the CMake and makefile builds, the CI, the test harness and the standard boilerplate every SMS++ module shares — do not start from scratch: clone the ModuleTemplate repository and run its `init.sh`, which generates a complete module skeleton in the standard layout (see the wiki page Creating a new module).

The Bin Packing problem: given n items of sizes $s_0, \ldots, s_{n-1}$ and bins of identical capacity C, pack every item into a bin so that no bin's contents exceed C, using as few bins as possible. Since no more than n bins are ever needed, we cap the number of bins at $m = n$.

A.1 The "physical" representation and a choice of "abstract" one

A useful first distinction (Chapter 7) is that the *physical* representation is not really *chosen*: it is dictated by the semantics of the problem — the data that defines an instance and that a specialised solver reads directly. What is genuinely a *design decision* is the *abstract* representation, because a given problem admits many equivalent mathematical formulations and the `:Block` author picks one (or, as `MCFBlock` and CFL do, makes the choice configurable, §7.3).

For Bin Packing the physical representation is dictated and tiny: the number of items n, the capacity C, and the vector of sizes s. That is all a specialised solver would need.

The abstract representation chosen here is the natural MILP (one formulation among several possible ones):

$$\min \sum_j y_j$$

subject to $\sum_j x_{ij} = 1$ for every item i (each item in exactly one bin), $\sum_i s_i x_{ij} \le C y_j$ for every bin j (capacity, and a bin counts as used when something is in it), and $x_{ij}, y_j \in \{0, 1\}$. So the abstract representation has two `ColVariable` groups — `x`, a `boost::multi_array< ColVariable , 2 >` of shape $n \times m$, and `y`, a `std::vector< ColVariable >` of size m — two `FRowConstraint` groups (assignment and capacity), and one `FRealObjective`.

A.2 The class skeleton and factory registration

A concrete `:Block` derives from `Block`, takes an optional father in its constructor, and registers itself in the `Block` factory (Chapter 18):

```
// BinPackingBlock.h
#include "Block.h"
#include "ColVariable.h"
#include "FRowConstraint.h"
#include "FRealObjective.h"
#include "LinearFunction.h"

namespace SMSpp_di_unipi_it {

class BinPackingBlock : public Block
{
 public:
  explicit BinPackingBlock( Block * father = nullptr ) : Block( father ) {}

  virtual ~BinPackingBlock() {
   // explicitly clear the ThinVarDepInterface fields (Constraints and the
   // Objective's Function) BEFORE the ColVariable groups they point into are
   // destroyed; see the discussion below
   Constraint::clear( v_assign );
   Constraint::clear( v_cap );
   f_obj.clear();
  }

  void load( Index n , double C , std::vector< double > sizes );

  // physical-representation accessors a specialised Solver reads
  Index  get_NItems  ( void ) const { return( f_n ); }
  double get_Capacity( void ) const { return( f_C ); }
  double get_Size( Index i )  const { return( v_s[ i ] ); }

  void generate_abstract_variables( Configuration * = nullptr ) override;
  void generate_abstract_constraints( Configuration * = nullptr ) override;
  void generate_objective( Configuration * = nullptr ) override;

  void chg_size( double new_size , Index item ,
                 ModParam issueMod = eModBlck , ModParam issueAMod = eModBlck );

  // deposit a solution in compact (physical) form and reflect it into the
  // abstract Variables; implemented and discussed in Appendix B (§B.6)
  void set_assignment( const std::vector< Index > & bin_of );

  bool is_feasible( bool useabstract = false , Configuration * = nullptr )
   override;

  void add_Modification( sp_Mod mod , ChnlName chnl = 0 ) override;

  void load( std::istream & , char = 0 ) override { /* text format */ }

 private:
  // decode an abstract Modification into a physical change (§A.6)
  void guts_of_add_Modification( c_p_Mod mod , ChnlName chnl );

  // physical representation
  Index               f_n = 0;
  double              f_C = 0;
  std::vector< double > v_s;          // item sizes

  // compact (physical) solution: for each item, the index of its bin;
  // a single n-vector, as opposed to the n*m + m abstract binaries (§B.6)
  std::vector< Index >  v_bin_of;

  // abstract representation (built on demand)
  boost::multi_array< ColVariable , 2 > v_x;   // x[ i ][ j ]
  std::vector< ColVariable >            v_y;   // y[ j ]
  std::vector< FRowConstraint >         v_assign;   // one per item
```

```
  std::vector< FRowConstraint >         v_cap;      // one per bin
  FRealObjective                        f_obj;

  SMSpp_insert_in_factory_h;          // factory hook (Chapter 18)
};

}  // namespace
```

```
// BinPackingBlock.cpp
SMSpp_insert_in_factory_cpp_1( BinPackingBlock );   // ctor takes 1 arg
```

The `SMSpp_insert_in_factory_cpp_1` macro (the `_1` because the constructor takes one optional argument, the father) registers the class so that `Block::new_Block("BinPackingBlock")` works, and runs the class's `static_initialization()` at start-up (Chapter 18, §18.1; mind the linker caveat of §18.2).

The non-trivial destructor deserves a word, because it follows a framework convention that is easy to miss. A `Constraint` (like any `ThinVarDepInterface`) and the `Variables` it is "active" in are *doubly* linked: each side holds pointers into the other. The framework's invariant (the `ThinVarDepInterface` documentation) is that *`Variables` are constructed before the things they are active in and destroyed after them*, so that when a `Constraint` (or a `Function` inside an `Objective`) is destroyed, the `Variables` it references are still live and the back-pointers can be cleaned up safely. Inside a `:Block`, however, the order in which member fields are destroyed is "usually not very clear" — it is the reverse of declaration order, which is fragile to rely on — so the recommendation is to *explicitly* clear the dependent objects in the destructor, guaranteeing that every `Constraint` and the `Objective`'s `Function` is destroyed (and unlinks itself) before the `ColVariable` groups `v_x` / `v_y` they point into. `clear()` on each object does exactly this; `Constraint.h` provides static `Constraint::clear(...)` helpers that apply it to every group shape — `std::vector`, `std::vector` of `std::vector`, `boost::multi_array`, `std::list`, and their combinations — so the whole groups can be cleared with one call each. (Were the order left to the compiler, a `ColVariable` might be destroyed while a `Function` still held a pointer to it, leaving a dangling reference.)

A.3 `load()` and the physical representation

`load()` simply copies the data and, if any `:Solver` is already attached, issues the "nuclear" `NBModification` (§4.6) to tell it the instance has been replaced wholesale:

```
void BinPackingBlock::load( Index n , double C , std::vector< double > sizes )
{
 f_n = n;
 f_C = C;
 v_s = std::move( sizes );

 if( anyone_there() )                       // a Solver is listening
  add_Modification( std::make_shared< NBModification >( this ) );
}
```

At this point the `BinPackingBlock` is fully usable by a *specialised* solver (Appendix B) that reads `f_n`, `f_C`, `v_s` directly. The abstract representation does not exist yet.

A.4 Generating the abstract representation

The three `generate_*` overrides build the abstract face on demand (Chapter 7). `generate_abstract_variables()` materialises the `ColVariable` groups, declaring each binary:

```
void BinPackingBlock::generate_abstract_variables( Configuration * )
{
 if( ! v_y.empty() )  return;               // already done

 v_x.resize( boost::extents[ f_n ][ f_n ] );  // m == n bins
 for( auto & row : v_x ) for( auto & xij : row )
```

```
  xij.set_type( ColVariable::kBinary );

 v_y.resize( f_n );
 for( auto & yj : v_y ) yj.set_type( ColVariable::kBinary );

 add_static_variable( v_x , "x" );          // register the groups
 add_static_variable( v_y , "y" );
}
```

`generate_abstract_constraints()` builds the assignment and capacity rows, each an `FRowConstraint` carrying a linear `Function`. A `Configuration` could gate which groups are built (as `MCFBlock` and CFL do, §7.3); here we build both:

```
void BinPackingBlock::generate_abstract_constraints( Configuration * )
{
 if( ! v_assign.empty() )  return;

 // assignment: sum_j x[i][j] == 1  for each item i
 v_assign.resize( f_n );
 for( Index i = 0 ; i < f_n ; ++i ) {
  // a linear Function summing x[i][0..m-1] with coeff 1: build the
  // (Variable* , coefficient) pairs, one per bin j, then hand the new
  // LinearFunction (constant term 0) to the FRowConstraint
  LinearFunction::v_coeff_pair cp( f_n );
  for( Index j = 0 ; j < f_n ; ++j ) {
   cp[ j ].first  = & v_x[ i ][ j ];
   cp[ j ].second = 1.0;
  }
  v_assign[ i ].set_function( new LinearFunction( std::move( cp ) , 0 ) ,
                              eNoMod );      // not built yet, no Solver to tell
  v_assign[ i ].set_both( 1.0 );             // == 1
 }

 // capacity: sum_i s_i x[i][j] - C y[j] <= 0  for each bin j
 v_cap.resize( f_n );
 for( Index j = 0 ; j < f_n ; ++j ) {
  // linear Function: sum_i s_i x[i][j] - C y[j]. The n item terms come
  // first (so coefficient k corresponds to x[k][j], matching chg_size
  // and the abstract-stream decoding in add_Modification), then the
  // single -C y[j] term
  LinearFunction::v_coeff_pair cp( f_n + 1 );
  for( Index i = 0 ; i < f_n ; ++i ) {
   cp[ i ].first  = & v_x[ i ][ j ];
   cp[ i ].second = v_s[ i ];
  }
  cp[ f_n ].first  = & v_y[ j ];
  cp[ f_n ].second = - f_C;
  v_cap[ j ].set_function( new LinearFunction( std::move( cp ) , 0 ) , eNoMod );
  v_cap[ j ].set_lhs( -Inf< double >() );
  v_cap[ j ].set_rhs( 0.0 );                 // <= 0
 }

 add_static_constraint( v_assign , "assign" );
 add_static_constraint( v_cap , "cap" );
}
```

`generate_objective()` sets the minimisation of $\sum_j y_j$:

```
void BinPackingBlock::generate_objective( Configuration * )
{
 // linear Function: sum_j 1 * y[j]
 LinearFunction::v_coeff_pair cp( f_n );
 for( Index j = 0 ; j < f_n ; ++j ) {
```

```
  cp[ j ].first  = & v_y[ j ];
  cp[ j ].second = 1.0;
 }
 f_obj.set_function( new LinearFunction( std::move( cp ) , 0 ) , eNoMod );
 f_obj.set_sense( Objective::eMin , eNoMod );
 set_objective( & f_obj , eNoMod );
}
```

Two things to read off these snippets. First, the linear expressions are `LinearFunction` objects (Chapter 13), built from a `LinearFunction::v_coeff_pair` — a vector of `(ColVariable * , Coefficient)` pairs — plus a constant term; ownership of each `new LinearFunction(...)` passes to the `FRowConstraint` / `FRealObjective` that receives it via `set_function()`, §5.7. Second, every `set_*` here is passed `eNoMod`: the abstract representation is being built for the first time, so there is nothing to notify — no `Modification` need (or should) be issued (§8.3). The deliberate ordering of the capacity row's terms (the n item coefficients first, then the single $-Cy_j$ term) is what lets `chg_size` (§A.5) and the abstract-stream decoding in `add_Modification` (§A.6) address coefficient k as "the x_{kj} term".

A.5 A `chg_*()` method and its `Modification`

A `:Block` author exposes mutators for the data that may change. `chg_size` changes one item's size in the physical representation and, when the abstract representation exists, mirrors the change there — issuing the appropriate `Modification`s on each face (Chapter 8). Giving it one of the methods-factory shapes (§15.3) would also let it be called by name; here the single-item form:

```
void BinPackingBlock::chg_size( double new_size , Index item ,
                                ModParam issueMod , ModParam issueAMod )
{
 if( v_s[ item ] == new_size )  return;     // nothing to do
 v_s[ item ] = new_size;                    // physical change

 // physical Modification, for a specialised Solver
 if( issue_pmod( issueMod ) )
  add_Modification( std::make_shared< BinPackingBlockMod >(
                    this , item ) , Observer::par2chnl( issueMod ) );

 // mirror into the abstract representation, if it exists: the size
 // appears as the coefficient of x[item][j] in each capacity row j,
 // which (by construction, §A.4) is coefficient number `item` of that
 // row's LinearFunction. modify_coefficient() updates it and, driven by
 // issueAMod, issues the abstract C05FunctionModLin on its own. The
 // not_dry_run() guard is what makes this safe to call from
 // add_Modification() with issueAMod == eDryRun: there the abstract change
 // has *already* happened, so we must touch the physical face only
 if( not_dry_run( issueAMod ) && ( ! v_cap.empty() ) )
  for( Index j = 0 ; j < f_n ; ++j ) {
   auto * lf = static_cast< LinearFunction * >( v_cap[ j ].get_function() );
   lf->modify_coefficient( item , new_size , issueAMod );
  }
}
```

The `BinPackingBlockMod` is the `:Block`-specific physical `Modification` defined in Appendix C.

A.6 `add_Modification()`: catching abstract changes

The Janus discipline (Chapter 8, §8.4) requires that a change made through the *abstract* face be reflected in the physical one. The `:Block` does this by overriding `add_Modification()` to intercept the abstract `Modification`s whose `concerns_Block()` is `true`, apply the matching physical change, reset the flag, and forward:

```
void BinPackingBlock::add_Modification( sp_Mod mod , ChnlName chnl )
```

```
{
 if( mod->concerns_Block() ) {
  mod->concerns_Block( false );
  guts_of_add_Modification( mod.get() , chnl );
 }
 Block::add_Modification( mod , chnl );      // base mechanics (Chapter 8)
}
```

The decoding is delegated to a private helper. The *only* abstract change this `:Block` knows how to fold back into its physical data is a change to an item-size coefficient in a capacity row — i.e. a `C05FunctionModLinRngd` / `C05FunctionModLinSbst` on one of the `v_cap[j]` `LinearFunctions`, touching an x_{ij} coefficient (index $< n$). Anything else (a change to the structural $-Cy_j$ coefficient, to an assignment row, to the objective, an added or removed `Variable`, ...) has no physical counterpart, so the helper throws:

```
void BinPackingBlock::guts_of_add_Modification( c_p_Mod mod , ChnlName chnl )
{
 // helper: which capacity row owns this Function? (npos if none)
 auto cap_row = [ this ]( const Function * f ) -> Index {
  for( Index j = 0 ; j < f_n ; ++j )
   if( v_cap[ j ].get_function() == f )  return( j );
  return( Inf< Index >() );
 };

 // helper: fold one changed coefficient k of a capacity row back into v_s.
 // k must be an x-term (k < f_n); the n-th coefficient is the structural
 // -C * y[j] and must never change through the abstract face
 auto fold = [ & ]( LinearFunction * lf , Index k ) {
  if( k >= f_n )
   throw( std::logic_error( "BinPackingBlock: structural coefficient "
                            "changed through the abstract face" ) );
  // chg_size with eNoBlck on the physical face (tell specialised Solvers)
  // and eDryRun on the abstract face (it has already changed); §8.4
  chg_size( lf->get_coefficient( k ) , k , make_par( eNoBlck , chnl ) ,
            eDryRun );
 };

 if( auto tmod = dynamic_cast< const C05FunctionModLinRngd * >( mod ) ) {
  auto * lf = dynamic_cast< LinearFunction * >( tmod->function() );
  if( ( ! lf ) || ( cap_row( lf ) == Inf< Index >() ) )
   throw( std::logic_error( "BinPackingBlock: unsupported abstract change" ) );
  for( Index k = tmod->range().first ; k < tmod->range().second ; ++k )
   fold( lf , k );
  return;
 }

 if( auto tmod = dynamic_cast< const C05FunctionModLinSbst * >( mod ) ) {
  auto * lf = dynamic_cast< LinearFunction * >( tmod->function() );
  if( ( ! lf ) || ( cap_row( lf ) == Inf< Index >() ) )
   throw( std::logic_error( "BinPackingBlock: unsupported abstract change" ) );
  for( auto k : tmod->subset() )  fold( lf , k );
  return;
 }

 // any other abstract Modification has no physical counterpart here
 throw( std::logic_error( "BinPackingBlock: unsupported abstract change" ) );
}
```

Two honest caveats. First, *failing loud* is deliberate: a `:Block` is free to support only some abstract changes and to throw for the rest, exactly as `MCFBlock` does for arc add/remove (§8.4). Second, a subtlety peculiar to this formulation: the size s_i is *shared* across all m capacity rows, but the abstract face exposes it as m independent coefficients. When the change arrives on a single row $j^\star$, `fold` calls `chg_size` with `eDryRun`, which (by the guard added in §A.5) updates only the physical v_s and the originating row, leaving the sibling rows momentarily stale. A production-grade `BinPackingBlock` would either reject per-row coefficient edits outright

or re-propagate the new size to all m rows; the minimal example shows the mechanism on the row that changed and flags the consequence rather than hiding it.

A.7 `is_feasible()` on the physical representation

Finally, `is_feasible()` checks a candidate solution. Written on the *physical* representation it is cheap (§7.5): read each item's bin assignment from the abstract `x` variables (or from wherever the solution lives), and verify that every item is assigned to exactly one bin and that no bin exceeds capacity.

```
bool BinPackingBlock::is_feasible( bool useabstract , Configuration * )
{
 std::vector< double > load( f_n , 0.0 );
 for( Index i = 0 ; i < f_n ; ++i ) {
  int assigned = -1;
  for( Index j = 0 ; j < f_n ; ++j )
   if( v_x[ i ][ j ].get_value() > 0.5 ) {  // item i in bin j
    if( assigned >= 0 )  return( false );    // assigned twice
    assigned = int( j );
    load[ j ] += v_s[ i ];
   }
  if( assigned < 0 )  return( false );       // unassigned
 }
 for( Index j = 0 ; j < f_n ; ++j )
  if( load[ j ] > f_C + 1e-9 )  return( false );  // over capacity
 return( true );
}
```

The `useabstract` flag (§7.5) would select an alternative path that walks the `FRowConstraints` and calls their `feasible()`; for a leaf `:Block` like this the physical check above is the one a specialised solver wants.

A.8 Optional pieces (sketched)

A minimum-viable `:Block` ends at §A.7. A "complete" `:Block` typically adds, each only when the use case calls for it:

- a `:Solution` (Chapter 9) — a `BinPackingSolution` storing the per-item bin assignment, so a solution can be saved, restored and combined without going through the abstract `Variables`;
- an `R3Block` (Chapter 10) — at least the trivial copy, via `get_R3_Block` plus `map_back_solution` / `map_forward_solution`;
- a `:Change` family (Chapter 16) — for serialisable / undoable edits; **Status — beta** (Appendix C).

None of these is required to have a working, solver-attachable `:Block`; they are the difference between a teaching example and a production module.

A.9 Checklist

To recapitulate, a minimum-viable `:Block` needs:

1. a class deriving from `Block`, with a father-taking constructor and the `SMSpp_insert_in_factory_h` / `_cpp_k` macros (§A.2, Chapter 18);
2. the physical representation as member data, populated by `load()` / `deserialize()`, issuing `NBModification` on reload (§A.3);
3. the `generate_abstract_*()` overrides building the abstract face on demand (§A.4);
4. `chg_*()` mutators issuing physical (and, where relevant, abstract) `Modifications` (§A.5);
5. `add_Modification()` catching abstract changes and keeping the physical face in sync, or throwing for unsupported ones (§A.6);
6. `is_feasible()` (and, for an optimisation `:Block`, `get_objective_sense()` and valid bounds) on the physical representation (§A.7).

Appendix B — Writing a new :Solver

This appendix shows how to write a new :Solver, using a trivial first-fit greedy heuristic for the BinPackingBlock of Appendix A as the worked example. As there, the code is illustrative rather than production-grade. For the repository scaffolding around a new :Solver module (builds, CI, tests, boilerplate) start from the ModuleTemplate repository, as described in the wiki page Creating a new module.

B.1 Solver versus CDASolver

The first choice is the base class (Chapter 6):

- derive from Solver for a :Solver that produces *primal* solutions only;
- derive from CDASolver ("Convex Duality Aware") when the :Solver also produces *dual* solutions / bounds with an associated dual object — as MCFSolver and the MILPSolver family do.

A greedy Bin Packing heuristic produces a feasible assignment and no dual information, so it derives from Solver.

B.2 The compute() contract

compute(bool changedvars) is the heart of a :Solver (Chapter 6, §6.3). Its obligations:

- run the algorithm and return an sol_type. An *exact* solver returns kOK / kUnbounded / kInfeasible when it concludes; a *heuristic* one that finds a feasible solution but cannot prove optimality returns kLowPrecision; resource-limited stops return kStopTime / kStopIter; unrecoverable errors return a value $\geq$ kError (§6.3).
- honour changedvars (§6.1): with false, the caller guarantees the relevant Variable values are unchanged since the last call, so the :Solver may resume; with true it must assume they may have changed. (A one-shot heuristic can simply ignore the hint and recompute.)
- record the solution so that it can later be read. A :Solver deposits its solution into the Block — for a :Block with abstract Variables, by writing their values; the user then reads it through the Block (§6.9).

B.3 Parameters

A :Solver inherits the parameter machinery of §6.4 (set_par / get_par over integer / double / string slots: intMaxIter, dblMaxTime, ...) and may extend it with its own enum values. A simple heuristic may need none and can rely on the defaults; a :Solver that does add parameters extends int_par_type_S / dbl_par_type_S past their ...LastAlgPar sentinels, exactly as DPBinaryKnapsackSolver adds dblReopt (Recipe R2).

B.4 Reacting to Modifications

The framework fills the :Solver's pending list with the Modifications issued since it was attached (Chapter 8). A :Solver consumes that list at the start of compute() and reacts: an exact, reoptimizing solver uses the fine-grained information to warm-start (Chapter 13); a one-shot heuristic may simply note "the data changed, recompute from scratch", clearing the list. Either way, the :Solver is responsible for emptying its own list.

B.5 Optional: `compute_async` and `State`

`compute_async()` comes for free from `ThinComputeInterface` (§17.2); a `:Solver` only needs to ensure its `compute()` is thread-safe with respect to the `Block` (the recursive locking of §17.1). A `:Solver` with meaningful internal state implements `get_State()` / `put_State()` for checkpointing and warm-starting (§17.3); one without — like the greedy heuristic — returns `nullptr` and pays nothing.

B.6 Worked example: a first-fit heuristic for `BinPackingBlock`

The heuristic reads the physical representation of the `BinPackingBlock` (the sizes and capacity, through the accessors of §A.2), places each item into the first bin that has room (opening a new bin when none does), and deposits the resulting assignment into the `Block`. It also computes a cheap *lower* bound — no packing can use fewer than $\lceil (\sum_i s_i)/C \rceil$ bins — alongside the *upper* bound given by the feasible solution it just built (the number of bins it actually used). It reports these separately through `get_lb()` / `get_ub()`, and returns `kOK` when they happen to coincide (the greedy solution is then provably optimal) or `kLowPrecision` otherwise (feasible, but optimality not proven):

```
// FirstFitBPSolver.h
#include "Solver.h"
#include "BinPackingBlock.h"

namespace SMSpp_di_unipi_it {

class FirstFitBPSolver : public Solver
{
 public:
  void set_Block( Block * block ) override {
   f_bp = dynamic_cast< BinPackingBlock * >( block );
   if( block && ! f_bp )
    throw( std::invalid_argument( "FirstFitBPSolver: not a BinPackingBlock" ) );
   Solver::set_Block( block );
  }

  int compute( bool /* changedvars */ = true ) override {
   // consume any pending Modifications: this heuristic just restarts,
   // so we clear the list and recompute from scratch
   v_mod.clear();

   const Index  n = f_bp->get_NItems();
   const double C = f_bp->get_Capacity();
   std::vector< double > load( n , 0.0 );      // current load of each bin
   std::vector< Index >  bin_of( n , 0 );      // bin chosen for each item

   double total = 0;                           // sum of all item sizes
   Index  nbins = 0;                           // number of bins opened

   for( Index i = 0 ; i < n ; ++i ) {
    const double s = f_bp->get_Size( i );
    if( s > C )  return( kInfeasible );         // item bigger than a bin
    total += s;
    Index j = 0;
    while( load[ j ] + s > C + 1e-9 )  ++j;     // first bin with room
    load[ j ] += s;
    bin_of[ i ] = j;
    if( j + 1 > nbins )  nbins = j + 1;         // a new bin was opened
   }

   // deposit the solution into the Block (compact + abstract; §B.6)
   f_bp->set_assignment( bin_of );

   // upper bound: the value of the feasible solution just built
   f_ub = OFValue( nbins );
   // lower bound: no packing can use fewer than ceil( total / C ) bins
```

```
   f_lb = std::ceil( total / C );

   // if the two bounds coincide, the greedy solution is provably optimal
   return( f_lb == f_ub ? kOK : kLowPrecision );
  }

  // report the two bounds separately (BinPackingBlock is a minimization)
  OFValue get_lb( void ) override { return( f_lb ); }
  OFValue get_ub( void ) override { return( f_ub ); }

  // a one-shot heuristic keeps no warm-startable state
  // (get_State() inherits the nullptr default)

 private:
  BinPackingBlock * f_bp = nullptr;
  OFValue f_lb = -Inf< OFValue >();   // valid lower bound (none yet)
  OFValue f_ub =  Inf< OFValue >();   // best feasible value (none yet)

  SMSpp_insert_in_factory_h;     // so Solver::new_Solver("FirstFitBPSolver") works
};

}  // namespace
```

```
// FirstFitBPSolver.cpp
SMSpp_insert_in_factory_cpp_0( FirstFitBPSolver );  // ctor takes 0 args
```

The `set_assignment` helper the solver calls is the `Block`-side method declared in §A.2. It is worth seeing in full, because it makes a point that is easy to gloss over: the *compact* (physical) form of a solution and the *abstract* form generally have very different shapes, and translating between them is real work the `:Block` must do. Here the compact form is a single n-vector (`v_bin_of[i]` = the bin holding item i); the abstract form is the $n \times m$ matrix of `x` binaries plus the m-vector of `y` binaries. `set_assignment` stores the compact form and then *derives* the abstract one from it — in particular it has to infer which bins are used (the `y[j]`s) from the per-item assignment, which is not given explicitly in the compact representation:

```
// BinPackingBlock.cpp
void BinPackingBlock::set_assignment( const std::vector< Index > & bin_of )
{
 v_bin_of = bin_of;                          // store the compact solution

 // reflect it into the abstract Variables, if they have been generated
 if( v_y.empty() )  return;                   // no abstract face to fill

 for( auto & row : v_x )                      // start from all-zero ...
  for( auto & xij : row )  xij.set_value( 0 );
 for( auto & yj : v_y )    yj.set_value( 0 );

 for( Index i = 0 ; i < f_n ; ++i ) {
  const Index j = v_bin_of[ i ];
  v_x[ i ][ j ].set_value( 1 );               // item i is placed in bin j
  v_y[ j ].set_value( 1 );                    // hence bin j counts as used
 }
}
```

A few points to read off:

- `set_Block` downcasts to the concrete `:Block` the solver understands and rejects anything else — the standard guard of a *specialised* `:Solver`.
- `compute()` returns `kInfeasible` with a certificate-by-construction (an item larger than any bin); otherwise it returns `kOK` when the cheap lower bound matches the bins actually used — the greedy solution is then *provably* optimal — and `kLowPrecision` when it does not, honestly signalling a feasible but not proven-optimal solution (§6.3). The two bounds are exposed separately through the inherited `get_lb()` / `get_ub()`.

- the solution is written *into the* `Block` through `set_assignment`, which keeps both the compact physical solution and the abstract `x` / `y` values, so the user can read it back through the `Block` in either form, as for any `:Solver` (§6.9).
- `SMSpp_insert_in_factory_cpp_0` (zero-argument constructor) registers the `:Solver` so it can be named in a `BlockSolverConfig` (Chapter 11) — and so it is subject to the linker caveat of §18.2.

This heuristic is intentionally minimal: no parameters, no dual information, no reoptimization, no async state. It is a complete, attachable `:Solver` nonetheless — enough to make a `BinPackingBlock` solvable, which (as Chapter 2, §2.4 noted) is the precondition for testing the `:Block` at all.

Appendix C — Writing a new :Modification / :Change

The last appendix covers the two notification/edit objects a new :Block author may need to define: a :Modification (always, if the :Block supports any change at all) and, optionally, a :Change (for serialisable / undoable / transmissible edits). The running example continues with BinPackingBlock (Appendix A).

C.1 Why a :Block needs its own :Modifications

A :Block that exposes any chg_*() mutator must, when the change happens and a :Solver is listening, tell that :Solver *what* changed (Chapter 8). The base Modification hierarchy already covers the generic cases — NBModification for a wholesale load() (§4.6), the abstract-stream C05FunctionMod / VariableMod / RowConstraintMod for changes made through the abstract face — but a *physical* change to a :Block's own data that a specialised :Solver should react to needs a :Block-specific :Modification carrying the description of that change.

C.2 Naming and structure conventions

SMS++ follows consistent naming for :Block-specific Modifications (Chapter 8, §8.2):

- <Block>Mod — the base class for that :Block's physical Modifications (e.g. MCFBlockMod, BinaryKnapsackBlockMod);
- <Block>RngdMod — a *range*-based change: a contiguous [start, stop) interval of the data changed;
- <Block>SbstMod — a *subset*-based change: an arbitrary set of indices changed;
- <Block>ModAdd / <Block>ModRmv — additions / removals, where the :Block supports dynamic objects.

A physical :Modification derives from Modification (not from AModification); its concerns_Block() is false, because the change in the :Block has already happened by the time the object is constructed (§8.3). It carries just enough to let a :Solver identify what changed — for BinPackingBlock, the index (or range, or subset) of the items whose size was modified:

```
// BinPackingBlock.h (alongside the Block)
namespace SMSpp_di_unipi_it {

class BinPackingBlockMod : public Modification
{
 public:
  BinPackingBlockMod( BinPackingBlock * blk , Block::Index item )
   : f_block( blk ) , f_item( item ) {}

  Block * get_Block( void ) const override { return( f_block ); }
  Block::Index item( void ) const { return( f_item ); }

 private:
  BinPackingBlock * f_block;
  Block::Index      f_item;     // which item's size changed
};
```

```
} // namespace
```

A `RngdMod` / `SbstMod` would carry a `Range` / `Subset` instead of a single index, mirroring the `Rngd` / `Sbst` shapes of the methods factory (§15.3). This is the object `BinPackingBlock::chg_size` issued in §A.5.

C.3 Registering with the factory

A plain `:Modification` does not generally need to be in a factory (it is constructed by the `:Block`, dispatched, consumed, and deallocated when the last holder releases its `shared_ptr`, §8.1). It is `:Change` and the serialisable objects that need the class-name factory. A `:Modification` that *does* need to be reconstructed from netCDF would use the same `SMSpp_insert_in_factory_*` macros as everything else (Chapter 18).

C.4 Defining a `:Change` (optional)

> **Status — beta.** The `Change` mechanism (Chapter 16) is in `develop` and its interface may change. Add a `:Change` family only when the use case calls for serialisable, transmissible, or undoable edits (a distributed solver, a move-based search).

A `:Change` for `BinPackingBlock` would represent a *prospective* size change — carrying the new size(s) so it can be `apply()`-ed to any compatible `BinPackingBlock`, serialised to netCDF, and (where the change is invertible) produce its UndoChange:

```
class BinPackingBlockRngdChange : public Change
{
 public:
  // ... constructor storing the affected range and the new sizes ...

  Change * apply( Block * block , bool doUndo = false ,
                  ModParam issueMod = eNoBlck ,
                  ModParam issueAMod = eNoBlck ) override
  {
   auto * bp = dynamic_cast< BinPackingBlock * >( block );
   // if doUndo, first capture the current sizes into an inverse Change
   Change * undo = doUndo ? /* a Change restoring the old sizes */ : nullptr;
   // apply the new sizes through the Block's own mutator
   for( /* each affected item i with new size s_i */ )
    bp->chg_size( s_i , i , issueMod , issueAMod );
   return( undo );
  }

  void serialize( netCDF::NcGroup & group ) const override { /* ... */ }
  void deserialize( const netCDF::NcGroup & group ) override { /* ... */ }

 private:
  SMSpp_insert_in_factory_h;     // Change::new_Change("...") and netCDF
};
```

```
SMSpp_insert_in_factory_cpp_0( BinPackingBlockRngdChange );
```

The essential points (Chapter 16): `apply()` enacts the change through the `:Block`'s own `chg_*()` interface (so the `:Change` can support at most what the `:Block` supports), optionally returns an UndoChange when `doUndo` is set, and the class is factory- and netCDF-registered so it can be reconstructed from a serialised description on another process or at a later time. A `:Change` that cannot cheaply build its inverse simply declines to support undo for those cases (§16.2).

C.5 Checklist

- Define a `<Block>Mod` (and `Rngd` / `Sbst` variants as needed) deriving from `Modification`, carrying the description of a physical change, with `concerns_Block() == false` (§C.2). Issue it from the corresponding `chg_*()` mutator (§A.5).
- Rely on the framework's existing abstract-stream `Modifications` (`C05FunctionMod`, `VariableMod`, ...) for changes made through the abstract face; intercept and mirror them in `add_Modification()` (§A.6).
- Add a `:Change` family **only** if serialisable / transmissible / undoable edits are needed; register it in the factory and implement `apply()` (with optional undo), `serialize()`, `deserialize()`. **Status — beta.**

This is the end of the SMS++ User Manual, version 0.1, for SMS++ 0.7.0.

Appendix D — Concept-to-Block map

The manual threads three running `Block`s through the whole of Part II and Part III, using whichever one shows a given concept most clearly: `MCFBlock` (a leaf `Block` with a wrapper-style specialised `Solver`), `BinaryKnapsackBlock` (a leaf `Block` with a *native* specialised `Solver`), and `CapacitatedFacilityLocationBlock` (a non-leaf `Block` with four formulations, a non-trivial `R3Block`, and a hidden `BendersBFunction`). This appendix is the cross-index: for each concept, it names the `Block` (or other class) that carries the worked example and the section(s) where the example is developed. It is a navigation aid, not a substitute for the chapters themselves.

Concept	Illustrated mainly by	Section(s)
`Block` tree, the identity-is-address rule	`MCFBlock`	§4
`Variable` / `Constraint` / `Objective`	`BinaryKnapsackBlock`	§5
Specialised `Solver` on a leaf `Block` (wrapper)	`MCFSolver`	§6
Specialised `Solver` on a leaf `Block` (native)	`DPBinaryKnapsackSolver`	§6, R2
`is_feasible` / `is_dual_feasible` / `is_optimal`	`MCFBlock`	§6, §7
Physical vs abstract representation	`MCFBlock`, `BinaryKnapsackBlock`	§7
`Modification`, abstract `Modification`, the Janus discipline	`MCFBlock`, `BinaryKnapsackBlock`	§8
`Solution` (`Block`-specific, physical and abstract)	`MCFSolution`, `BinaryKnapsackSolution`	§9
`R3Block` (non-trivial reformulation)	CFL → `MCFBlock` flow relaxation	§10, R3
`Configuration` tree, `[C/O/R]BlockConfig`	CFL formulations	§11
Sub-`Block` composition and recursive `Modification` flow	CFL / KskForm with `BinaryKnapsackBlock` sub-`Block`s	§12, R4
The `Function` family (C05/C15, linear / quadratic / polyhedral)	`MCFBlock`, `BinaryKnapsackBlock`	§13
`LagBFunction` wrapping a `Block`	CFL / KskForm	§14, R4
`BendersBFunction` wrapping a `Block`	CFL / BenForm	§14, R5
The methods factory	`BinaryKnapsackBlock::chg_weights`	§15, R2
`Change` (*beta*)	`BinaryKnapsackBlockChange`	§16, R2
Parallel / asynchronous computation (`lock`, `compute_async`, `State`)	CFL / KskForm with a parallel `BundleSolver`	§17
Factories and netCDF [de]serialisation	all three `Block`s	§18, R3
`LagrangianDualSolver`	CFL / KskForm	R4
`PrimalProximalHeur`	CFL / KskForm	R4
Benders cuts via a user-cut callback	CFL / BenForm	R5

Concept	Illustrated mainly by	Section(s)
Writing a new `:Block`, `:Solver`, `:Modification` / `:Change` from scratch	`BinPackingBlock` (pedagogical)	Appendices A, B, C

Read the table in either direction: a reader following a concept can jump to the `Block` that exercises it, and a reader interested in one of the three `Block`s can collect, by scanning the middle column, every place that `Block` appears in the manual.

Glossary

A compact, alphabetical reference for the SMS++ terms used in this manual. Each entry gives the working definition and a pointer to the chapter where the term is developed; class names link, where useful, to the chapter rather than to Doxygen, which remains the authoritative API reference.

Abstract representation. The explicit `Variable` / `Constraint` / `Objective` description of a `:Block`, built on demand by the `generate_abstract_*()` methods. General-purpose `Solver`s read it; specialised ones usually do not. The counterpart of the *physical representation* (Chapter 7).

AModification. The abstract base for `Modification`s that describe a change made through the *abstract* face of a `:Block` (a `Variable`, `Constraint`, `Function` ...). Its `concerns_Block()` is `true` until the `:Block` has reflected the change into its physical data (Chapter 8).

BendersBFunction. A `Function` that is *also* a `Block`, computing the value function of an inner "base" `Block` parameterised by an affine map `M(y) = A y + b` applied to the right/left-hand sides of some of its `RowConstraint`s. Each evaluation solves the inner `Block`; its linearizations are Benders optimality or feasibility cuts (Chapter 14).

Block. The cornerstone abstraction: a nested unit carrying the *semantics* of a mathematical sub-problem and, on demand, an abstract representation of it. A `Block` has a father, any number of static sub-`Block`s, `Variable` / `Constraint` / `Objective` groups, and any number of registered `Solver`s (Chapter 4).

BlockConfig. A `Configuration` that configures the *generation* of a `:Block`'s abstract representation; the `[C/O/R]BlockConfig` family selects which constraints (**C**), objective (**O**) and whether the configuration recurses into sub-`Block`s (**R**) are built (Chapter 11).

BlockSolverConfig. A `Configuration` that says which `Solver`s are attached to a `Block` and with which parameters; switching solver is "one line" of `BlockSolverConfig` (Chapters 6, 11).

C05Function / C15Function. Refinements of `Function` exposing, respectively, first-order information (values and linearizations, not necessarily from continuous gradients) and second-order information (partial Hessians) (Chapter 13).

CDASolver. "Convex Duality Aware" `Solver`: one that, besides a primal solution, produces (possibly several) dual solutions / valid bounds. `MCFSolver` and the `MILPSolver` family are `CDASolver`s (Chapter 6).

Change. *Status — beta.* A serialisable, transmissible, often invertible description of an edit to a `:Block`; unlike a `Modification` (a notification of a change that has already happened), a `Change` carries the data needed to *apply* the edit to a fresh `:Block` and to undo it (Chapter 16).

ColVariable. The concrete `Variable` representing a single real or integer decision variable, with a type (`kContinuous`, `kBinary`, `kInteger`, ...), a value, and fix/unfix state (Chapter 5).

compute(). The method, inherited from `ThinComputeInterface`, by which a `Solver` (or a `Function`, or any computable object) performs its computation and returns a `sol_type` status (Chapter 6).

concerns_Block(). A flag on a `Modification` that is `true` while a change made through the abstract face still has to be folded back into the physical data; the `:Block`'s `add_Modification()` resets it once done (Chapter 8).

Configuration. A tree-shaped object mirroring the `Block` tree that parameterises generation, solving, and computation; `SimpleConfiguration<T>`, the `[C/O/R]BlockConfig` family, `BlockSolverConfig`, and `ComputeConfig` are its concrete forms (Chapter 11).

Constraint. A primitive living inside a `Block`. `RowConstraint` has a real-valued left/right-hand side; `FRowConstraint` carries a `Function`; `OneVarConstraint` (e.g. `LBOConstraint`) bounds a single `ColVariable` (Chapter

5).

Dynamic vs static. Static `Variable` / `Constraint` groups are fixed in number and may live in vectors or `boost::multi_arrays`; dynamic ones may be added and removed at run time and must live in `std::lists`, because a SMS++ object's identity is its memory address (Chapters 4, 5).

Factory. A class-name (`std::string`) → object map allowing a `:Block` / `:Solver` / `:Configuration` / `:Modification` / `:Change` / `:Function` to be created by name and reconstructed from netCDF; registered with the `SMSpp_insert_in_factory_*` macros (Chapter 18). Distinct from the *methods factory* (below).

FRealObjective. The concrete `Objective` carrying a `Function` to be minimised or maximised (Chapter 5).

Function. A real-valued (extended-real) function of `ColVariables` that must be `compute()`-d; the family runs `Function` → `C05Function` → `C15Function` with leaf implementations such as `LinearFunction`, `DQuadFunction`, and `PolyhedralFunction` (Chapter 13).

Janus discipline. The rule that a `:Block`'s two faces (physical and abstract) be kept consistent: a `chg_*()` mutator updates the physical data and mirrors the change into the abstract one, while a change arriving through the abstract face is intercepted by `add_Modification()` and folded back into the physical data (Chapter 8).

LagBFunction. A `Function` that is *also* a `Block`, computing the Lagrangian (dual) function of an inner "base" `Block` with respect to a set of linear terms; its linearizations correspond to (approximately) optimal inner solutions, which is what lets a bundle method optimise the dual (Chapter 14).

Leaf Block. A `Block` with no sub-`Blocks` (`MCFBlock`, `BinaryKnapsackBlock`); the opposite of a *non-leaf* (composite) `Block` such as `CapacitatedFacilityLocationBlock` (Chapters 4, 12).

Linearization. A first-order piece of information produced by a `C05Function` at an evaluation point — for a convex function, a subgradient defining a supporting hyperplane; pools of linearizations drive bundle methods and Benders/Lagrangian schemes (Chapters 13, 14).

Modification. A lazily-propagated notification that something changed in a `Block`; the framework appends it to the pending list of every `Solver` registered with the `Block` or any ancestor, and the `Solver` consumes it when it reacts. `NBModification` is the wholesale "reload" notification (Chapter 8).

netCDF. The hierarchical binary format SMS++ uses for serialisation; nested `Blocks` map to nested netCDF groups, inspectable with `ncdump` / `ncview` / Panoply (Chapter 18).

Objective. The primitive expressing what a `Block` optimises; sub-`Block` objectives are summed into the father's (Chapter 5).

Physical representation. The problem data of a `:Block` in its natural, semantic form (a graph and costs for `MCFBlock`; weights, profits, capacity for `BinaryKnapsackBlock`). Dictated by the problem, not chosen; read directly by specialised `Solvers`. The counterpart of the *abstract representation* (Chapter 7).

PolyhedralFunction. A `C05Function` that *is* a polyhedral (piecewise-linear) function, useful as the recipient of externally-generated cuts (Chapter 13).

R3Block. A "Reformulation, Relaxation, or Restriction" `Block` produced by another `Block` to represent an algorithmic transformation of itself, kept in sync (where required) by the `map_*` methods and `UpdateSolver`. CFL's flow relaxation as an `MCFBlock` is the running example (Chapter 10).

Solution. A `Block`-specific (not `Solver`-specific) object that can read a solution from / write it into a `Block`, serialise it, and be linearly combined; abstract forms (`ColVariableSolution`, `RowConstraintSolution`) and physical forms (`MCFSolution`, ...) coexist (Chapter 9).

Solver. Any algorithm able to `compute()` on a `Block`; specialised `Solvers` exploit a specific `:Block`'s structure, general-purpose ones operate on the abstract representation. Many `Solvers` may be attached to one `Block` (Chapter 6).

sol_type. The status enum returned by `compute()`: `kOK`, `kInfeasible`, `kUnbounded`, `kLowPrecision`, `kStopTime`, `kStopIter`, values ≥ `kError`, etc. (Chapter 6).

State. A `Solver`'s checkpointable internal state, used for warm-starting and reoptimization across `compute()` calls (Chapter 17).

Sub-Block. A statically-nested `Block` inside another; data is split between a `Block` and its sub-`Blocks`, and `Modifications` flow upward to the `Solvers` of every ancestor (Chapter 12).

ThinComputeInterface / ThinVarDepInterface. The two lightweight mix-in interfaces underlying the framework: the first gives an object a `compute()` and a parameter machinery (so `Block`, `Solver`, `Function` are all computable); the second gives an object the notion of being "active" in a set of `Variable`s (so `Constraint` and `Function` track their variables) (Chapters 5, 6, 13).

UpdateSolver. A pseudo-`Solver` whose job is not to optimise but to forward `Modification`s from one `Block` to another, the canonical glue that keeps an `R3Block` in sync with its origin (Chapter 10).

Variable. The primitive representing a decision variable; `ColVariable` is the concrete single-variable form. A `Function` never copies a `Variable`, it references it by address (Chapter 5).

Bibliography

This bibliography is deliberately small. The manual is a companion to the SMS++ source code, Doxygen, and Wiki, which remain the authoritative references for the software itself; the entries below cover the reference paper for the framework and the foundational methodological works for the decomposition and nonsmooth-optimization algorithms that SMS++ implements and that are mentioned in the text.

The framework

1. A. Frangioni, L. Galli, N. Iardella, R. Durbano Lobato. *SMS++: a Software Framework for Structured Optimization.* Reference paper for the SMS++ project, Dipartimento di Informatica, Università di Pisa.
2. The SMS++ project: source repository https://gitlab.com/smspp/smspp-project, API reference (Doxygen) https://smspp.gitlab.io/smspp-project/, and project Wiki https://gitlab.com/smspp/smspp-project/-/wikis/home. Version covered by this manual: 0.7.0 (October 2026).

Decomposition and nonsmooth optimization

3. J. F. Benders. *Partitioning procedures for solving mixed-variables programming problems.* Numerische Mathematik **4** (1962), 238–252. The original Benders decomposition, the scheme underlying `BendersBFunction` (Chapter 14) and Recipe R5.
4. G. B. Dantzig, P. Wolfe. *Decomposition principle for linear programs.* Operations Research **8**(1) (1960), 101–111. The Dantzig–Wolfe reformulation dual to the Lagrangian view used in Chapter 14 and Recipes R3–R4.
5. A. Frangioni. *About Lagrangian Methods in Integer Optimization.* Annals of Operations Research **139** (2005), 163–193. Background for the Lagrangian dual computed by `LagBFunction` and `LagrangianDualSolver` (Chapter 14, Recipe R4).
6. A. Frangioni. *Standard Bundle Methods: Untrusted Models and Duality.* In: A. M. Bagirov, M. Gaudioso, N. Karmitsa, M. M. Mäkelä, S. Taheri (eds.), *Numerical Nonsmooth Optimization*, Springer, 2020, pp. 61–116. The bundle-method theory behind `BundleSolver`, the engine of the Lagrangian dual (Chapters 13–14).
7. M. V. F. Pereira, L. M. V. G. Pinto. *Multi-stage stochastic optimization applied to energy planning.* Mathematical Programming **52** (1991), 359–375. Stochastic Dual Dynamic Programming, the archetype application of the polyhedral / Benders machinery referenced in Chapter 13.

Serialisation

8. R. Rew, G. Davis. *NetCDF: an interface for scientific data access.* IEEE Computer Graphics and Applications **10**(4) (1990), 76–82. The serialisation format used throughout SMS++ (Chapter 18).

Funding acknowledgement

9. **RESILIENT** — *Resilient Energy System Infrastructure Layouts for Industry, E-Fuels and Network Transitions* — funded by the European Union under the Horizon Europe research and innovation programme, grant agreement no. 101069750 (https://resilient-project.github.io). Much of the recent work on the SMS++

`Block` library documented here was carried out in the framework of this project; the acknowledgement follows the form indicated in the umbrella project's `README.md`.